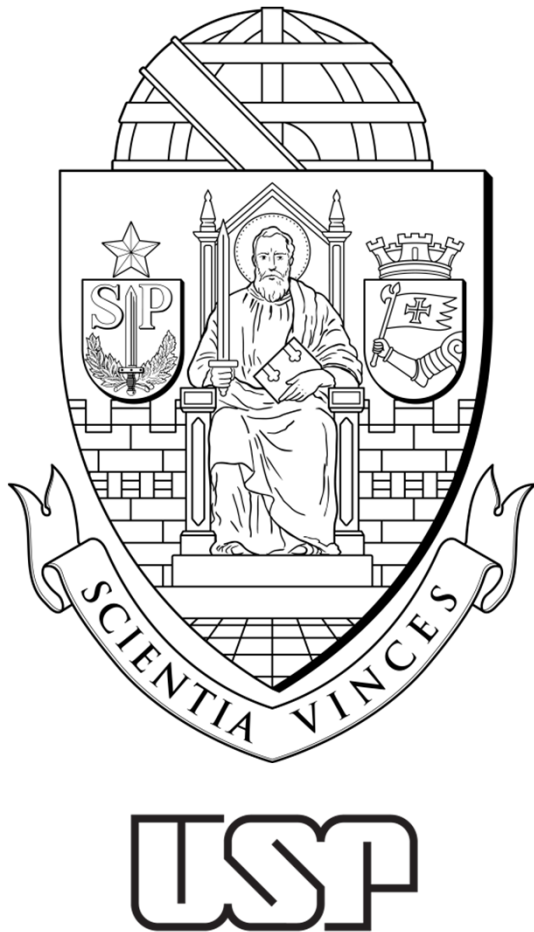


Estratégias de Busca Não Informada



- ❑ Nesta aula são descritas algumas estratégias de busca não informada em espaços de estados
- ❑ Na próxima apresentação serão vistas estratégias de busca informada

Busca

- ❑ Algoritmos inteligentes devem agir de maneira tal que o ambiente passa por uma sequência de estados que maximizam alguma medida de desempenho
- ❑ Dessa forma, o algoritmo adota um objetivo e tenta satisfazê-lo
- ❑ O algoritmo pode decidir o que realizar primeiramente examinando as sequências possíveis de ações que levam a estados com valores conhecidos e então escolher o melhor entre eles
- ❑ O processo de procurar por uma sequência é chamado de **busca**

Busca em Espaço de Estados

- ❑ Um grafo pode ser usado para representar um **espaço de estados** onde:
 - Os nós correspondem a situações de um problema
 - As arestas correspondem a movimentos permitidos ou ações ou passos da solução
 - Um dado problema é solucionado encontrando-se um caminho no grafo
- ❑ Um problema é definido por
 - Um espaço de estados (um grafo)
 - Um estado (nó) inicial
 - Uma condição de término ou critério de parada; estados (nós) terminais são aqueles que satisfazem a condição de término
- ❑ Se não houver custos, há interesse em soluções de caminho mínimo
- ❑ No caso em que custos são adicionados aos movimentos normalmente há interesse em soluções de custo mínimo
 - O custo de uma solução é o custo das arestas ao longo do caminho da solução

Exemplo: Quebra-Cabeça-8

8	1	3
7		2
6	5	4

Estado Inicial

1	2	3
8		4
7	6	5

Estado Final

- ❑ Estados?
- ❑ Operadores?
- ❑ Estado Final: = estado fornecido
- ❑ Custo do caminho?

Exemplo: Quebra-Cabeça-8

8	1	3
7		2
6	5	4

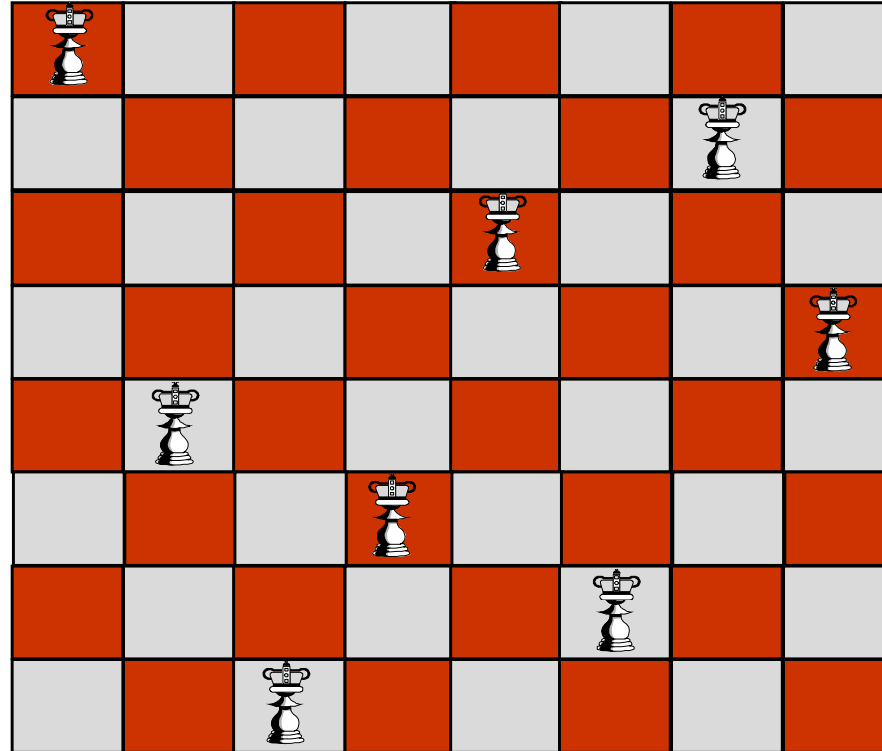
Estado Inicial

1	2	3
8		4
7	6	5

Estado Final

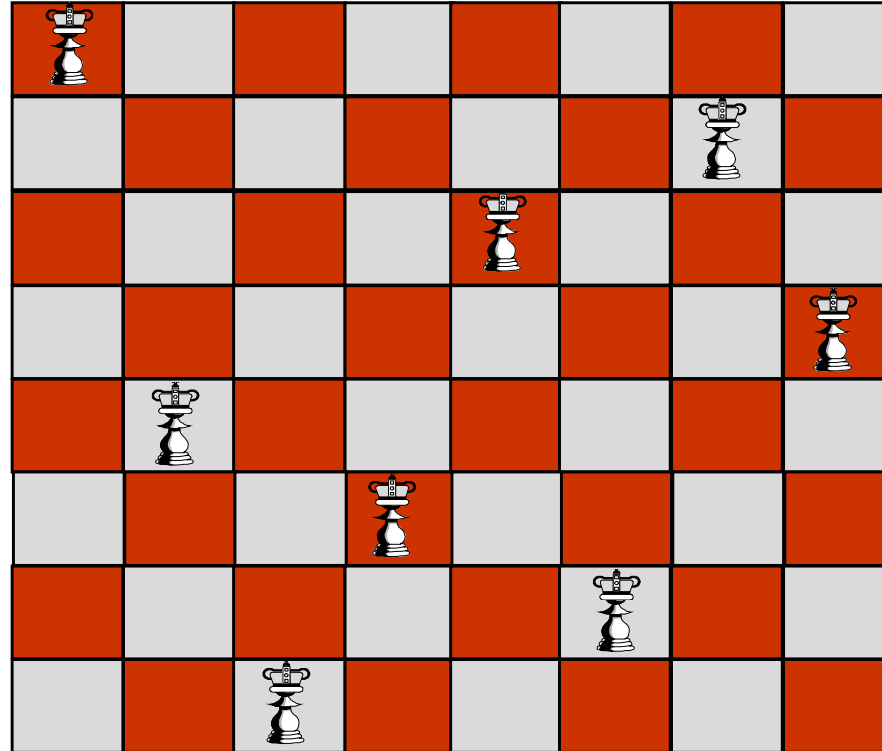
- ❑ Estados: posições inteiras dos quadrados (ignorar posições intermediárias)
- ❑ Operadores: mover se branco à esquerda, direita, em cima, em baixo (ignorar ação de desalojar, etc)
- ❑ Estado Final: = estado fornecido
- ❑ Custo do caminho: 1 por movimento
- ❑ Solução: [right,down,left,left,up,up,right,down]

Exemplo: 8 Rainhas



- ❑ Estados?
- ❑ Operadores?
- ❑ Estado Final?
- ❑ Custo do caminho?

Exemplo: 8 Rainhas



- ❑ Estados: qualquer arranjo de 0 a 8 rainhas no tabuleiro
- ❑ Operadores: adicionar uma rainha a qualquer quadrado
- ❑ Estado Final: 8 rainhas no tabuleiro, sem ataque
- ❑ Custo do caminho: zero (apenas o estado final é interessante)

Exemplo: Missionários e Canibais



Exemplo: Missionários e Canibais

❑ Estados

- um estado é uma sequência ordenada de três números representando o número de missionários, canibais e botes na margem do rio na qual eles iniciam
- Assim o estado inicial é $[3,3,1]$

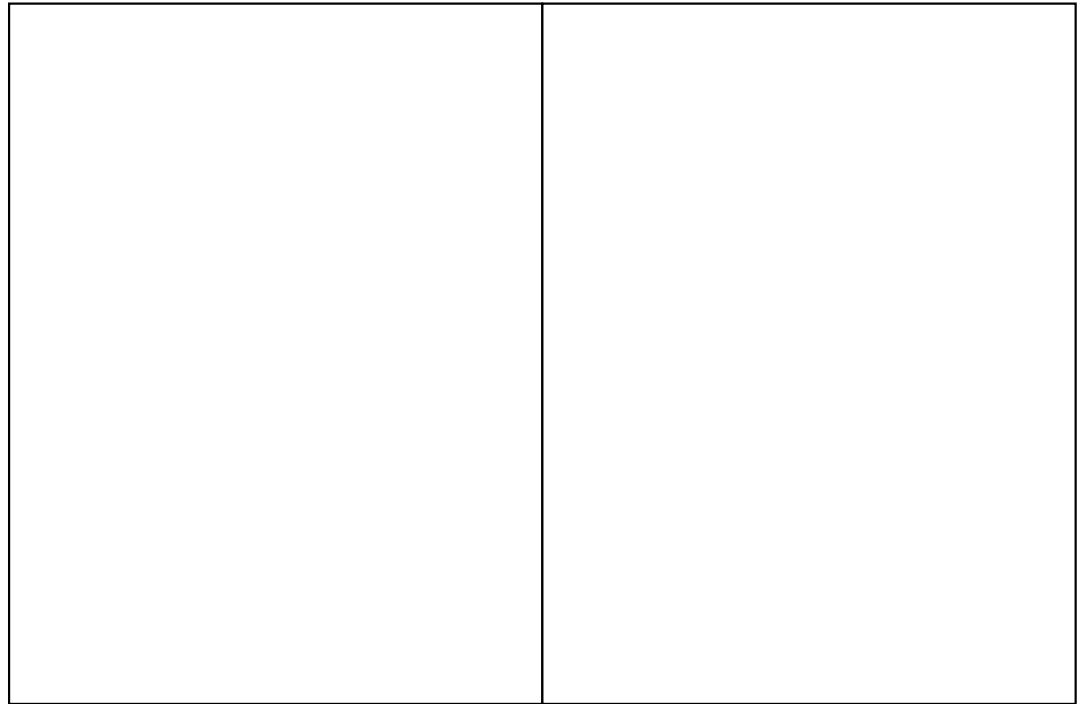
❑ Operadores

- a partir de cada estado os operadores possíveis são tomar um missionário e um canibal, dois missionários ou dois canibais
- Há no máximo 5 operadores, embora alguns estados tenham menos operadores uma vez que deve-se evitar estados inválidos
- Se for necessário distinguir os indivíduos, haveria 27 operadores ao invés de apenas 5

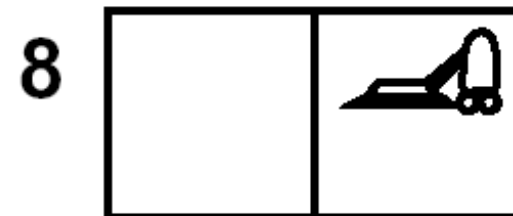
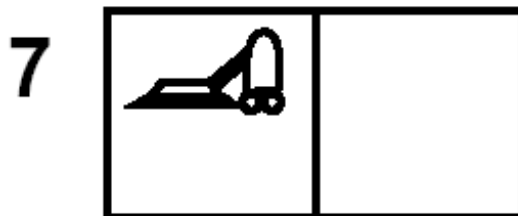
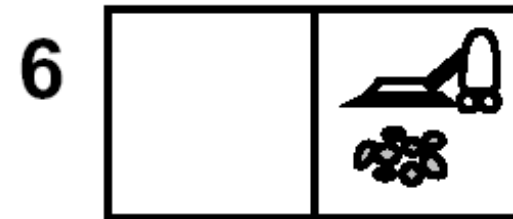
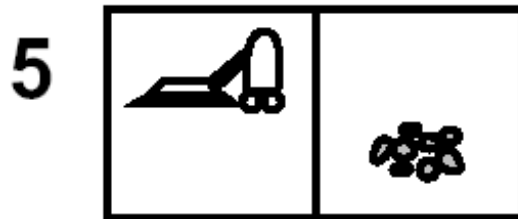
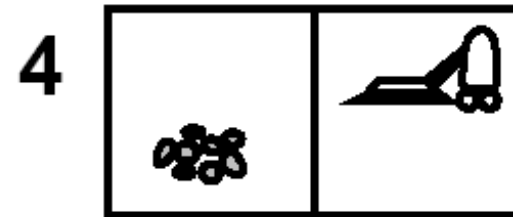
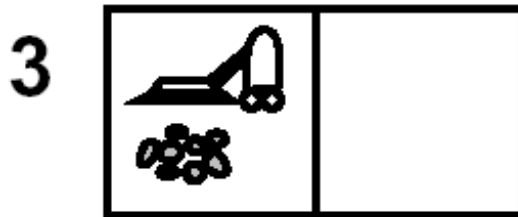
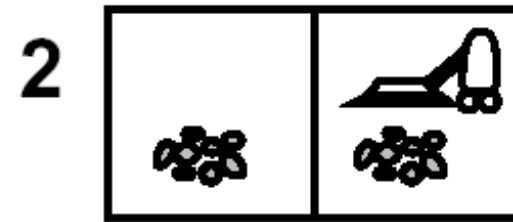
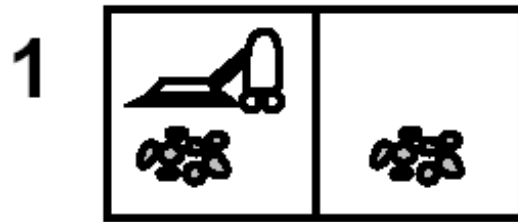
❑ Estado Final: $[0,0,0]$

❑ Custo do caminho: número de travessias

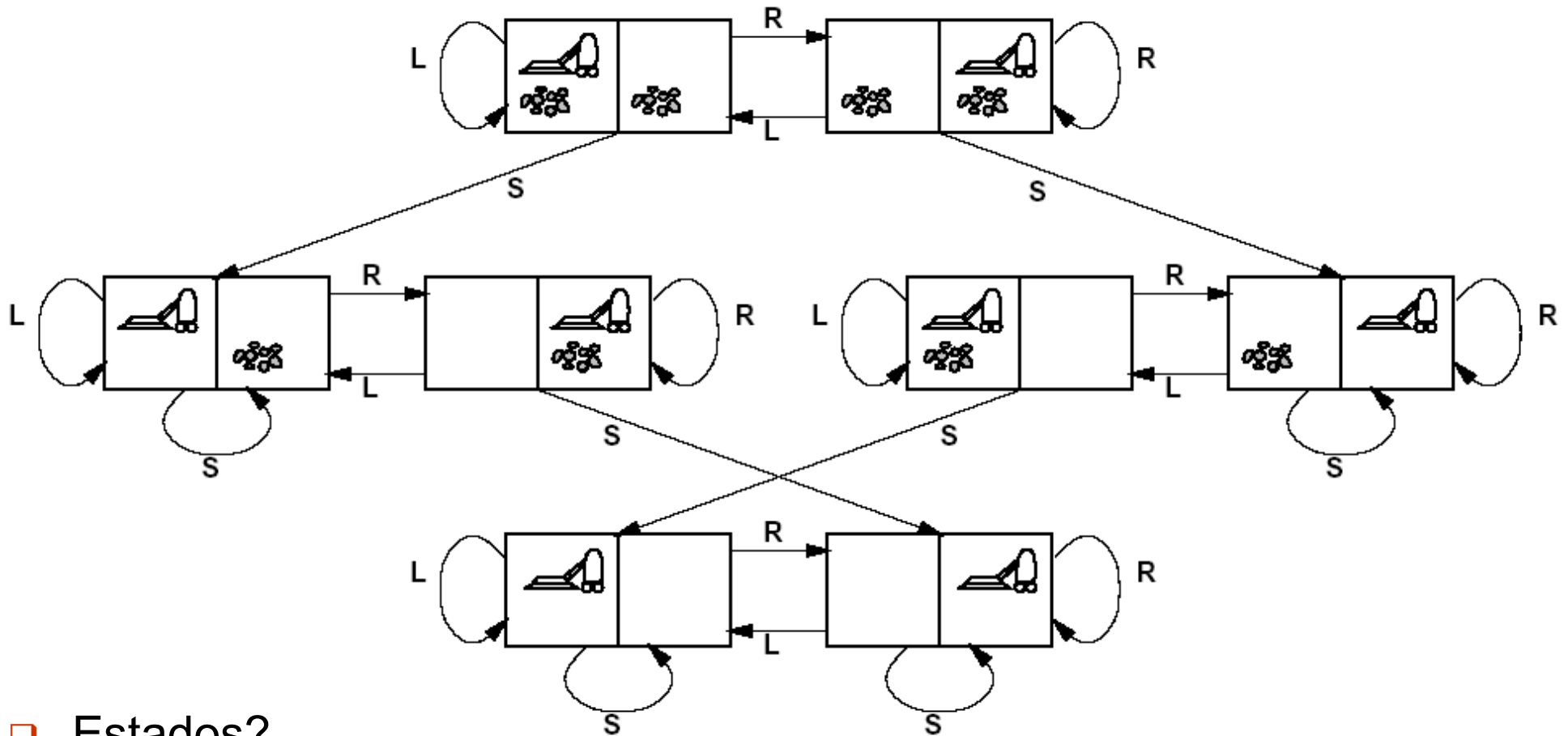
Exemplo: Mundo do Aspirador



Exemplo: Mundo do Aspirador

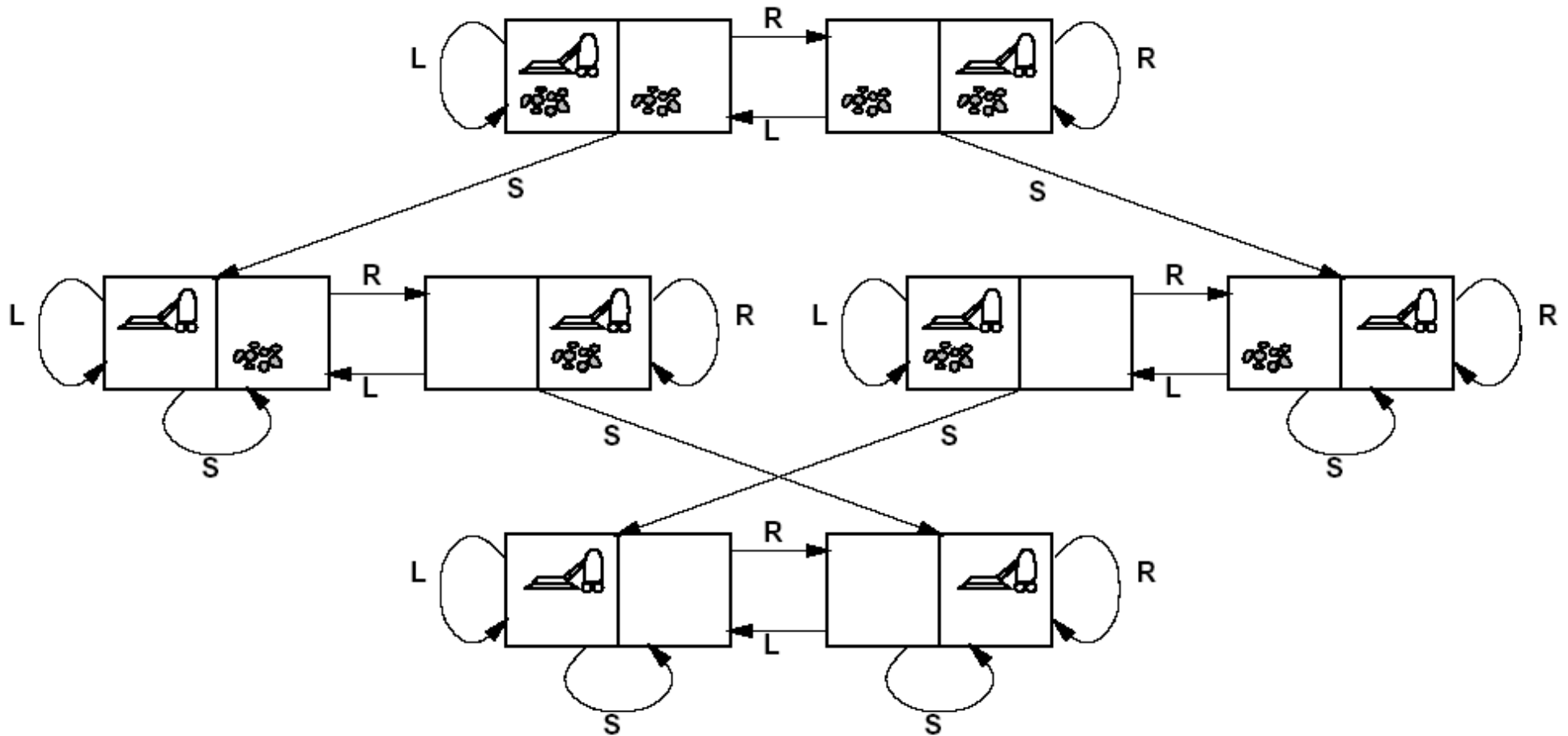


Exemplo: Mundo do Aspirador



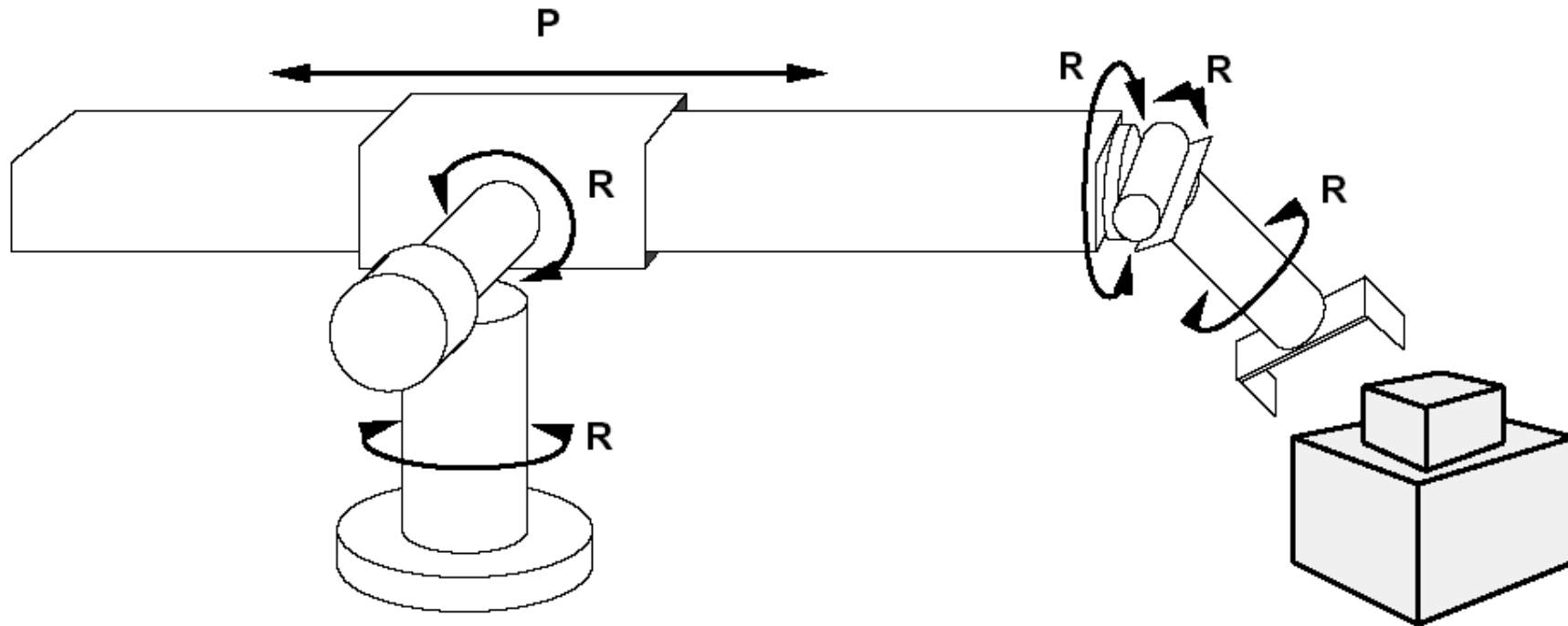
- ❑ Estados?
- ❑ Operadores?
- ❑ Estado Final?
- ❑ Custo do caminho?

Exemplo: Mundo do Aspirador



- ❑ Estados: um dos estados mostrados na figura
- ❑ Operadores: L (esquerda), R (direita), S (sucção)
- ❑ Estado Final: nenhuma sujeira em todos os ambientes
- ❑ Custo do caminho: cada ação tem custo unitário

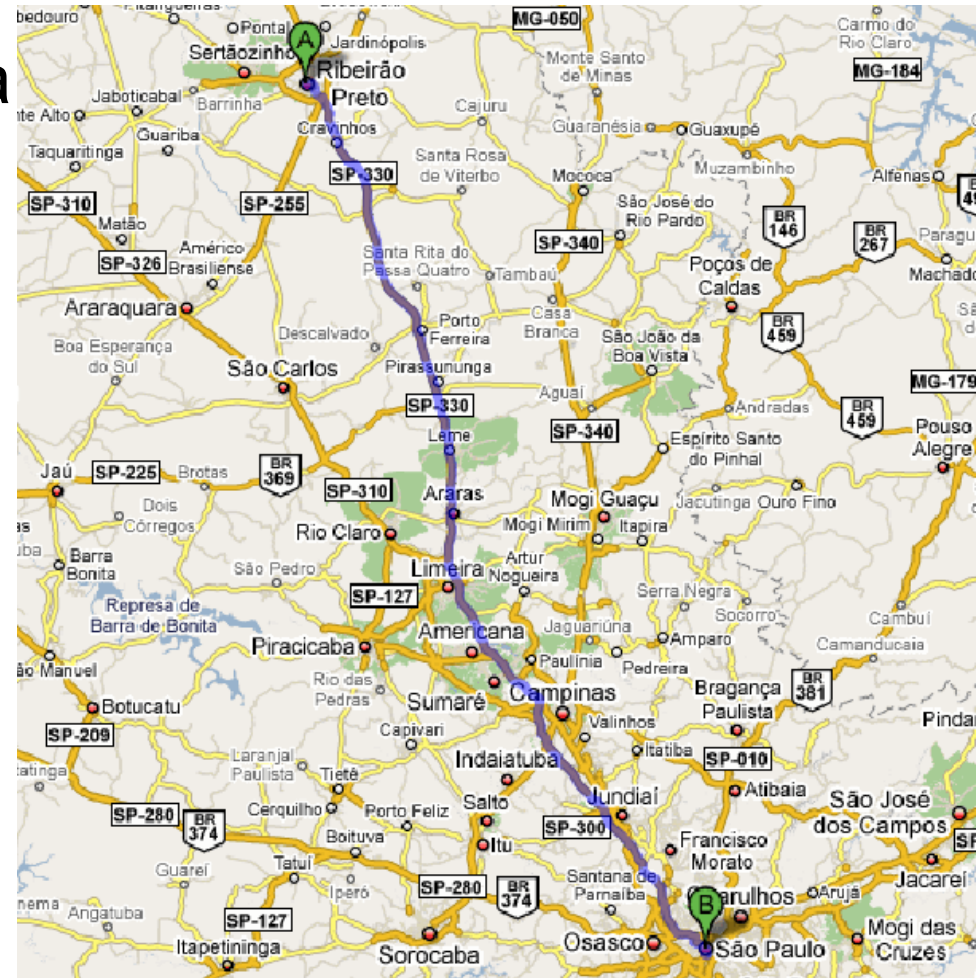
Exemplo: Montagem com Robô



- ❑ Estados:
 - coordenadas de valor real de ângulos das junções do robô
 - partes do objeto a ser montado
- ❑ Operadores: movimentos contínuos das junções do robô
- ❑ Estado Final: montagem completa
- ❑ Custo do caminho: tempo para montagem

Exemplo: Planejamento de Rotas

- ❑ Estado Inicial
 - Cidade em que a jornada inicia
- ❑ Operadores
 - Mover de uma cidade a outra
- ❑ Estado Final
 - Chegou a uma cidade específica?
- ❑ Custo do caminho
 - Distância entre cidade inicial e final



Exemplo: 2D folding

❑ Estado inicial

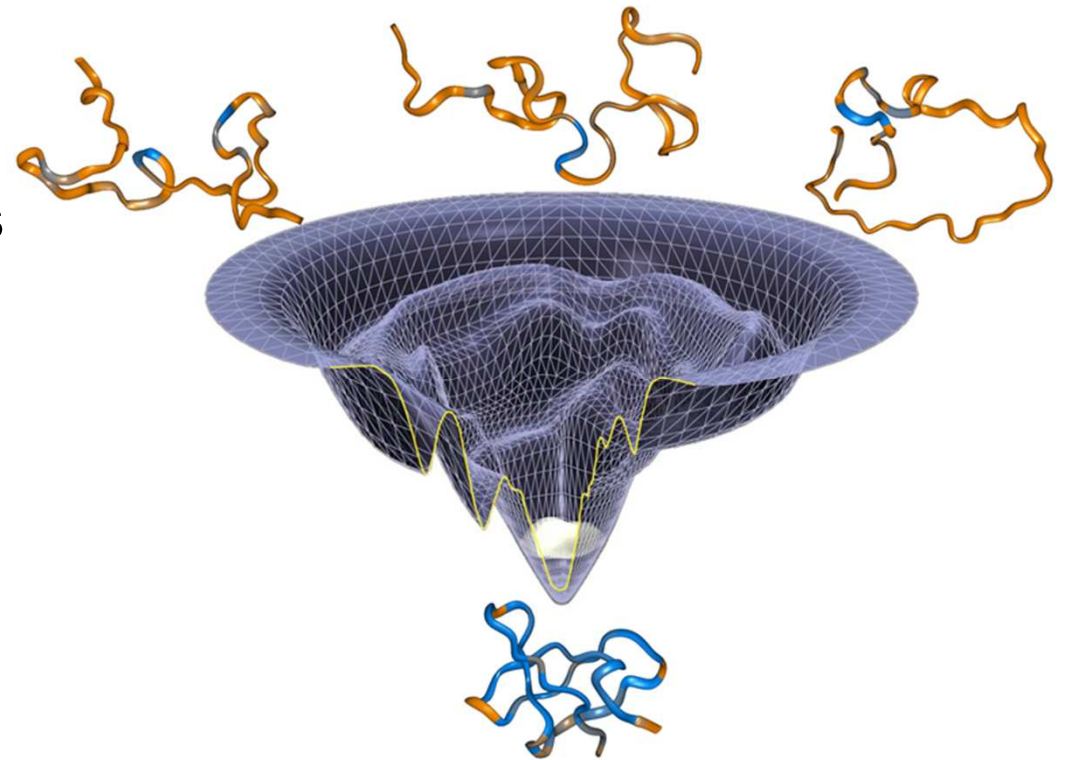
- Uma cadeia não enovelada (*non-folded*) de aminoácidos (identificadores hidrofóbicos ou hidrofílicos)

❑ Operadores

- Diferentes tipos de rotação

❑ Estado final

- Proteína completamente enovelada?
- Energia da proteína abaixo de um limite pré-definido?



Exemplo: Predição de Vias Metabólicas

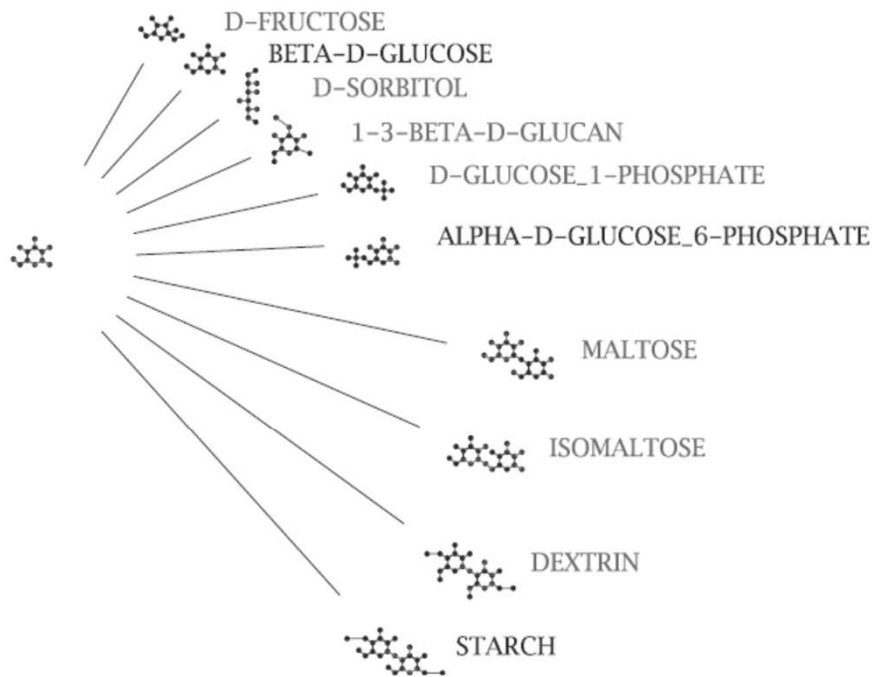


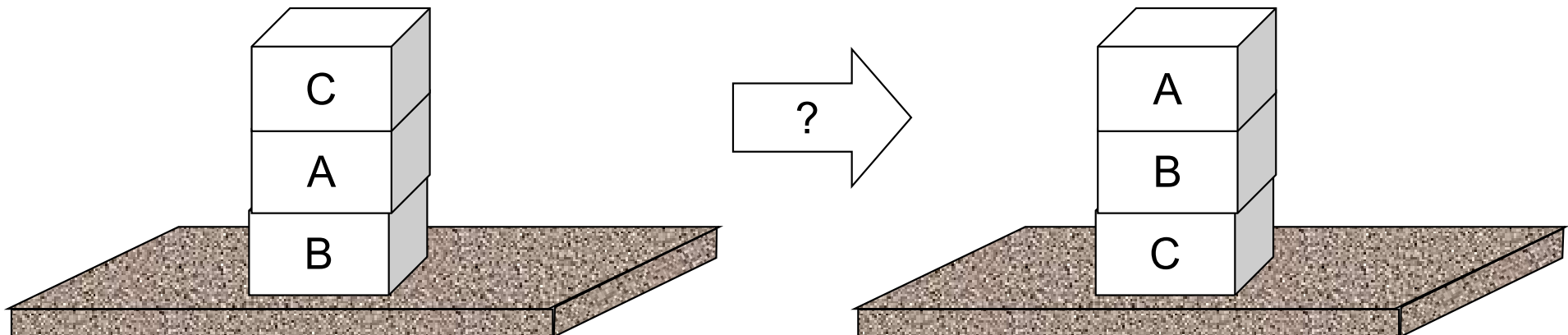
Fig. 2. Known chemical successors of α -D-glucose (adg), denoted as T^{adg} . The figure shows the chemical structure of adg on the left and the successors on the right.

- Predizer uma via metabólica (*metabolic pathway*) usando busca de um caminho a partir de um composto inicial até um composto final por meio de uma série de transições de estados

- McShan DC, Rao S, Shah I, *PathMiner: predicting metabolic pathways by heuristic search*, Bioinformatics, 2003 Sep 1;19(13):1692-8, <http://bioinformatics.oupjournals.org/cgi/reprint/19/13/1692>

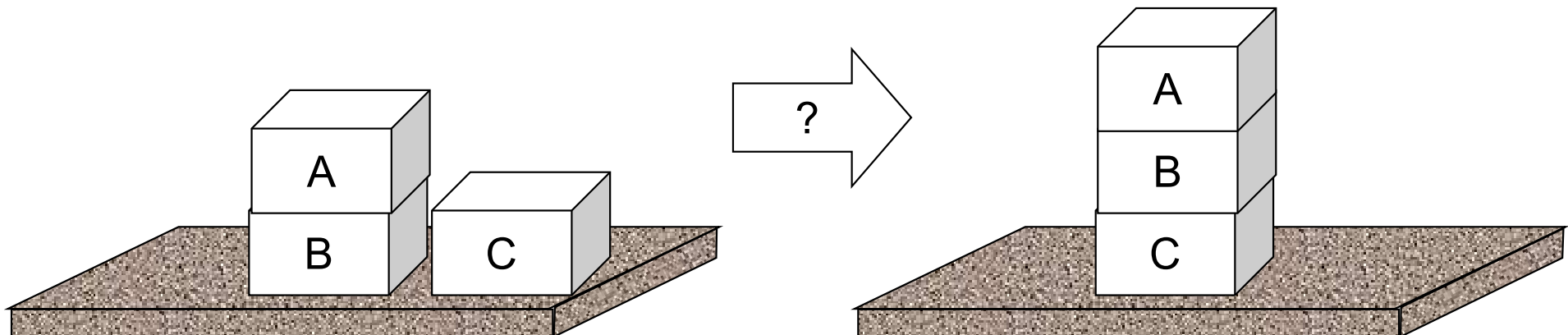
Exemplo: Pilha de Blocos

- ❑ Considere o problema de encontrar um plano (estratégia) para rearranjar uma pilha de blocos como na figura
 - Somente é permitido um movimento por vez
 - Um bloco somente pode ser movido se não há nada em seu topo
 - Um bloco pode ser colocado na mesa ou acima de outro bloco

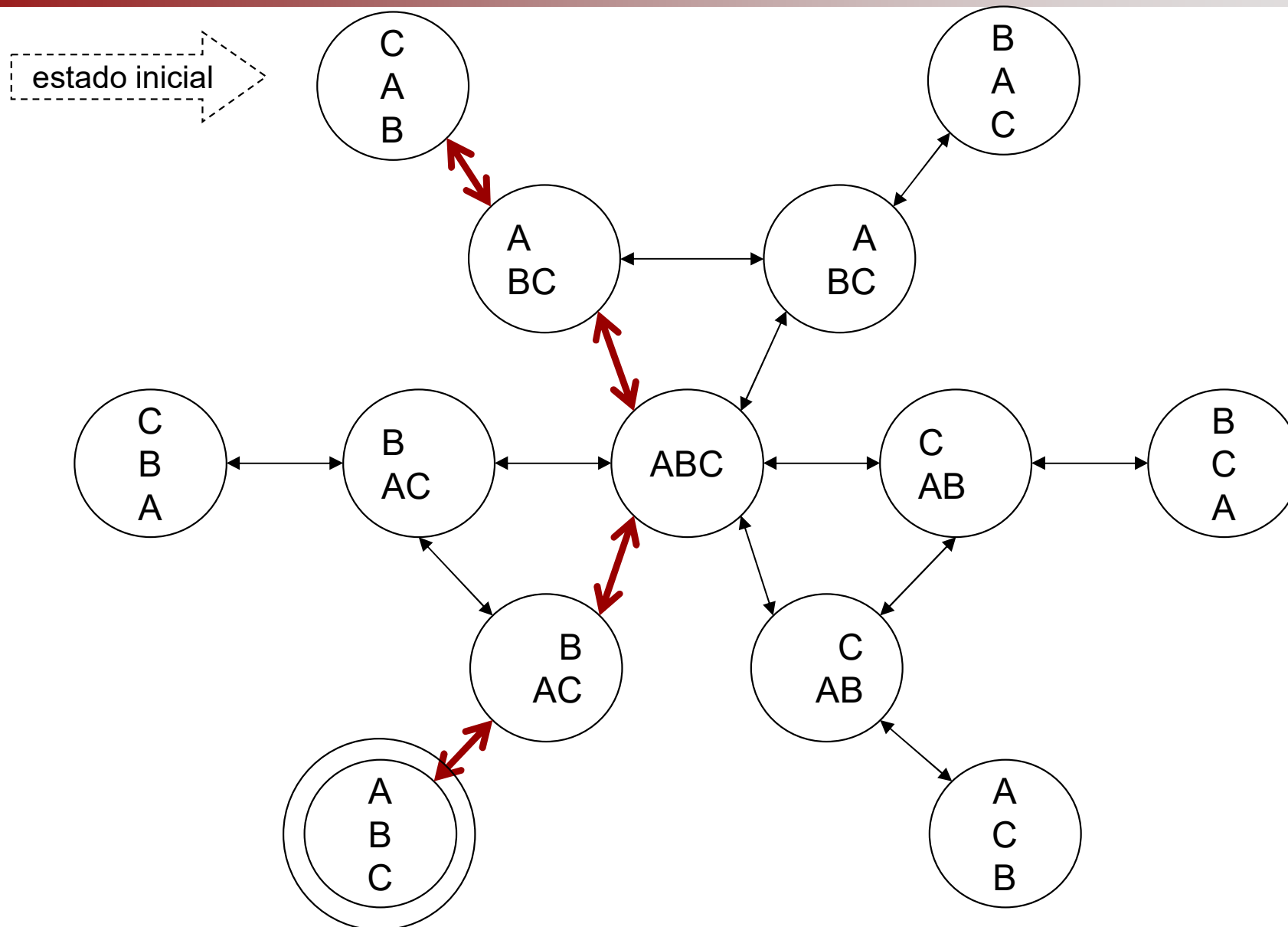


Exemplo: Pilha de Blocos

- ❑ Na situação inicial do problema, há apenas um movimento possível: colocar bloco C na mesa
- ❑ Depois que C foi colocado na mesa, há três alternativas
 - Colocar A na mesa ou
 - Colocar A acima de C ou
 - Colocar C acima de A (movimento que não deve ser considerado pois retorna a uma situação anterior do problema)



Exemplo: Pilha de Blocos



Considerações sobre Busca

- ❑ O que está sendo procurado?
 - O que é interessante: a solução ou o caminho para a solução?
- ❑ Completude
 - O algoritmo de busca garante encontrar uma solução quando há uma?
 - Busca exaustiva *versus* busca heurística (com poda)
- ❑ Complexidade de tempo e espaço
- ❑ É ótima?
 - A estratégia encontra a solução de melhor qualidade quando há diferentes soluções?

Busca em Espaço de Estados

- ❑ Como já mencionado, um espaço de estados pode ser visto como um grafo $G(V,E)$ onde V é um conjunto de **vértices** (ou **nós**) e E é um conjunto de **arestas** (ou **arcos**)
- ❑ Em um grafo representando um espaço de estados:
 - V corresponde ao espaço de estados e
 - E corresponde às transições em G que podem ser alcançadas pela busca a partir de um estado (vértice) até outro (seu sucessor)
- ❑ A escolha de qual vértice expandir e qual aresta visitar define a estratégia de busca
- ❑ O **tamanho do problema** é usualmente expresso em termos do número de estados possíveis
 - Jogo-da-velha: 3^9 estados
 - Cubo de Rubik: 10^{19} estados
 - Xadrez: 10^{120} estados
 - Quebra-cabeça de 8 peças: $9!/2 = 181.440$ ($\sim 2 \cdot 10^5$) estados
 - Mundo do aspirador: $d \cdot 2^n$ estados (grau de sujeira d , n locais)
 - 8-Rainhas: 64^8 ($\sim 3 \cdot 10^{14}$) possíveis estados, podendo ser reduzido para 2.057
- ❑ Alguns espaços de estados podem ser infinitos

Árvore de Busca

- ❑ A busca no **espaço de estados** pode ser representada como uma **árvore de busca**
 - Cada **nó** na árvore de busca representa um **estado**
 - Cada **aresta** na árvore representa uma **ação**, correspondendo a uma alteração no ambiente partindo do nó pai para o nó filho
- ❑ A cada aresta pode ser associado um custo ou utilidade
- ❑ Cada **caminho** a partir da raiz da árvore até outro nó representa uma **sequência de ações**
- ❑ Alguns nós folhas da árvore de busca representam **estados finais**
- ❑ O problema de **navegar** no espaço de busca então pode ser entendido como **gerar e manter** uma árvore de busca até que uma solução (nó final) ou caminho seja gerado
- ❑ A **árvore de busca de um problema**:
 - É um **conceito abstrato** que contém um nó para cada estado possível do ambiente do problema
 - Pode ser infinita
- ❑ A **árvore de busca gerada por um algoritmo**:
 - É uma estrutura de dados concreta
 - É um subconjunto da árvore de busca do problema que contém apenas os nós gerados e mantidos (ou seja, explorados) até o ponto atual da execução do algoritmo de busca e, portanto, é sempre finita
- ❑ O **fator de ramificação do problema (problem search tree branching factor)**: é o número médio de ações (sucessores) disponíveis em cada estado
- ❑ O **fator de ramificação do algoritmo de busca (algorithm effective branching factor)**: é o número médio de nós efetivamente gerados como sucessores de cada nó durante a busca

Busca em Espaço de Estados

- ❑ Estratégias Básicas de Busca (Uniforme, Exaustiva ou Cega)
 - não utiliza informações sobre o problema para guiar a busca
 - estratégia de busca exaustiva aplicada até uma solução ser encontrada (ou falhar)
 - ❖ Profundidade (Depth-first ou DFS)
 - ❖ Profundidade limitada (Depth-first Limited ou DLS)
 - ❖ Profundidade iterativa (Iterative Deepening, IDS ou IDDFS)
 - ❖ Largura (Breadth-first ou BFS)
 - ❖ Custo Uniforme (Uniform Cost ou UCS)
 - ❖ Bidirecional (BDS)
- ❑ Estratégias Heurísticas de Busca (Busca Informada)
 - Serão vistas na próxima apresentação

Busca em Espaço de Estados

- ❑ Vamos representar um espaço de estados pela relações
 - **s(X,Y)** que é verdadeira se há um movimento permitido no espaço de estados do nó X para o nó Y; neste caso, Y é um **sucessor** de X
 - **final(X)** que é verdadeira se X é um estado final
- ❑ Se houver custos envolvidos, um terceiro argumento será adicionado, o custo do movimento
 - **s(X,Y,Custo)**
- ❑ A relação **s** pode ser representada explicitamente no programa por um conjunto de fatos
- ❑ Entretanto, para espaços de estado complexos a relação **s** é usualmente definida implicitamente por meio de regras que permitam calcular o sucessor de um dado nó

Busca em Espaço de Estados

- ❑ Outra questão importante é como representar situações do problema, que são por si só nós no espaço de estados
- ❑ A representação deve ser compacta e permitir execução eficiente das operações requeridas
- ❑ No exemplo da manipulação de blocos, vamos considerar o caso geral em que há um número qualquer de blocos que devem ser dispostos em uma ou mais pilhas:
 - O número de pilhas será limitado para tornar o problema mais interessante, além de ser realista uma vez que robôs que manipulam blocos possuem um espaço limitado na mesa
 - Uma situação do problema pode ser representada por uma lista de pilhas; cada pilha é representada por uma lista de blocos (de forma ordenada) de forma que o bloco no topo da pilha é a cabeça da lista
 - Pilhas vazias serão representadas por listas vazias
 - Situação inicial: [[c,a,b], [], []]
 - Situação final:
 - ❖ [[a,b,c], [], []]
 - ❖ [[], [a,b,c], []]
 - ❖ [[], [], [a,b,c]]

Busca em Espaço de Estados

- ❑ A relação sucessor pode ser programada da seguinte forma:
Situação2 é sucessora de Situação1 se há duas pilhas, Pilha1 e Pilha2 em Situação1 e o topo da Pilha1 pode ser movido para Pilha2

- ❑ Todas as situação podem ser representadas por listas de pilhas

```
s(Pilhas, [Pilha1, [Topo|Pilha2]|OutrasPilhas]) :- % Move Topo p/ Pilha2
    del([Topo|Pilha1], Pilhas, Resto),           % encontre 1a pilha
    del(Pilha2, Resto, OutrasPilhas).             % encontre 2a pilha
```

```
del(E, [E|L], L).
```

```
del(E, [Y|L], [Y|L1]) :-
    del(E, L, L1).
```

- ❑ A situação final do nosso exemplo é:

```
final(Situacao) :-
    pertence([a,b,c], Situacao).
```

Busca em Espaço de Estados

- ❑ Vamos programar algoritmos de busca como a relação **resolva1(Início,Solucao)** onde **Início** é o nó inicial no espaço de estados e **Solucao** é um caminho entre Início e qualquer nó final
- ❑ No exemplo de manipulação de blocos, a chamada correspondente seria:

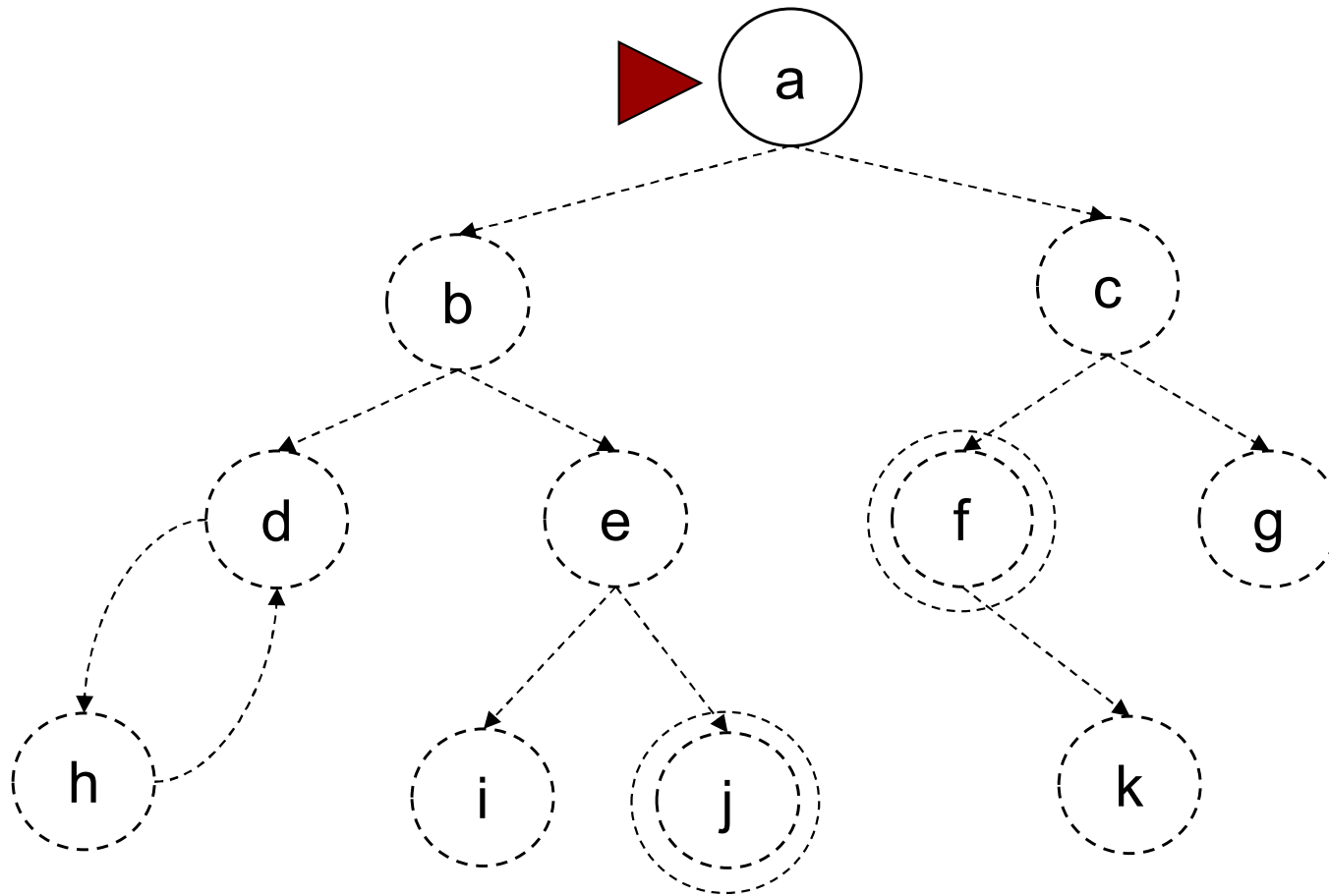
```
?- resolva1([[c,a,b],[],[ ]],Solucao).
```
- ❑ Como resultado de uma busca bem sucedida, **Solucao** é instanciada com uma lista de arranjos de blocos que representa um plano para transformar o estado inicial em um estado em que todos os três blocos estão em uma única pilha arranjados como [a,b,c]

Busca em Profundidade

- ❑ O algoritmo para busca em profundidade é o seguinte: para encontrar uma solução S para um dado nó N (até um nó final):
 - Se N é um nó final então $S = [N]$
 - Se N tem um sucessor $N1$ tal que há um caminho $S1$ de $N1$ até um nó final, então $S = [N \mid S1]$
- ❑ Em Prolog:

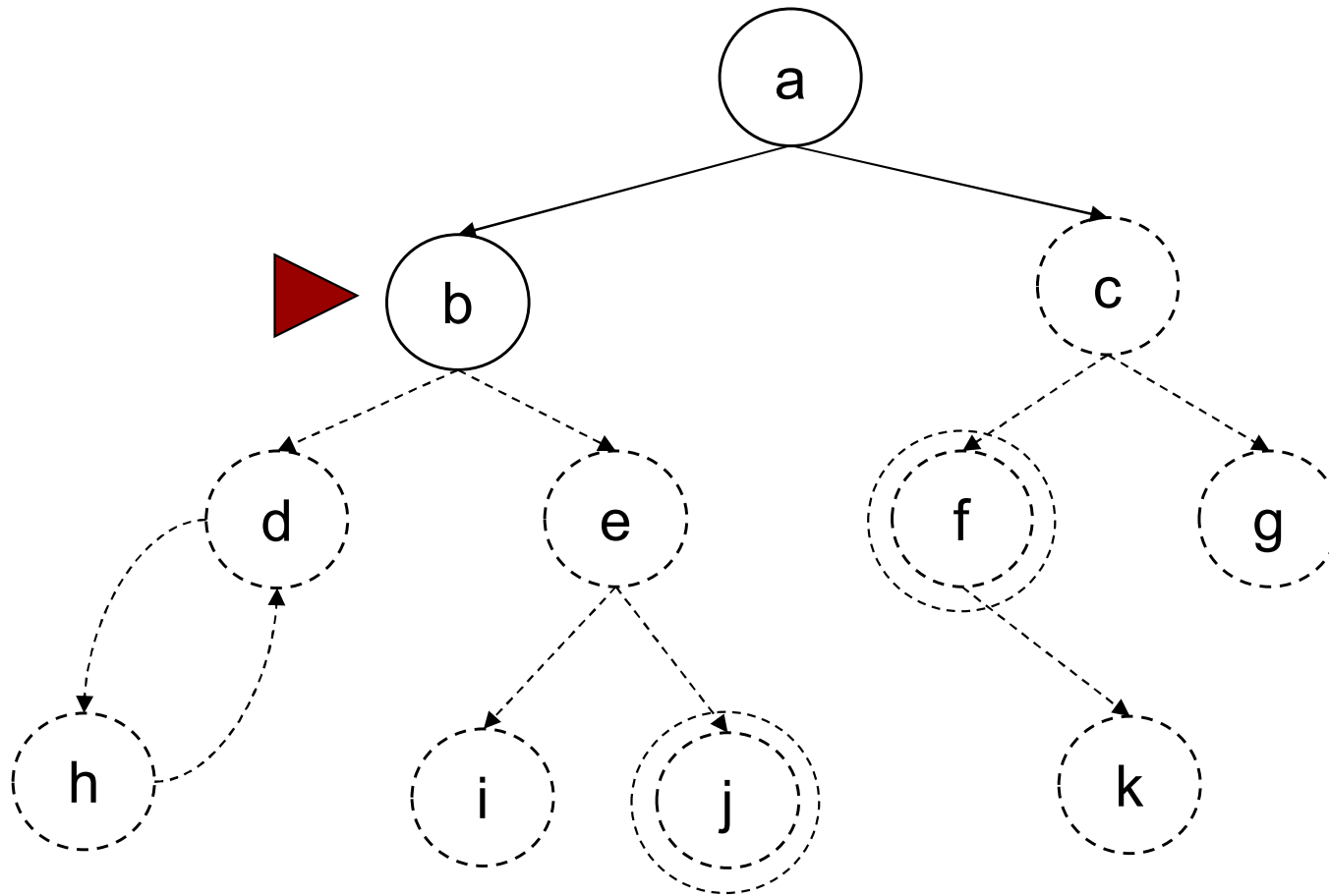
```
resolva1(N, [N]) :-  
    final(N).  
resolva1(N, [N|S1]) :-  
    s(N, N1),  
    resolva1(N1, S1).
```
- ❑ A busca em profundidade é o recurso mais simples em programação recursiva e, por isso, Prolog quando executa metas explora alternativas utilizando-a
- ❑ Entretanto, não há detecção de ciclos

Busca em Profundidade



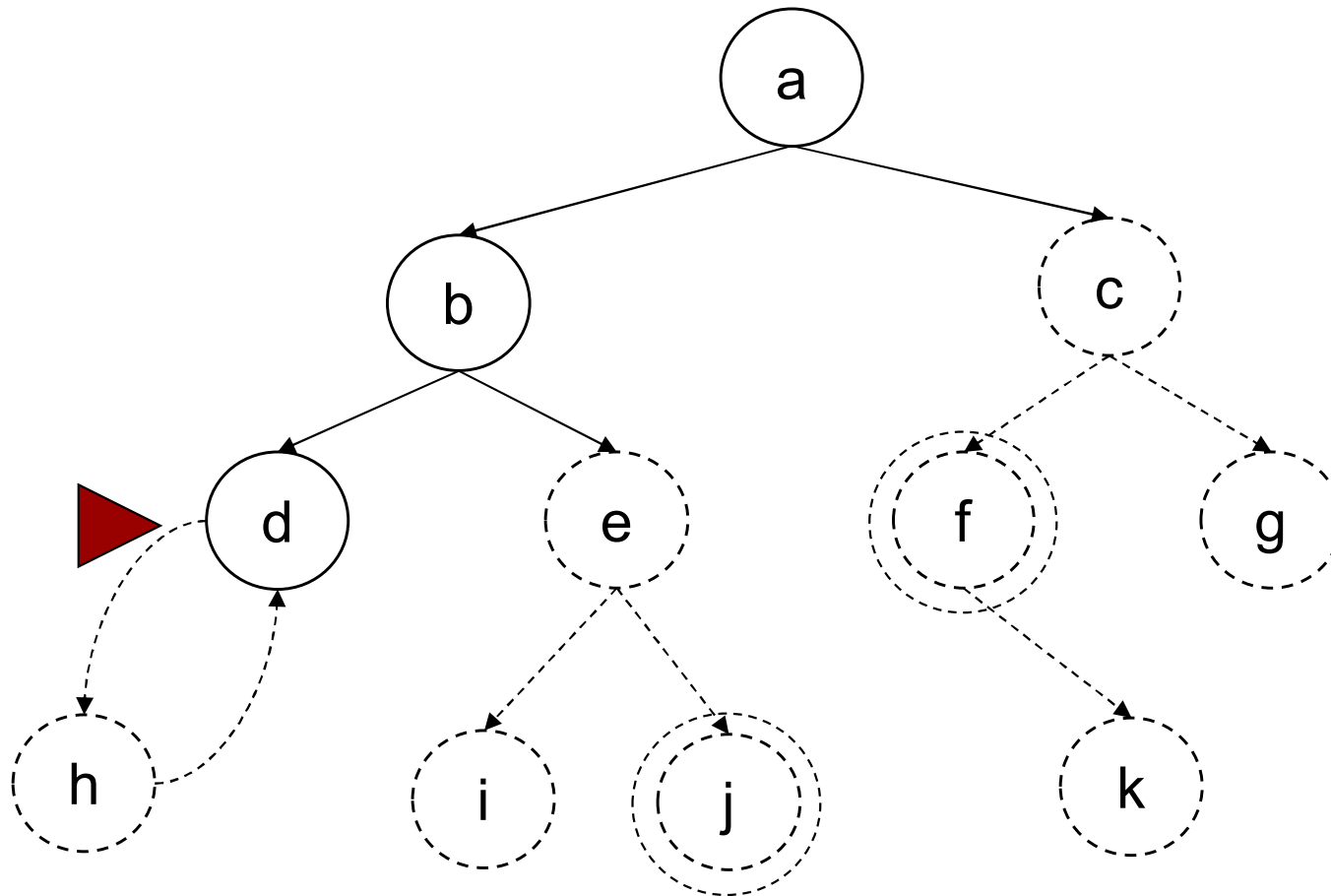
Inserir na frente, remover da frente: a

Busca em Profundidade



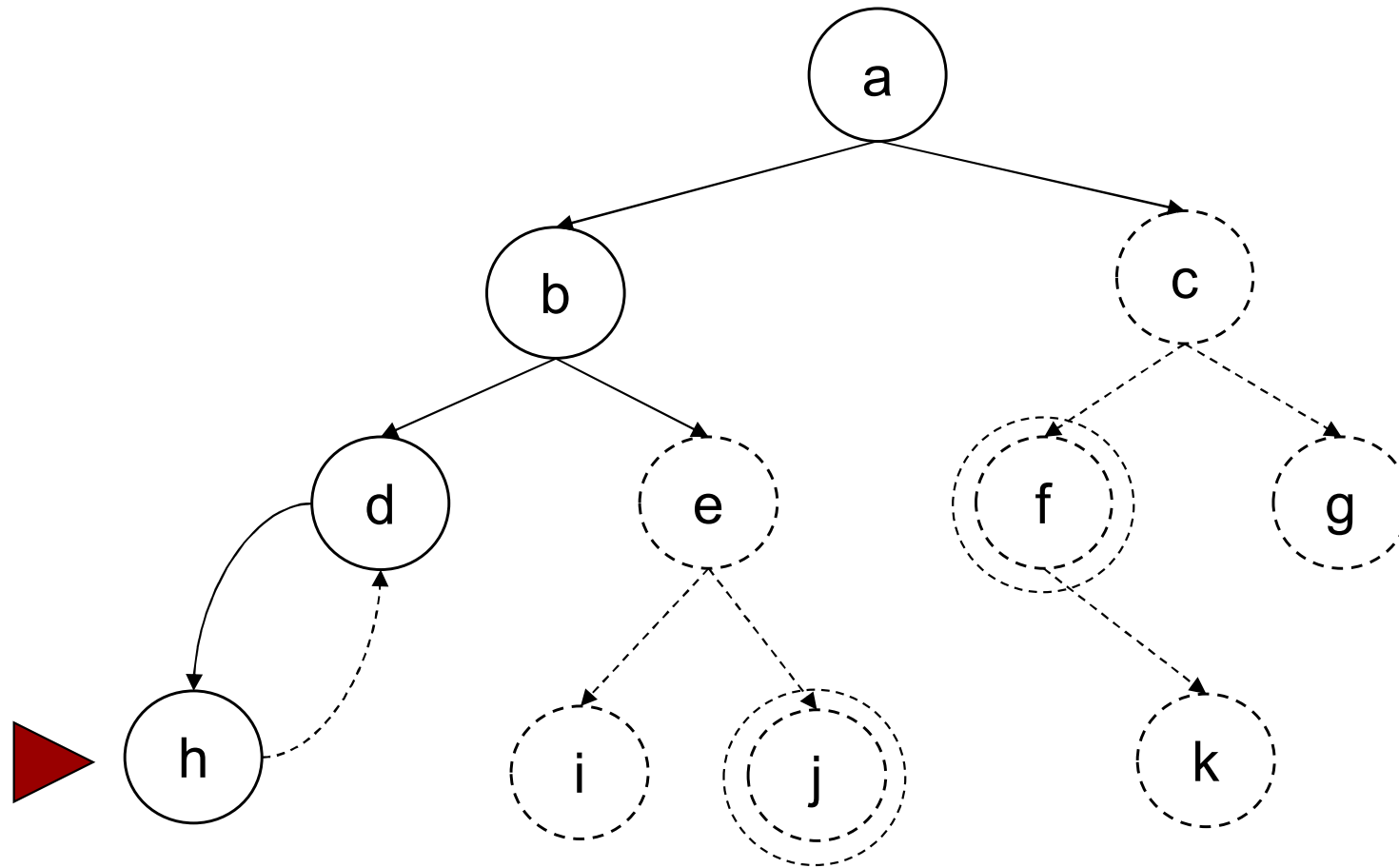
Inserir na frente, remover da frente: b, c

Busca em Profundidade



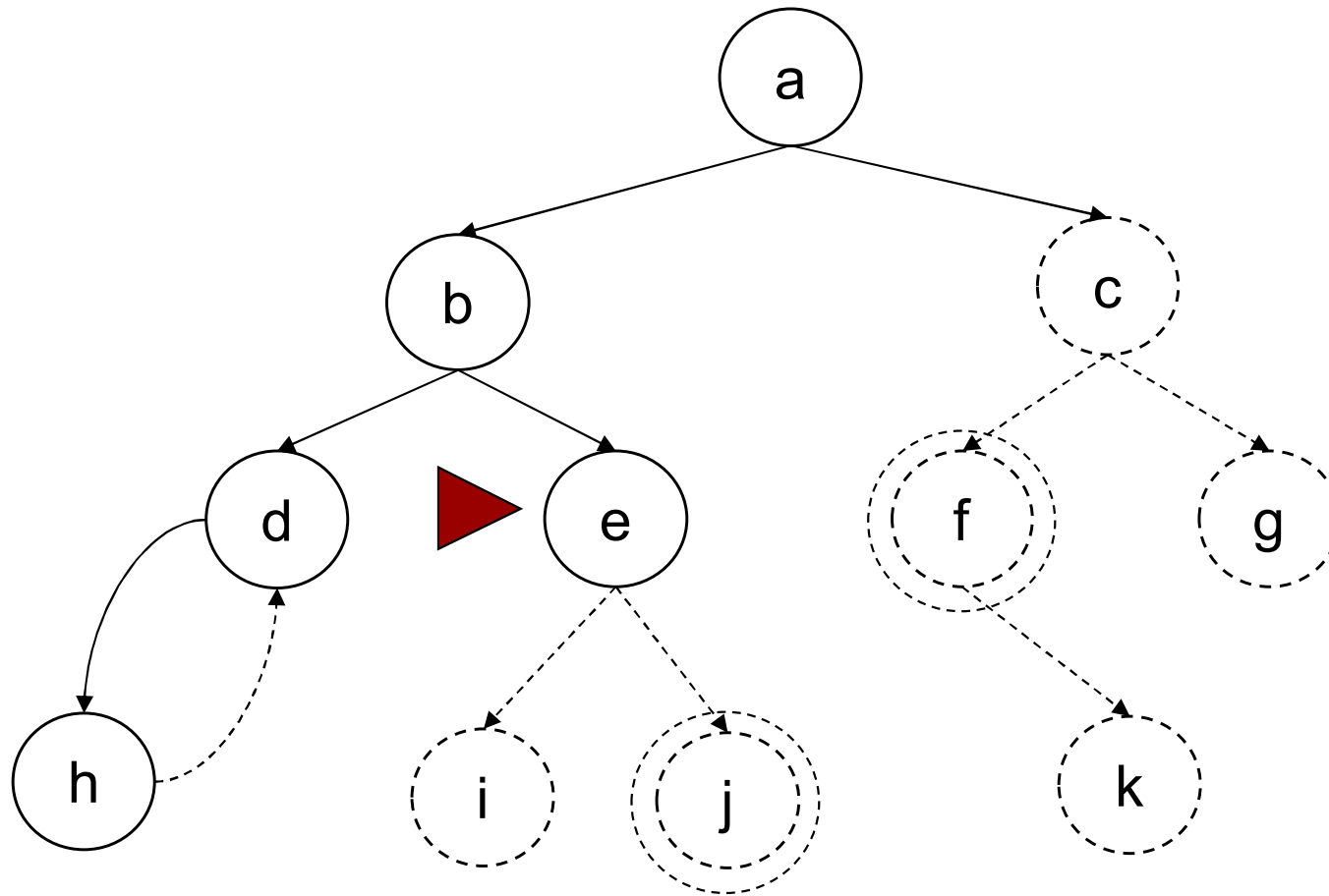
Inserir na frente, remover da frente: d, e, c

Busca em Profundidade



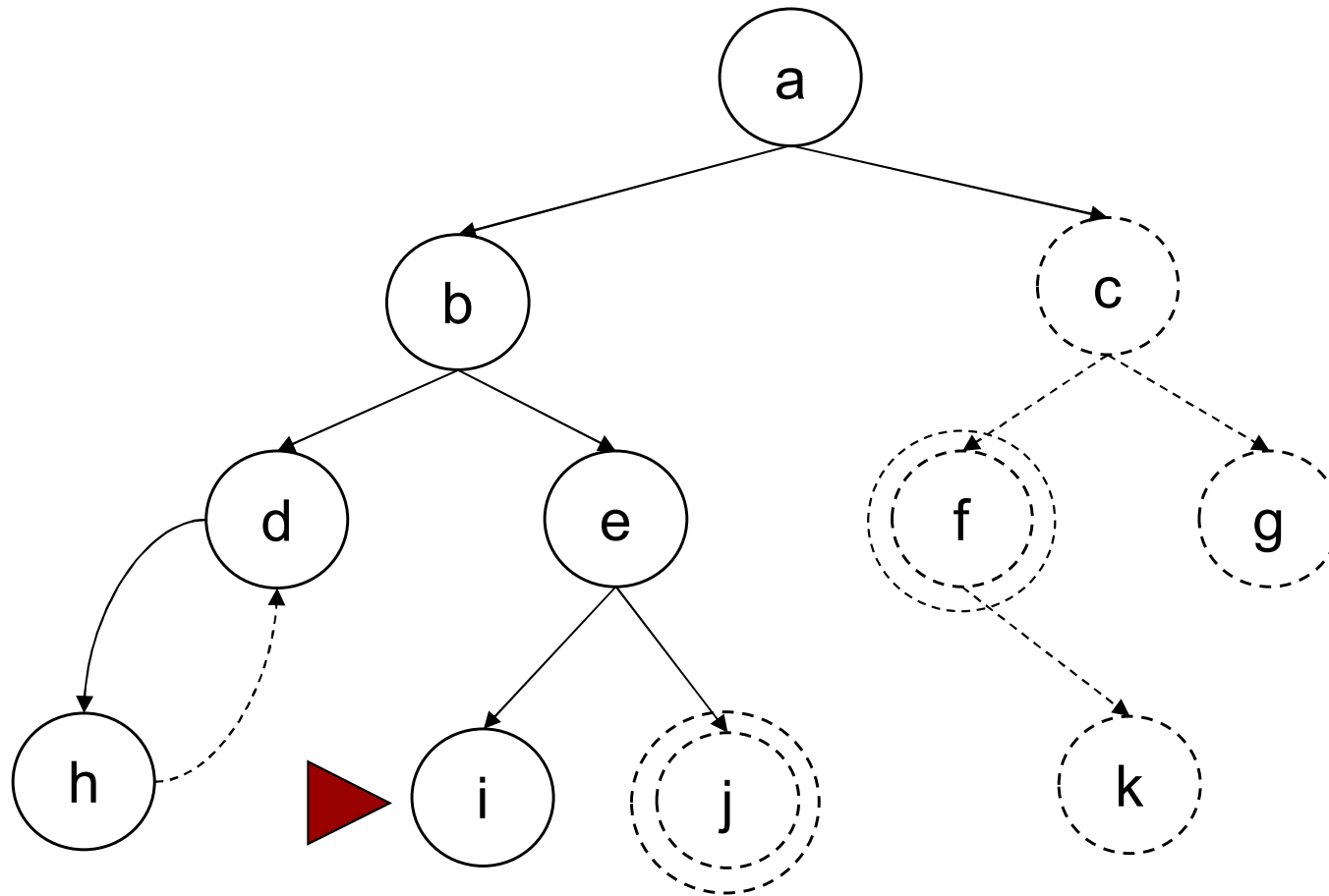
Inserir na frente, remover da frente: h, e, c

Busca em Profundidade



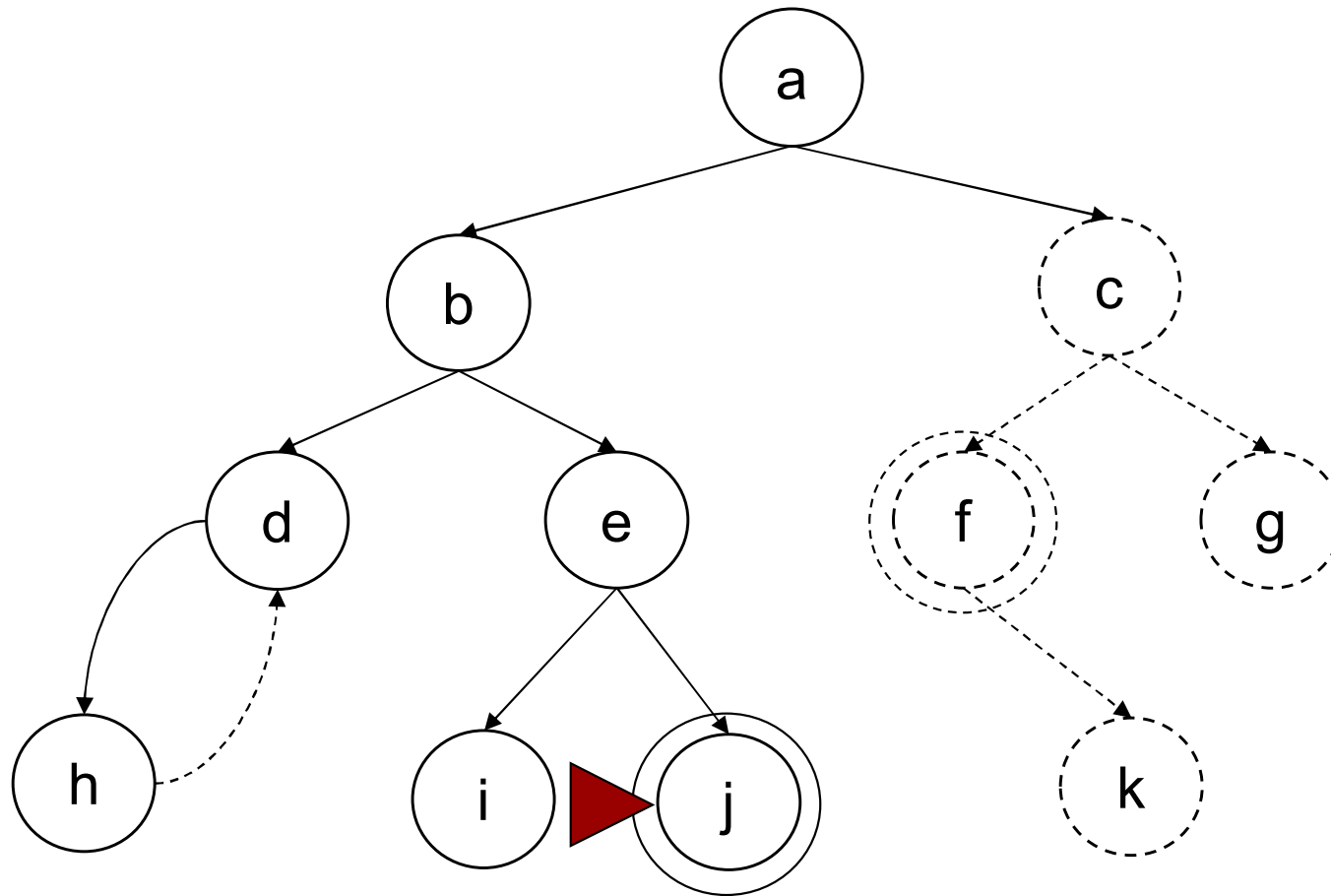
Inserir na frente, remover da frente: e, c

Busca em Profundidade



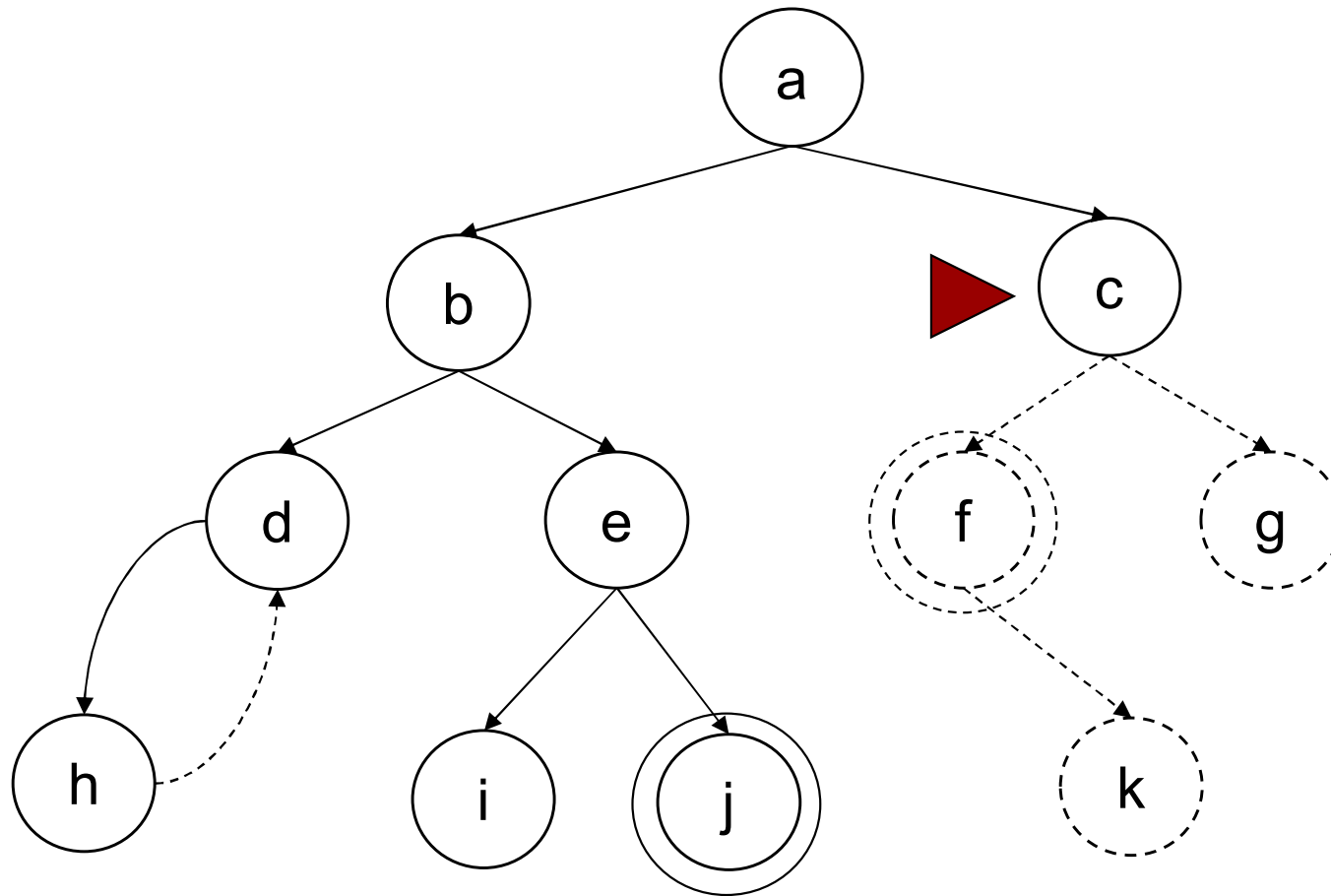
Inserir na frente, remover da frente: i, j, c

Busca em Profundidade



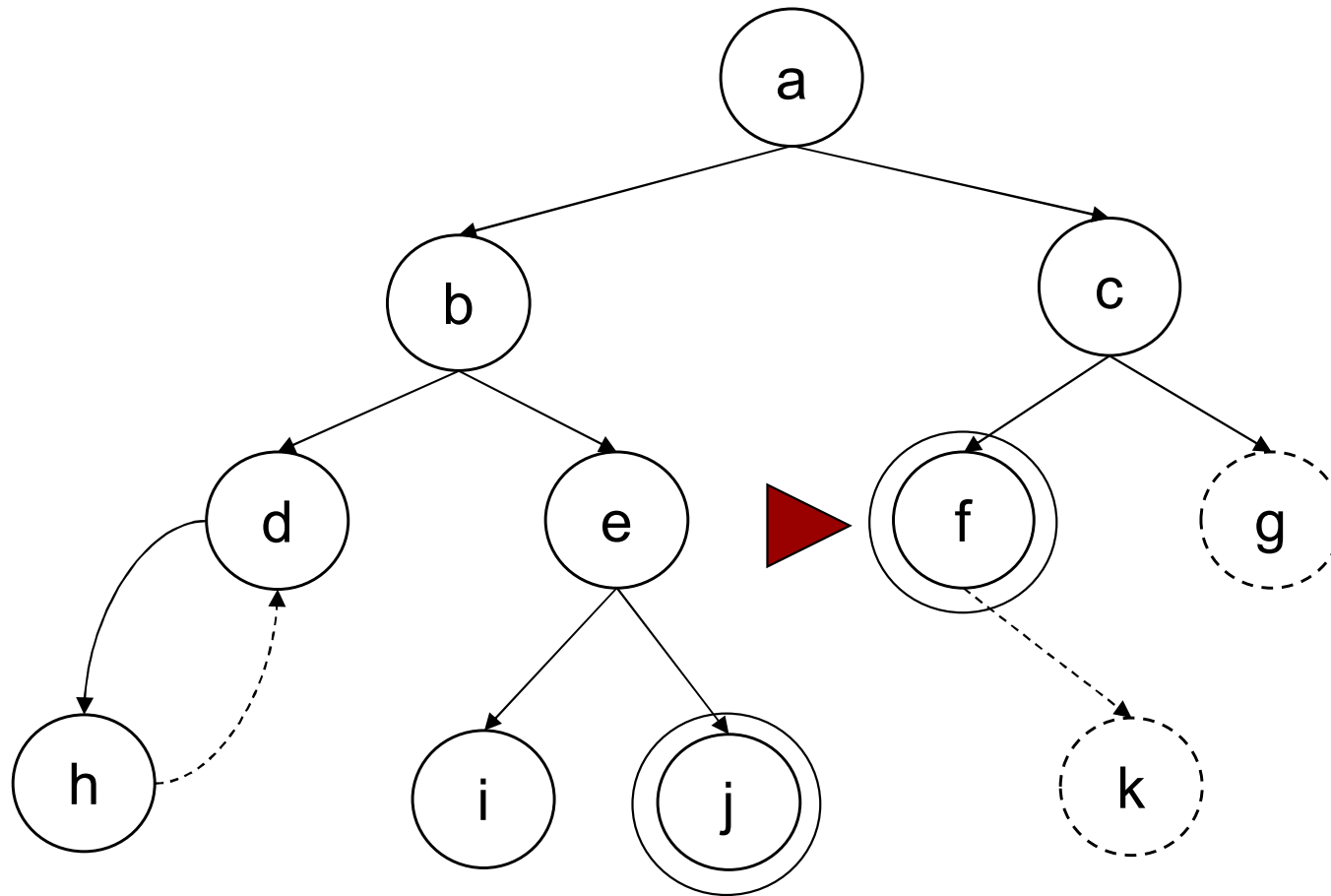
Inserir na frente, remover da frente: j, c

Busca em Profundidade



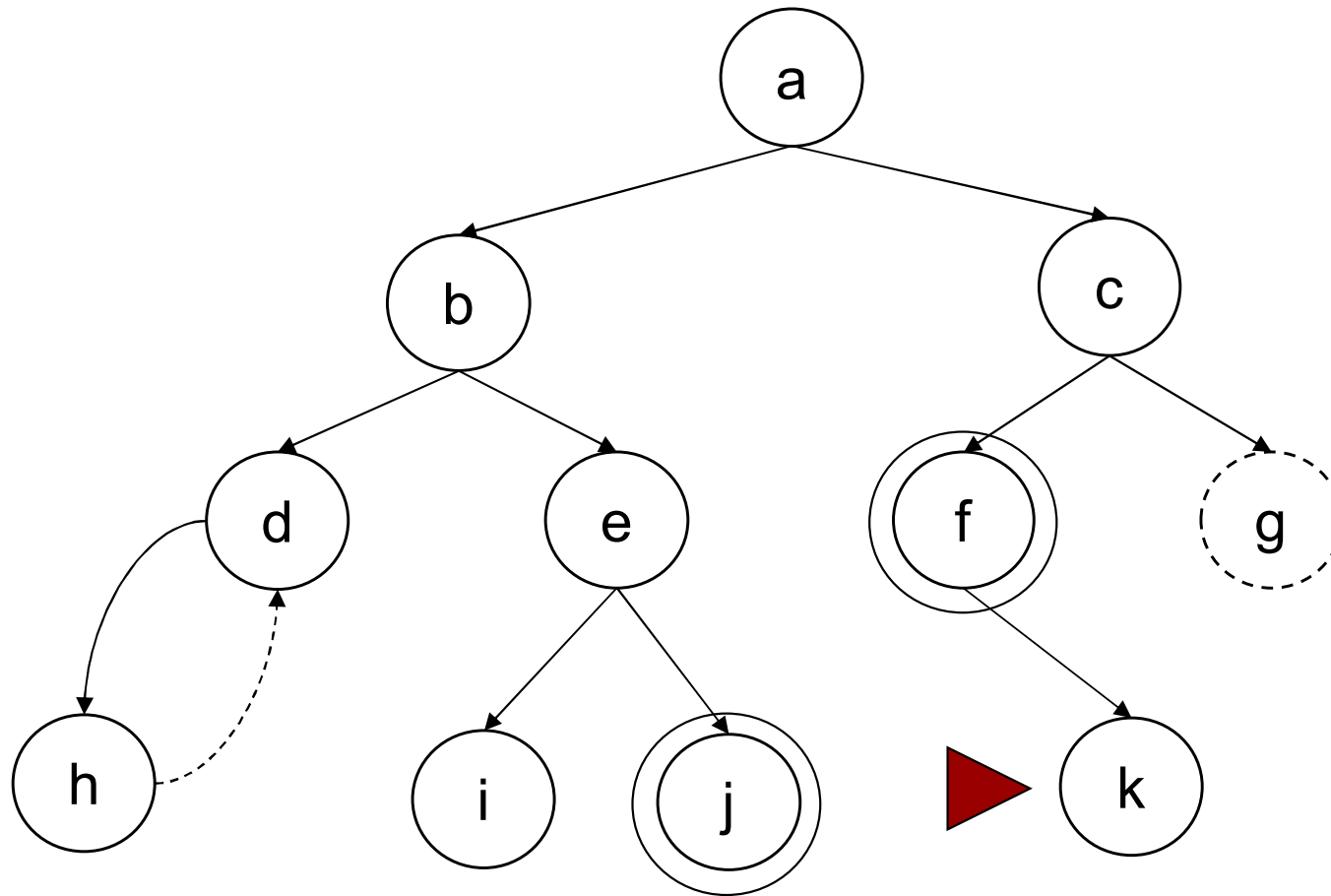
Inserir na frente, remover da frente: c

Busca em Profundidade



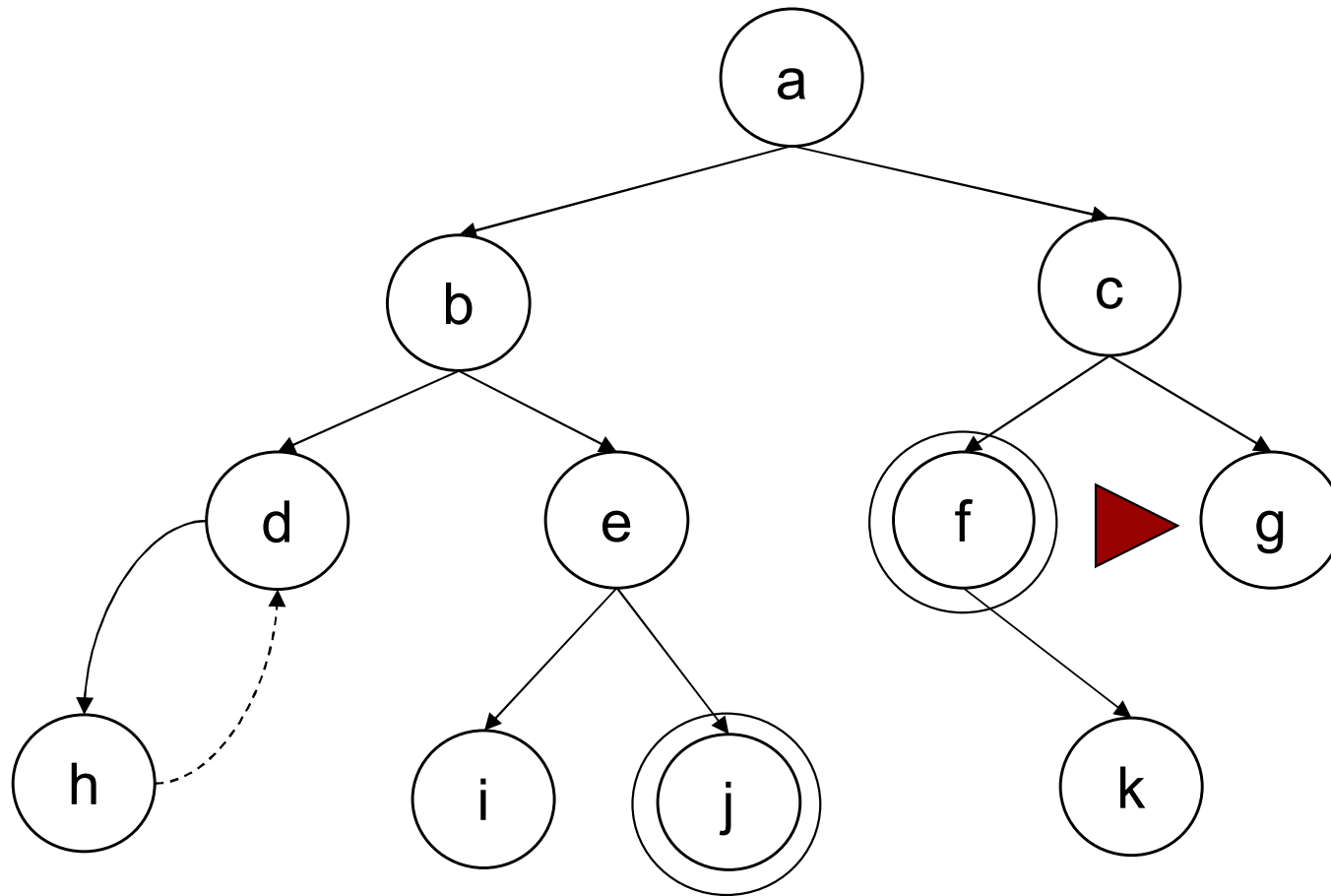
Inserir na frente, remover da frente: f, g

Busca em Profundidade



Inserir na frente, remover da frente: k, g

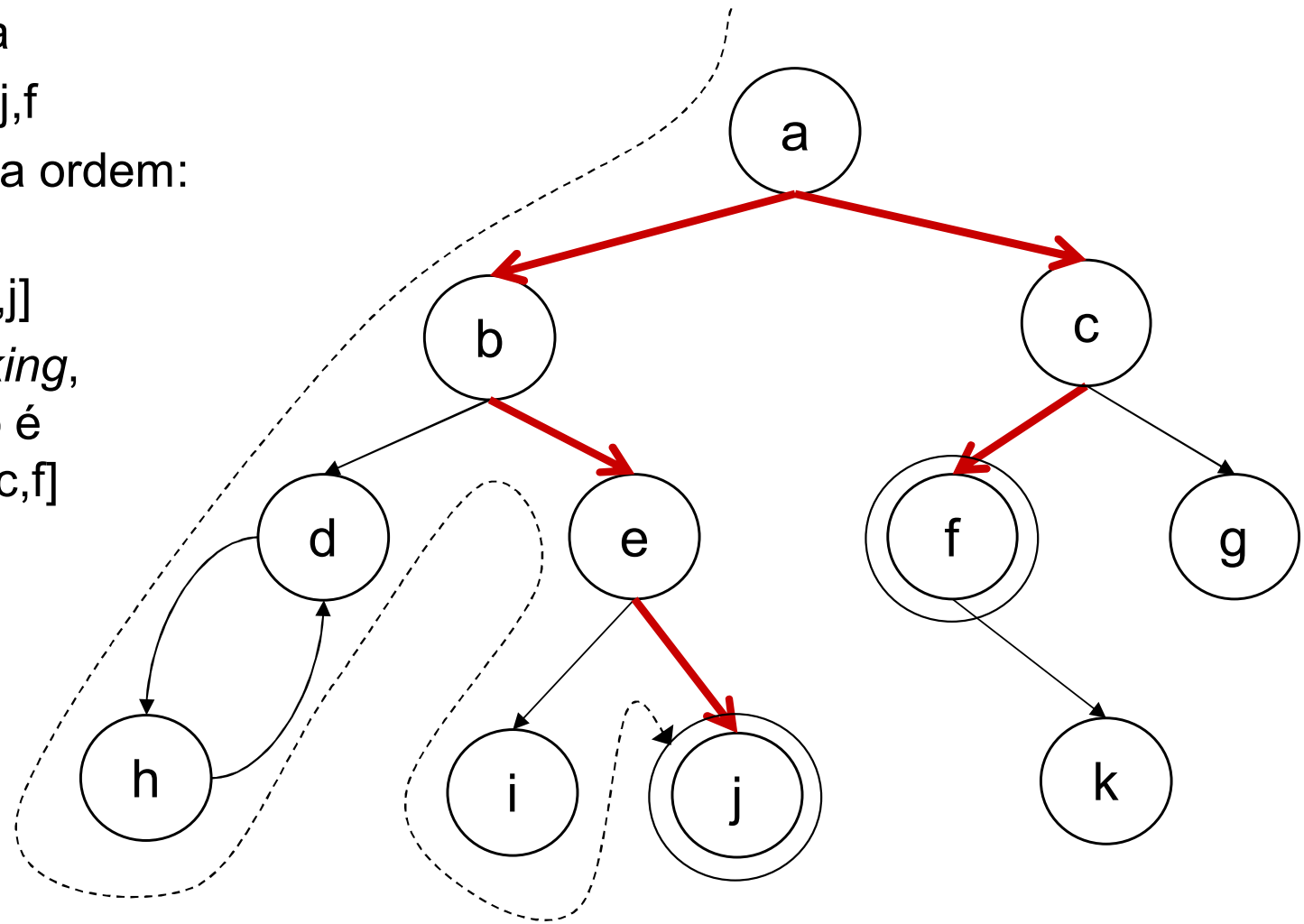
Busca em Profundidade



Inserir na frente, remover da frente: g

Busca em Profundidade

- ❑ Estado inicial: a
- ❑ Estados finais: j,f
- ❑ Nós visitados na ordem:
a,b,d,h,e,i,j
- ❑ Solução: [a,b,e,j]
- ❑ Após *backtracking*,
a outra solução é encontrada: [a,c,f]



Busca em Profundidade

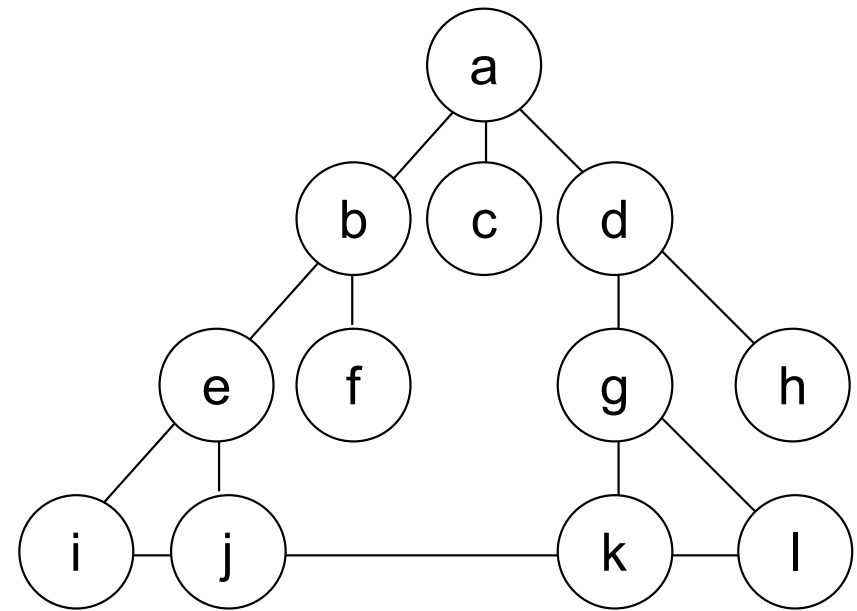
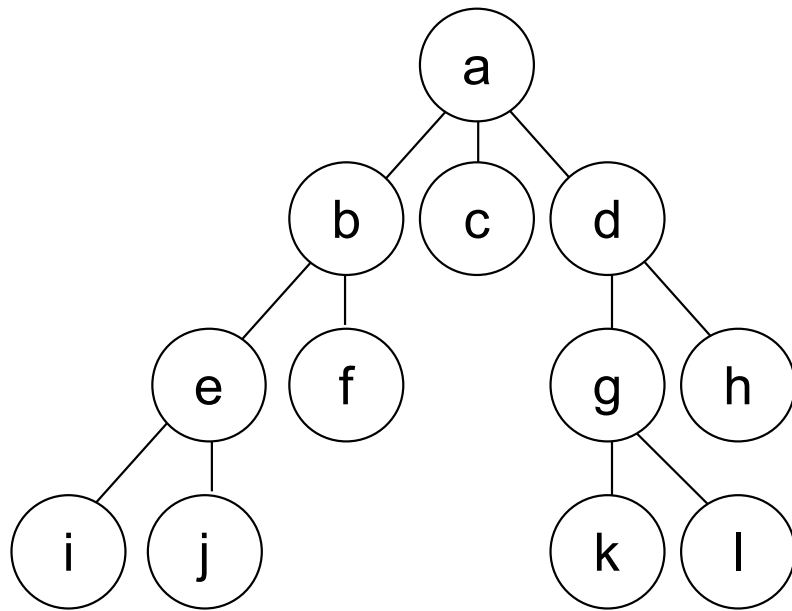
```
% resolve2(No,Solucao) Solucao é um caminho acíclico
% (na ordem reversa) entre nó inicial No e um nó final
resolve2(No,Solucao) :-
    depthFirst([],No,Solucao).

% depthFirst(Caminho,No,Solucao) estende o caminho [No|Caminho]
% até um nó final obtendo Solucao
depthFirst(Caminho,No,[No|Caminho]) :-
    final(No).
depthFirst(Caminho,No,S) :-
    s(No,No1),
    \+ pertence(No1,Caminho),           % evita um ciclo
    depthFirst([No|Caminho],No1,S).

pertence(E,[E|_]).
pertence(E,[_|T]) :-
    pertence(E,T).
```

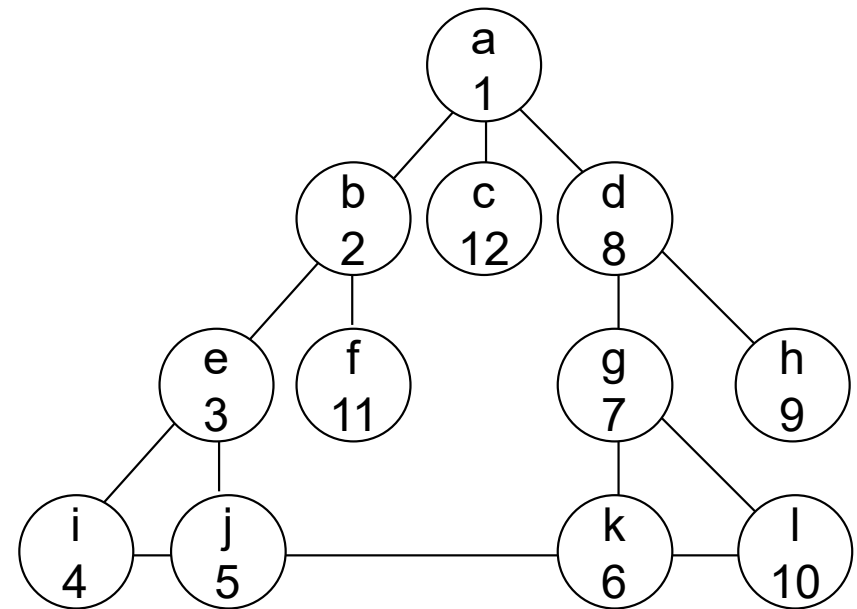
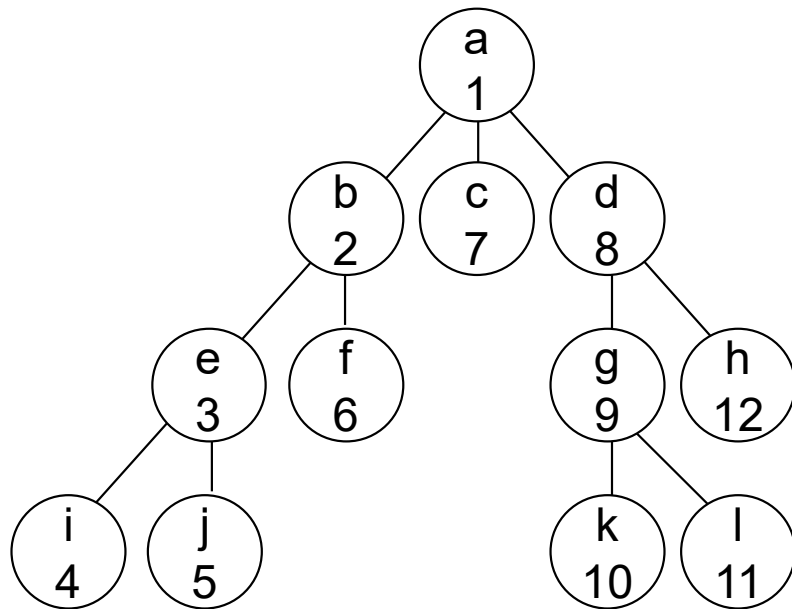
Exercício: Indique a ordem na qual os nós são visitados na busca em profundidade

Assuma que os sucessores de um nó são definidos em ordem alfabética

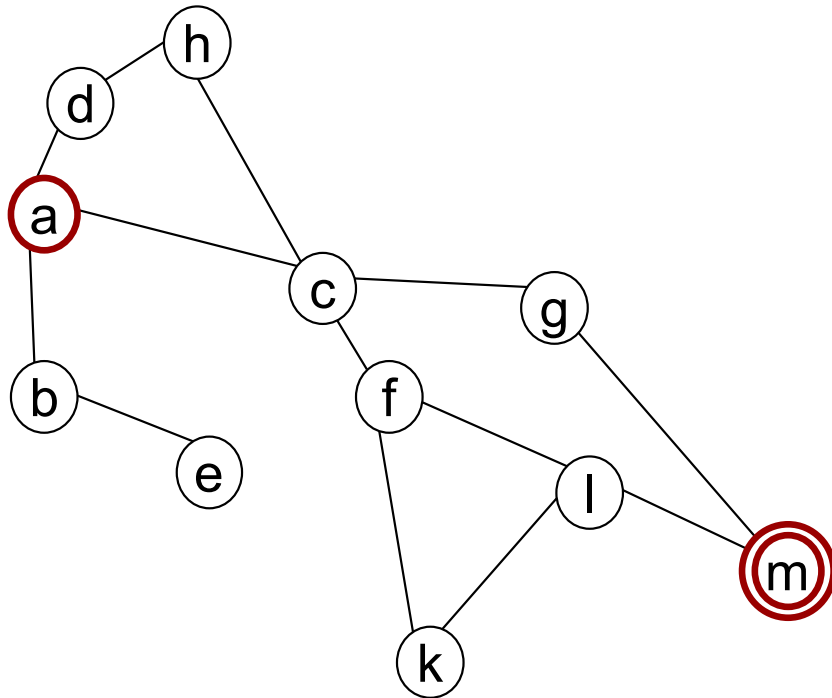


Solução

Assuma que os sucessores de um nó são definidos em ordem alfabética



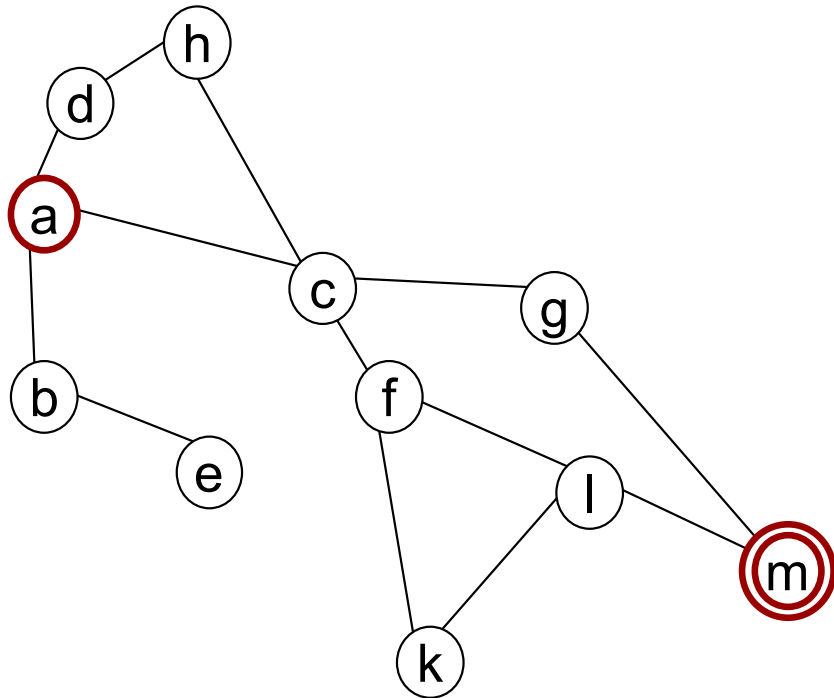
Exercício



Assuma que os sucessores de um nó são definidos em ordem alfabética

- ❑ Mostre a árvore de busca definida pelo algoritmo de busca em profundidade, partindo de **a** e chegando até **m**
- ❑ Mostre também os caminhos encontrados na ordem em que são encontrados pela busca em profundidade

Solução



Caminhos encontrados:

[a,c,f,k,l,m]

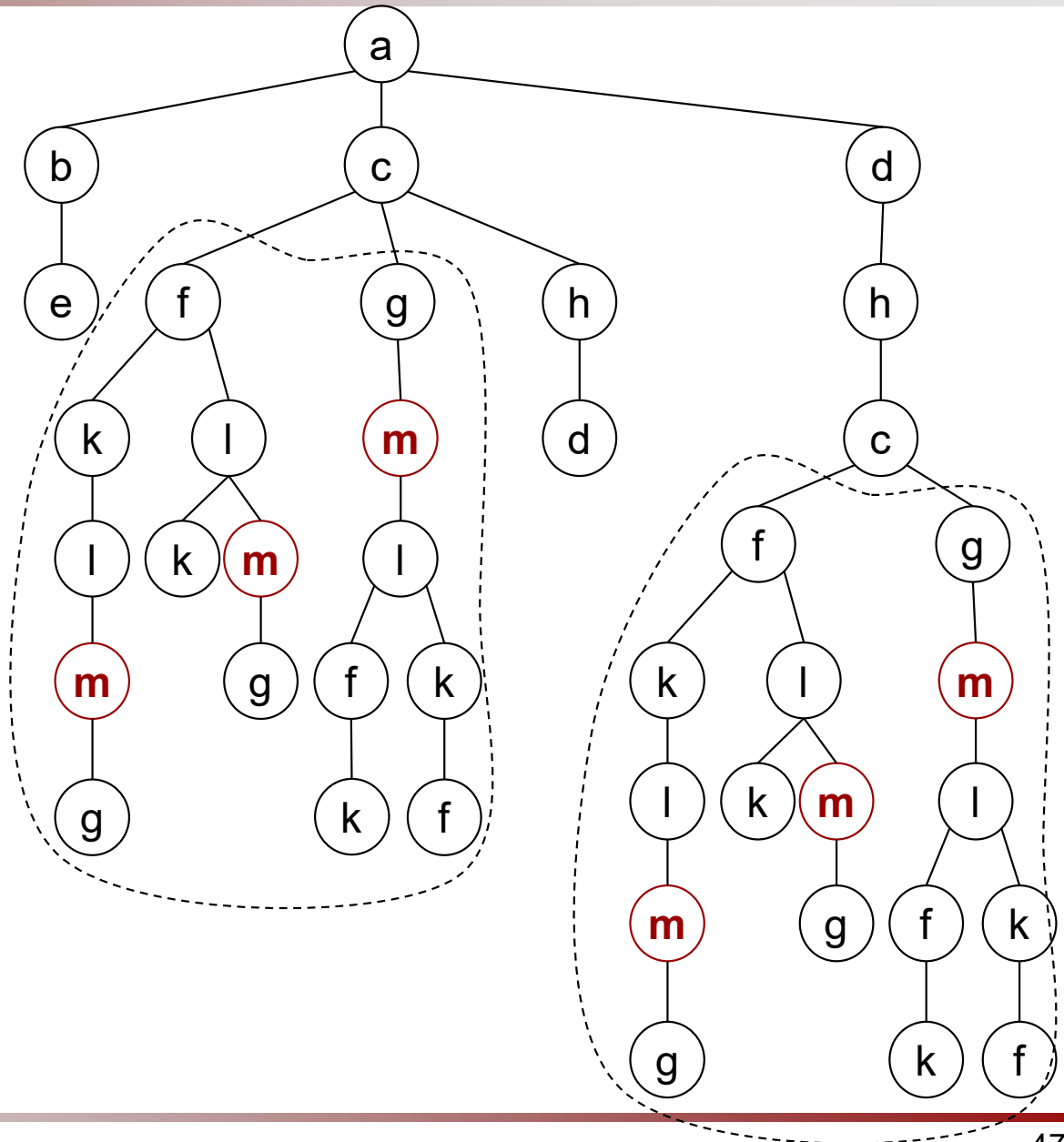
[a,c,f,l,m]

[a,c,g,m]

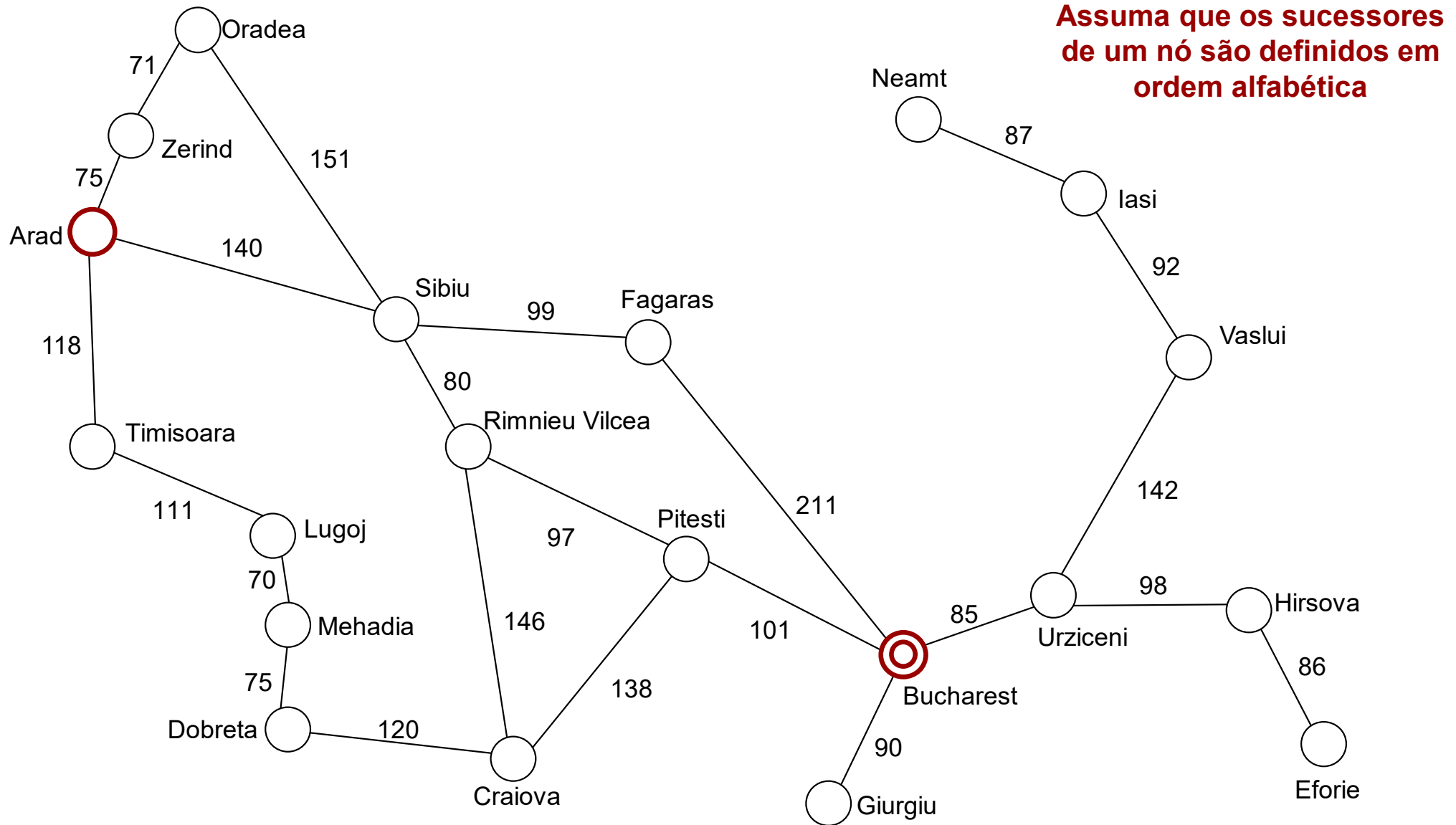
[a,d,h,c,f,k,l,m]

[a,d,h,c,f,l,m]

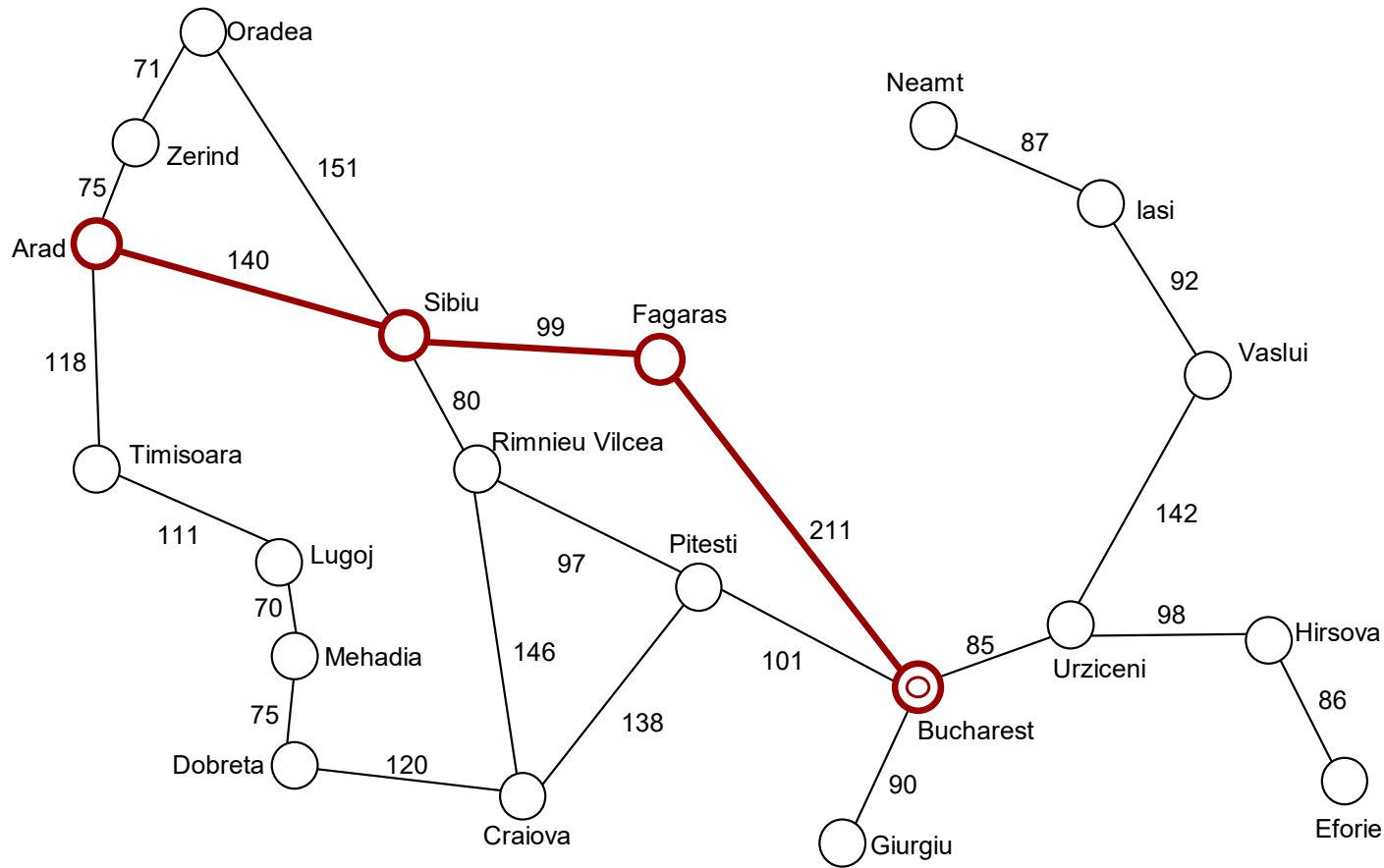
[a,d,h,c,g,m]



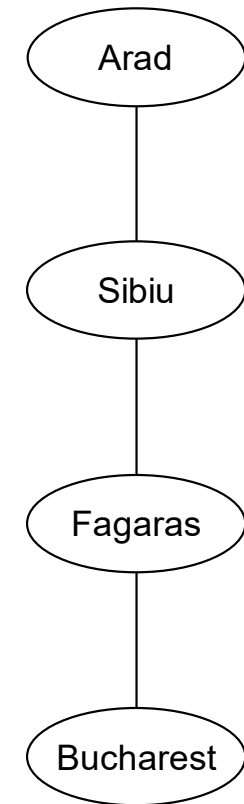
Encontre o Caminho e Árvore de Busca de Arad até Bucharrest usando Busca em Profundidade



Solução



Árvore de Busca

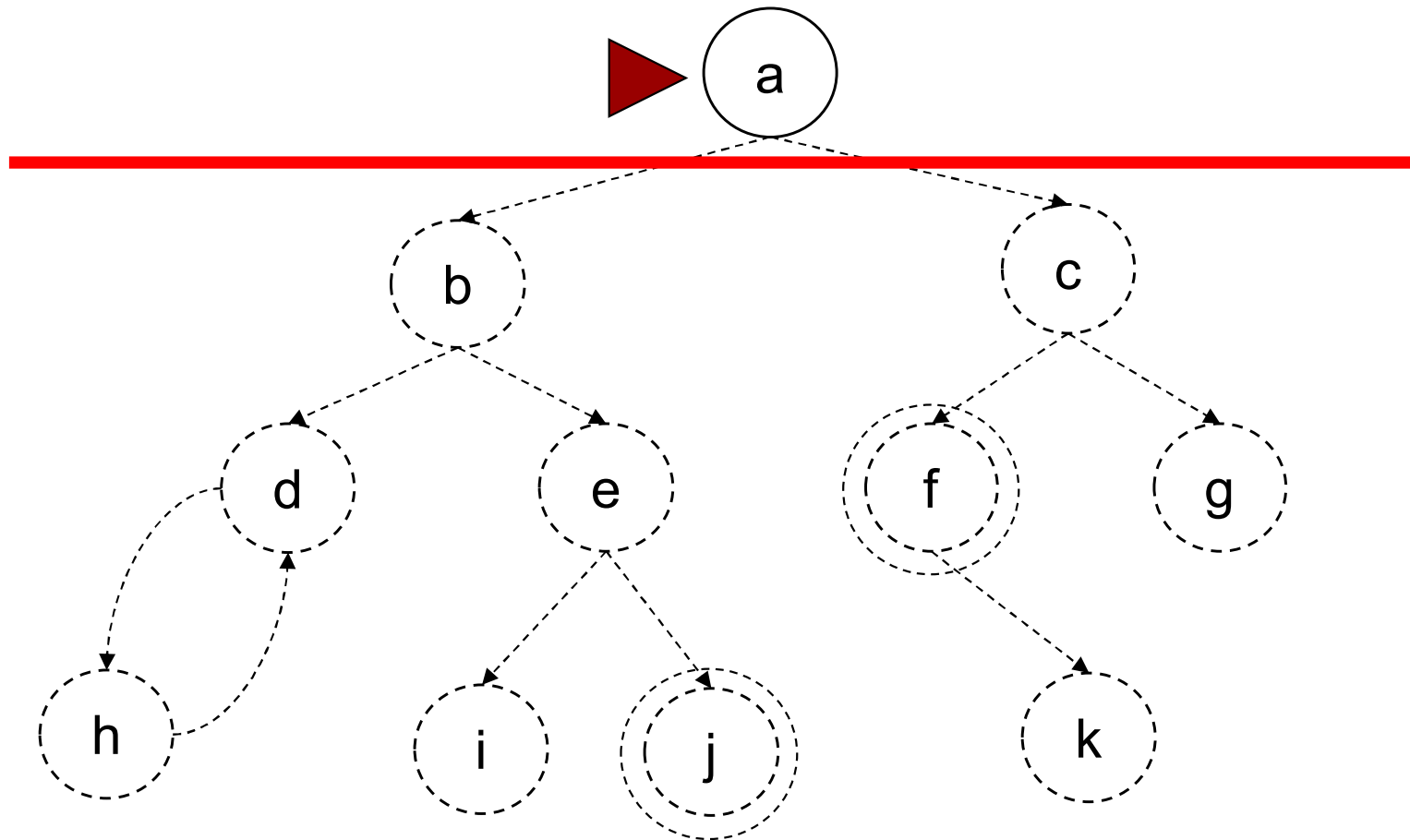


Busca em Profundidade

- ❑ Um problema com a busca em profundidade é que existem espaços de estados nos quais o algoritmo se perde
- ❑ Muitos espaços de estado são infinitos e, nesse caso, o algoritmo de busca em profundidade pode perder um nó final, prosseguindo por um caminho infinito no grafo
- ❑ O algoritmo então explora esta parte infinita do espaço, nunca chegando perto de um nó final
- ❑ Por exemplo, o problema das 8 Rainhas é susceptível a este tipo de armadilha, mas como o espaço é finito, as rainhas podem ser colocadas em segurança no tabuleiro, ou seja, uma solução é encontrada
- ❑ Para evitar caminhos infinitos (sem ciclos), um refinamento pode ser adicionado à busca em profundidade: limitar a profundidade de busca

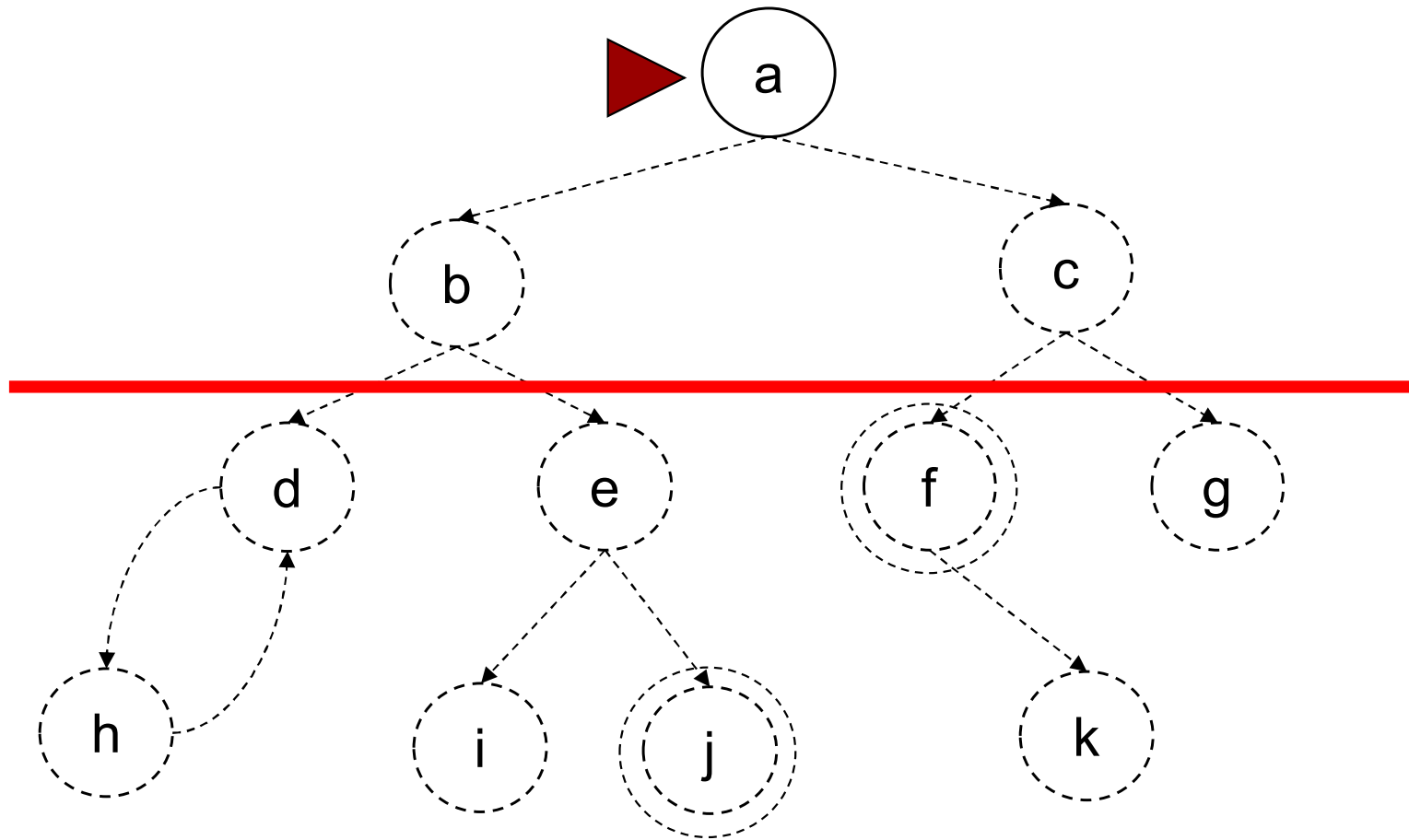
Busca em Profundidade Limitada ($L=0$)

Nenhuma solução
encontrada para $L=0$



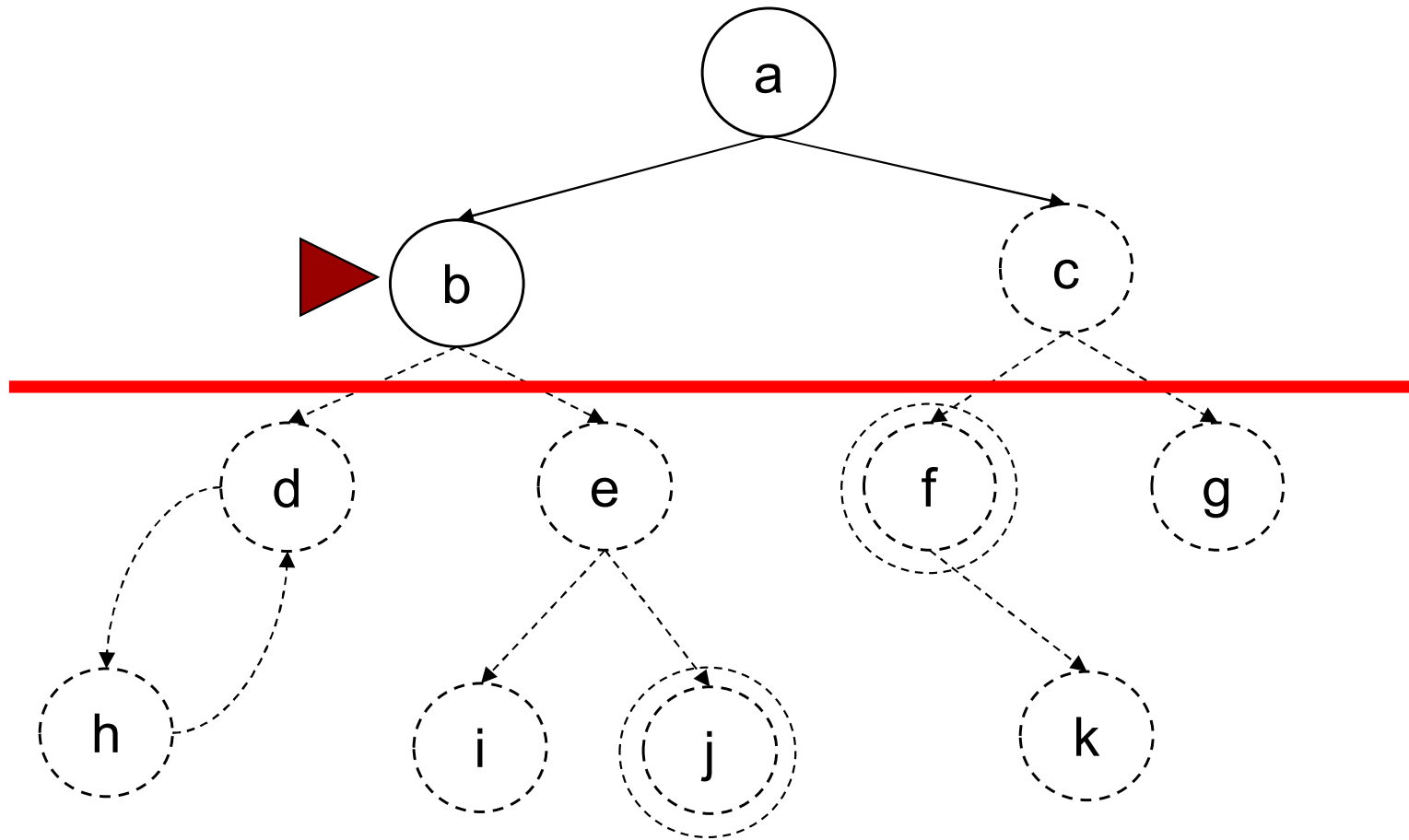
Inserir na frente, remover da frente: a

Busca em Profundidade Limitada ($L=1$)



Inserir na frente, remover da frente: a

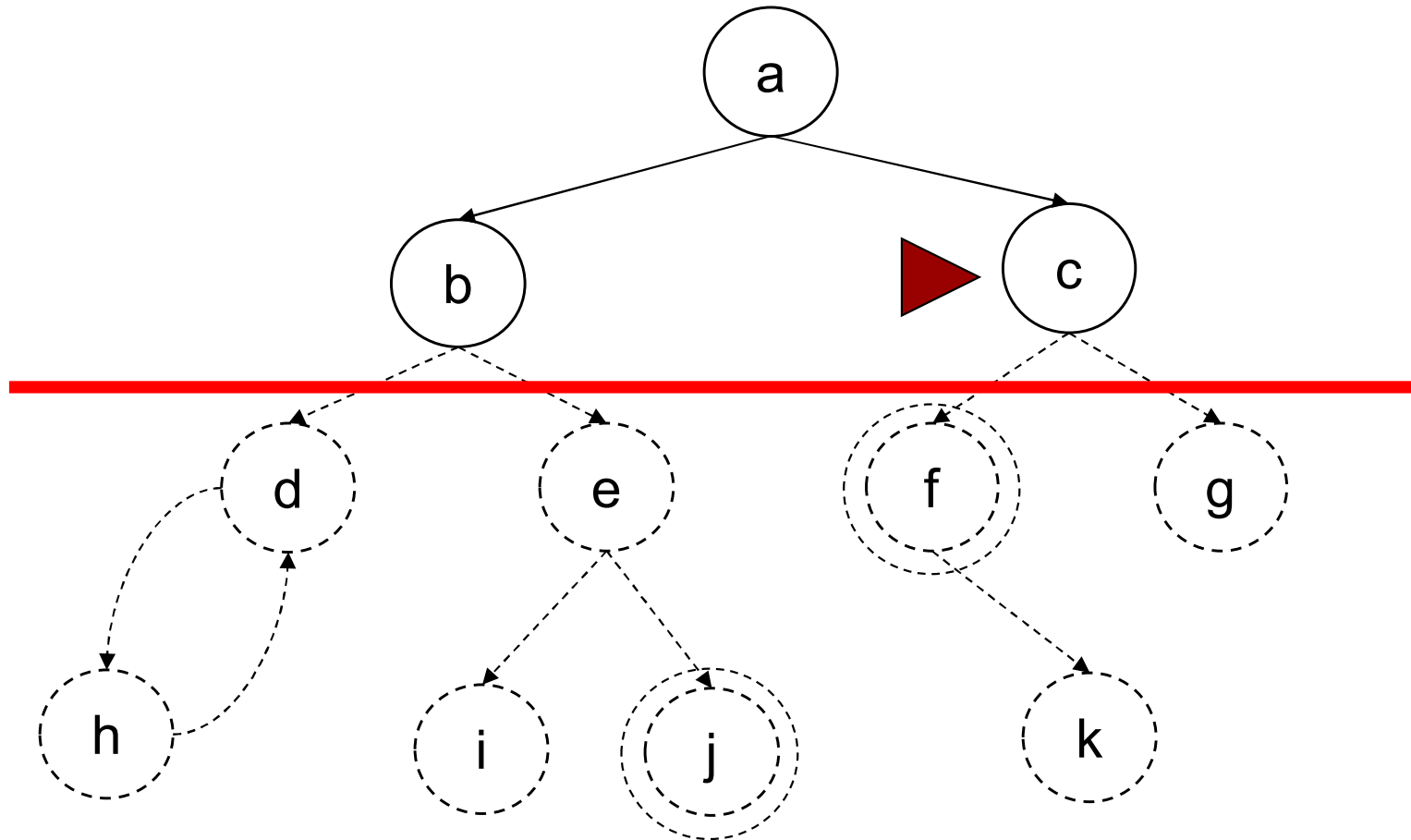
Busca em Profundidade Limitada ($L=1$)



Inserir na frente, remover da frente: b, c

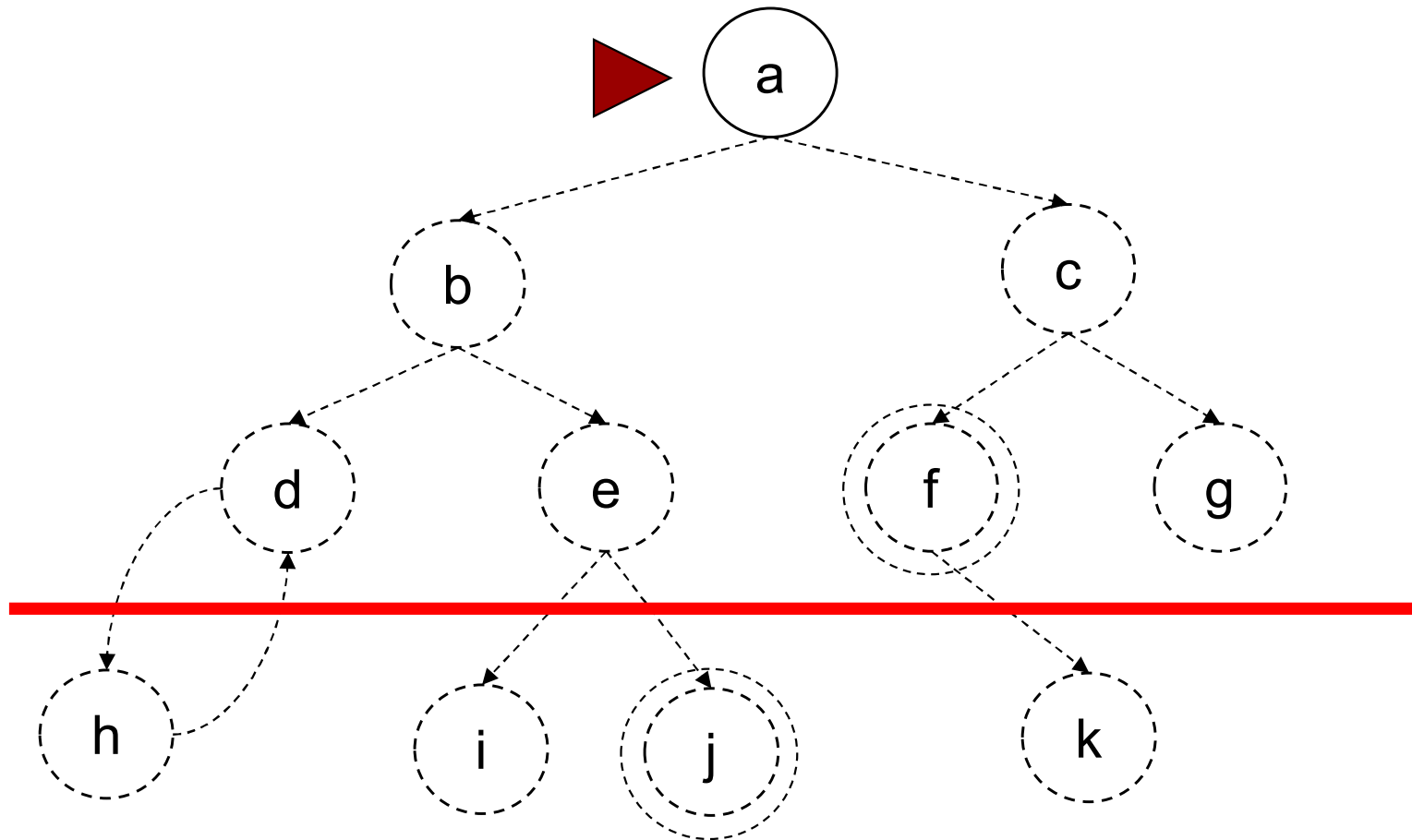
Busca em Profundidade Limitada ($L=1$)

Nenhuma solução
encontrada para $L=1$



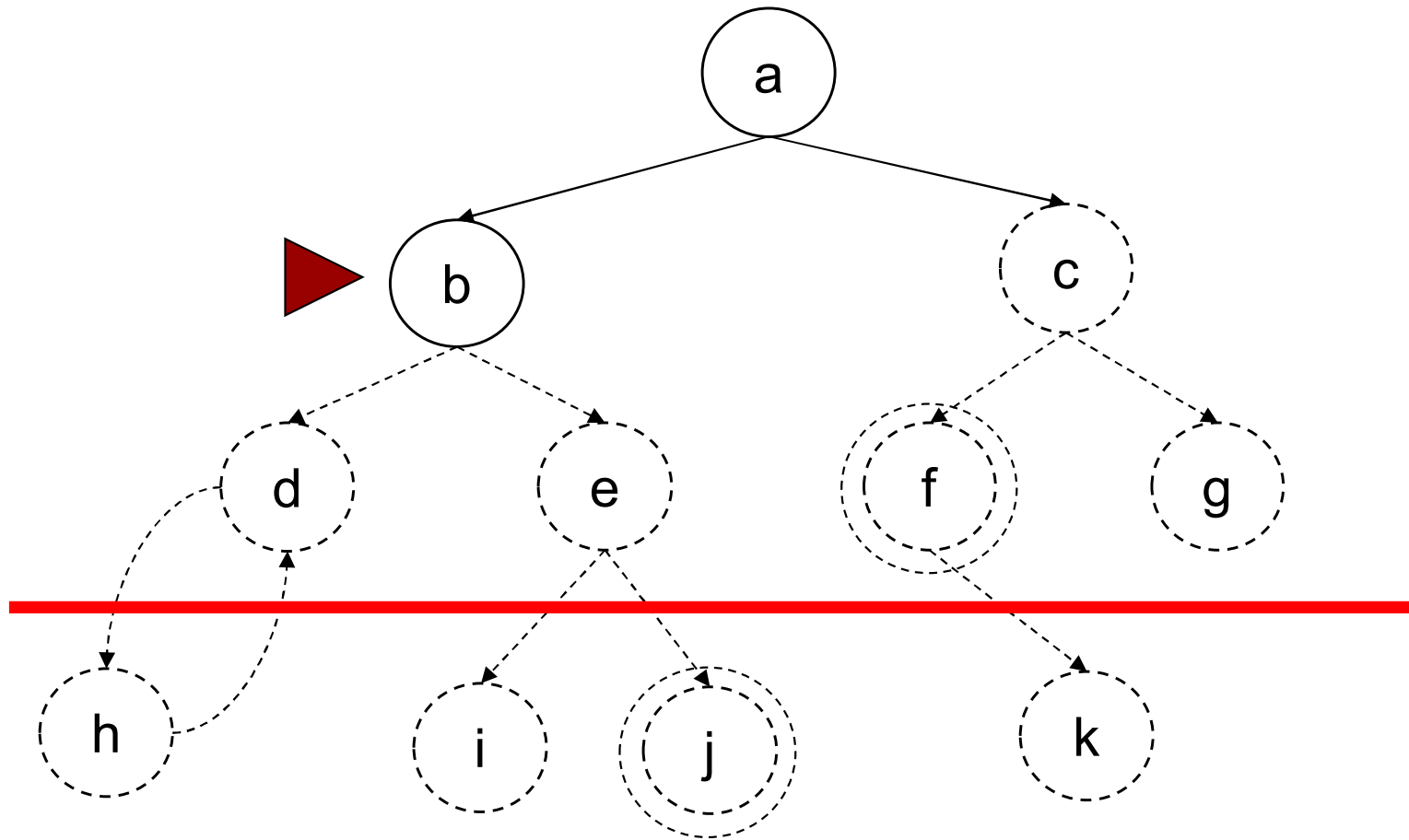
Inserir na frente, remover da frente: c

Busca em Profundidade Limitada ($L=2$)



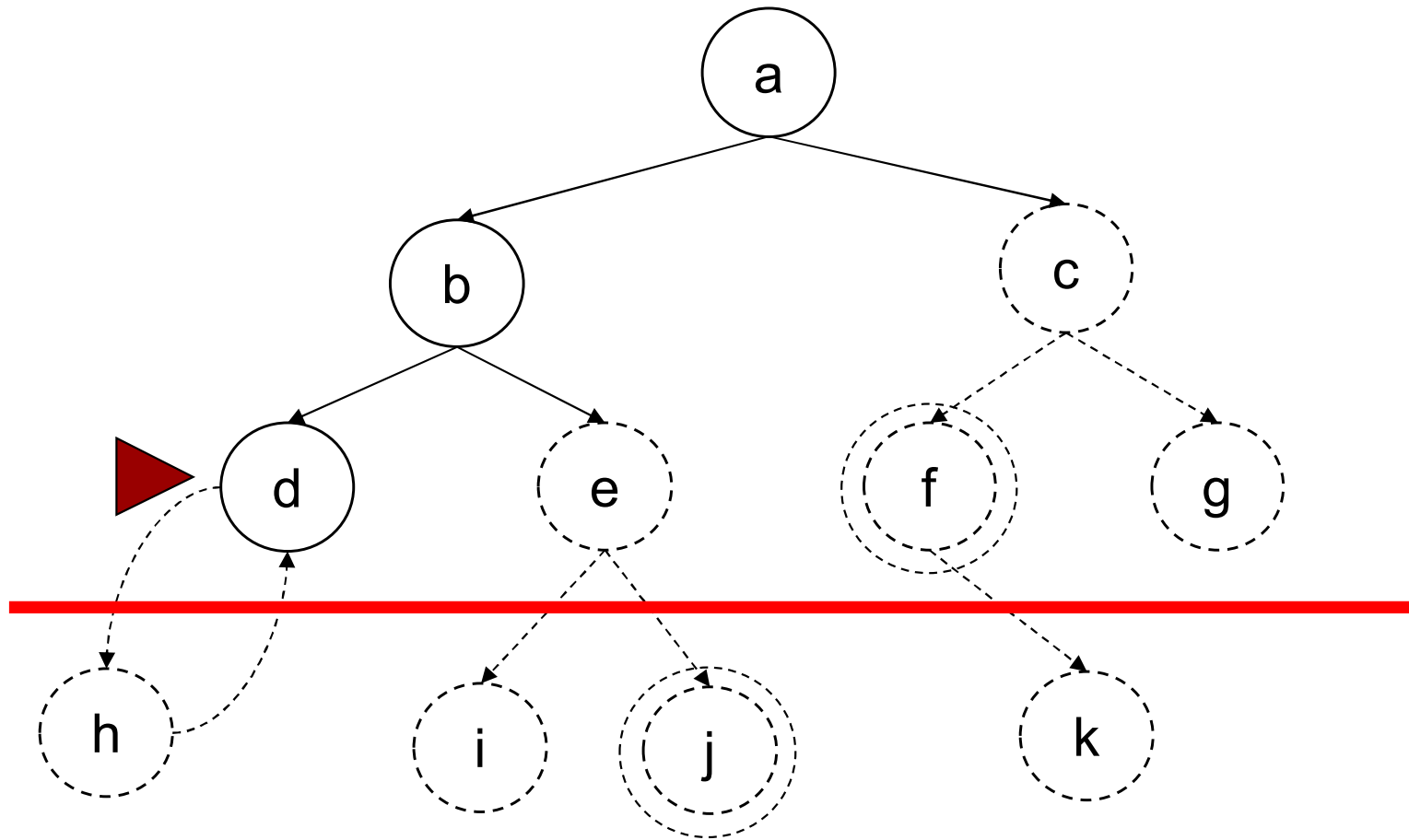
Inserir na frente, remover da frente: a

Busca em Profundidade Limitada ($L=2$)



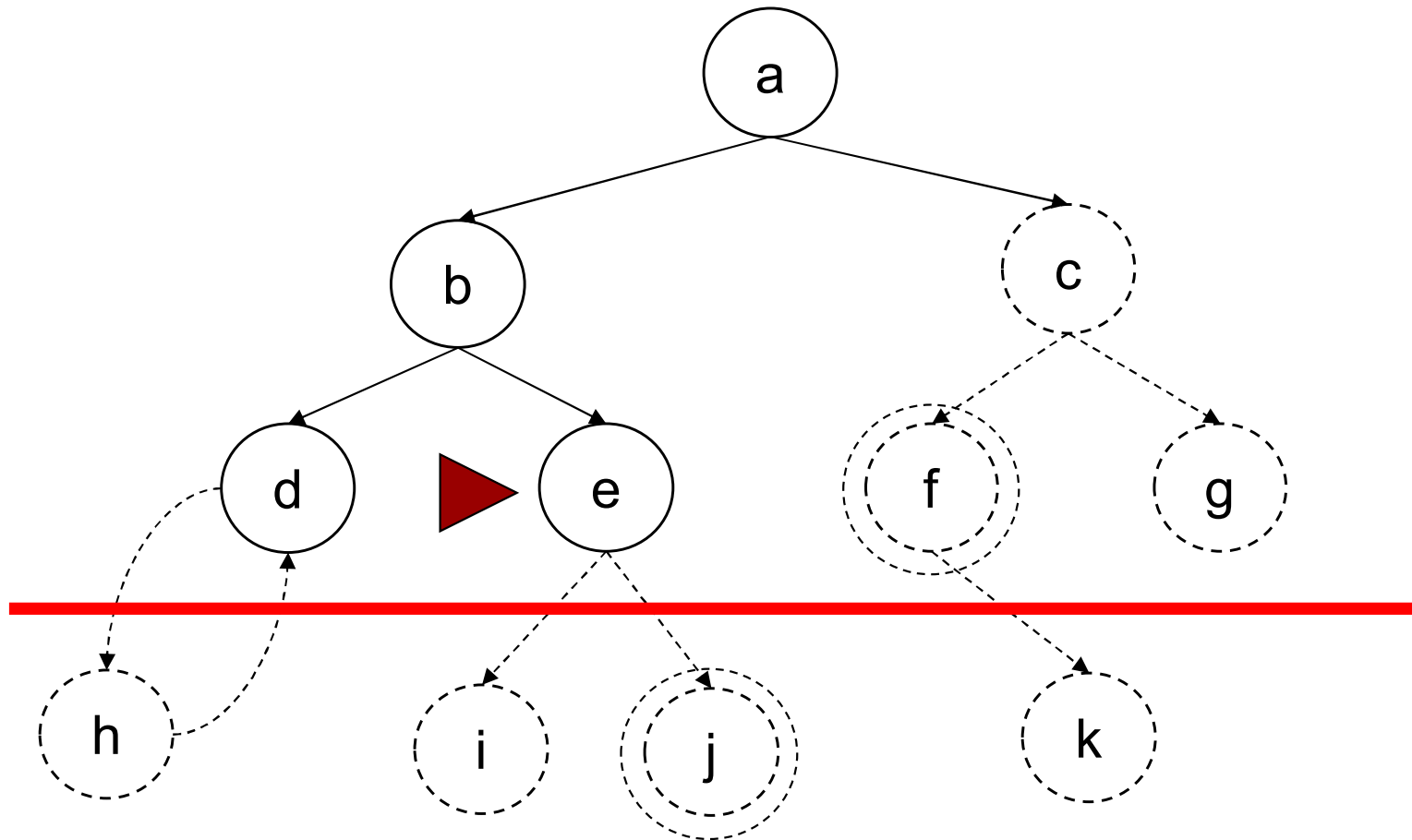
Inserir na frente, remover da frente: b, c

Busca em Profundidade Limitada ($L=2$)



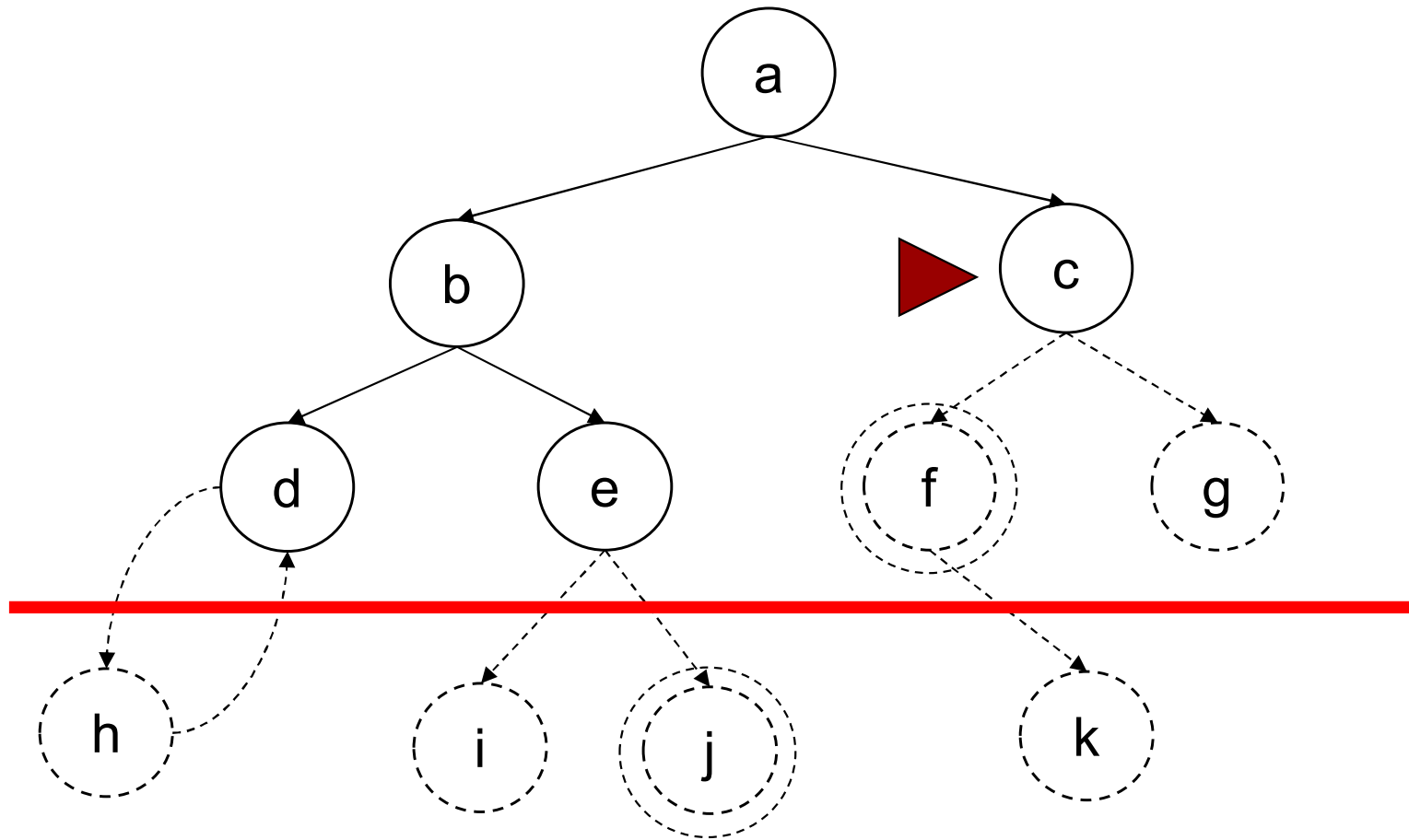
Inserir na frente, remover da frente: d, e, c

Busca em Profundidade Limitada ($L=2$)



Inserir na frente, remover da frente: e, c

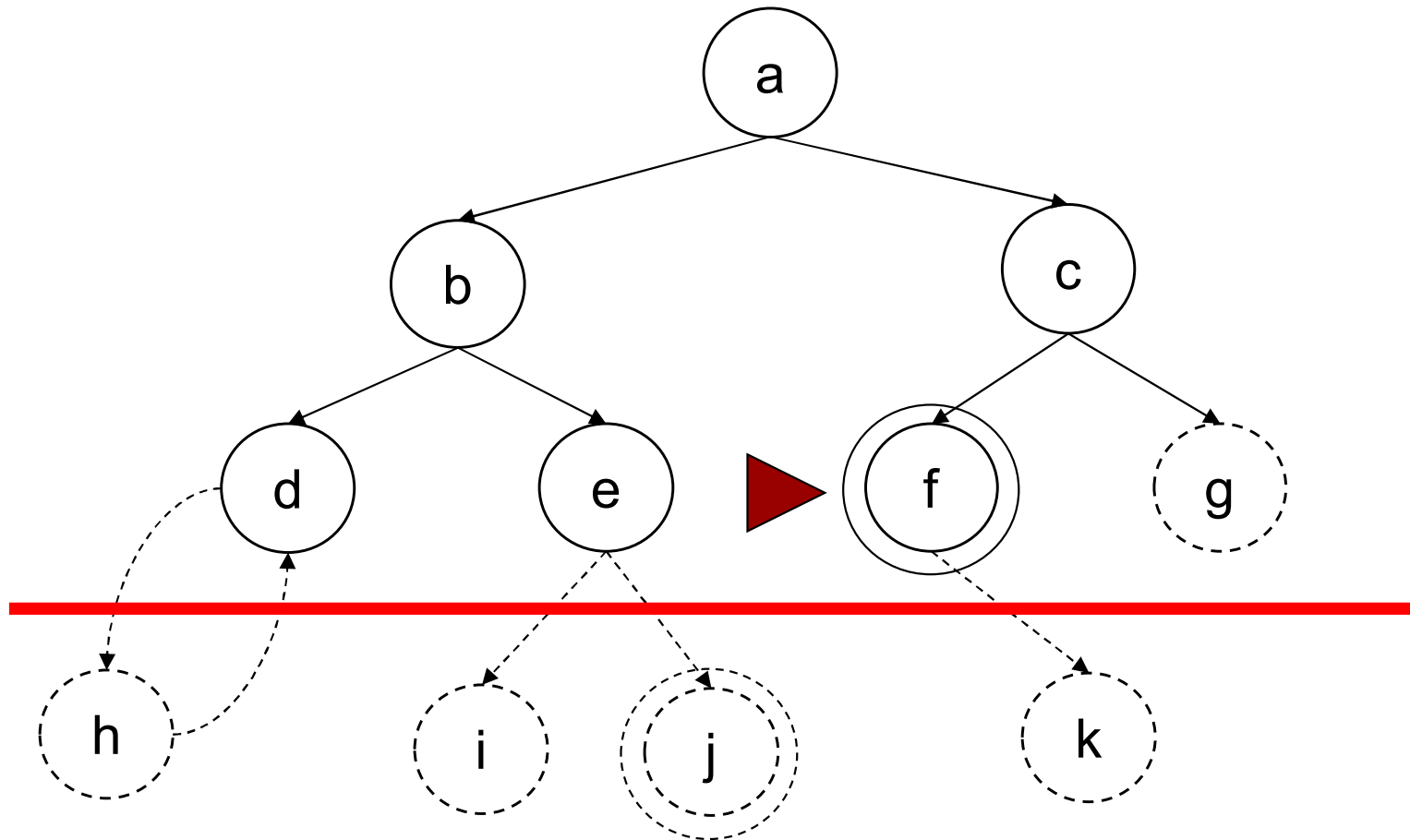
Busca em Profundidade Limitada ($L=2$)



Inserir na frente, remover da frente: c

Busca em Profundidade Limitada ($L=2$)

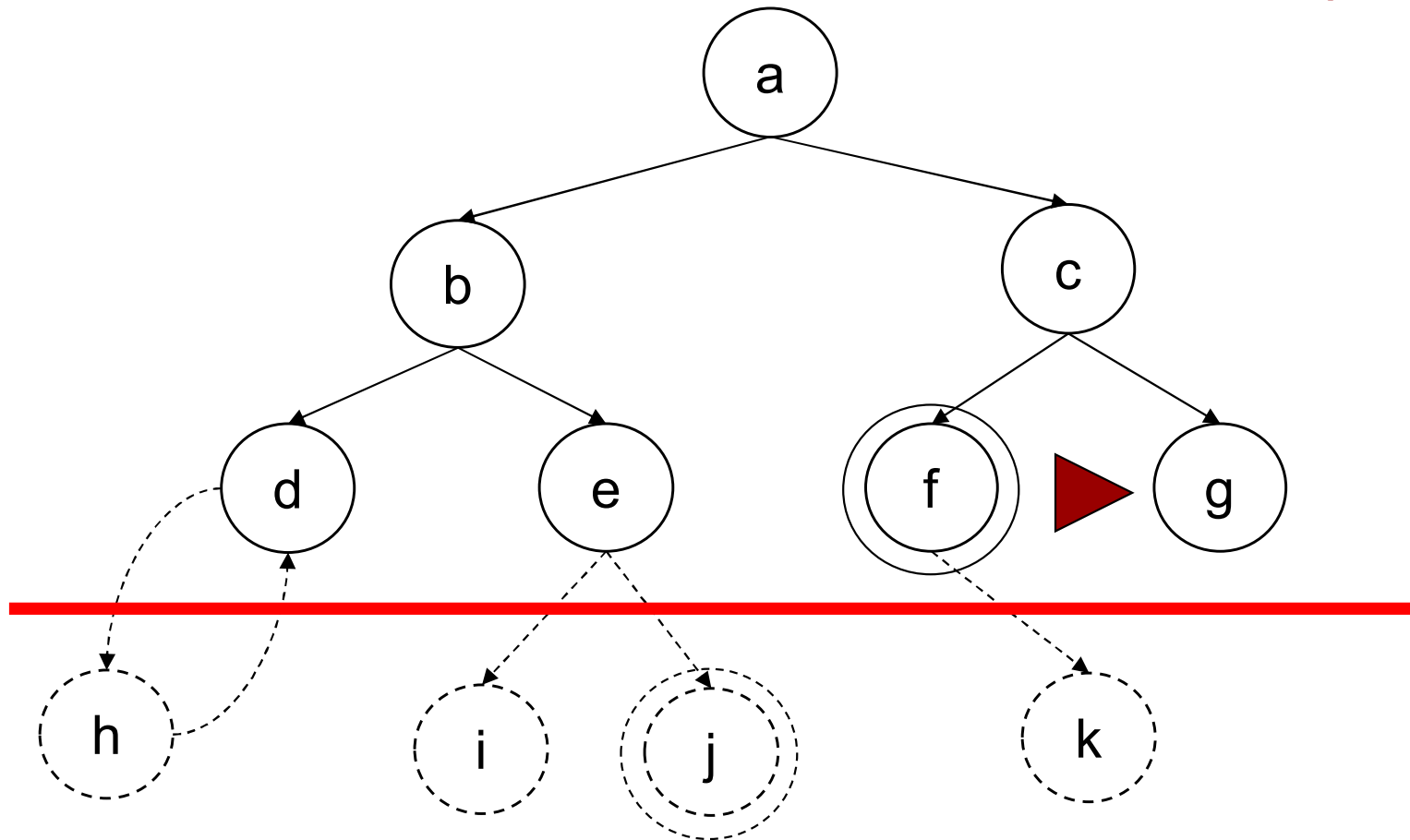
Solução encontrada para
 $L=2$: caminho [a,c,f]



Inserir na frente, remover da frente: f, g

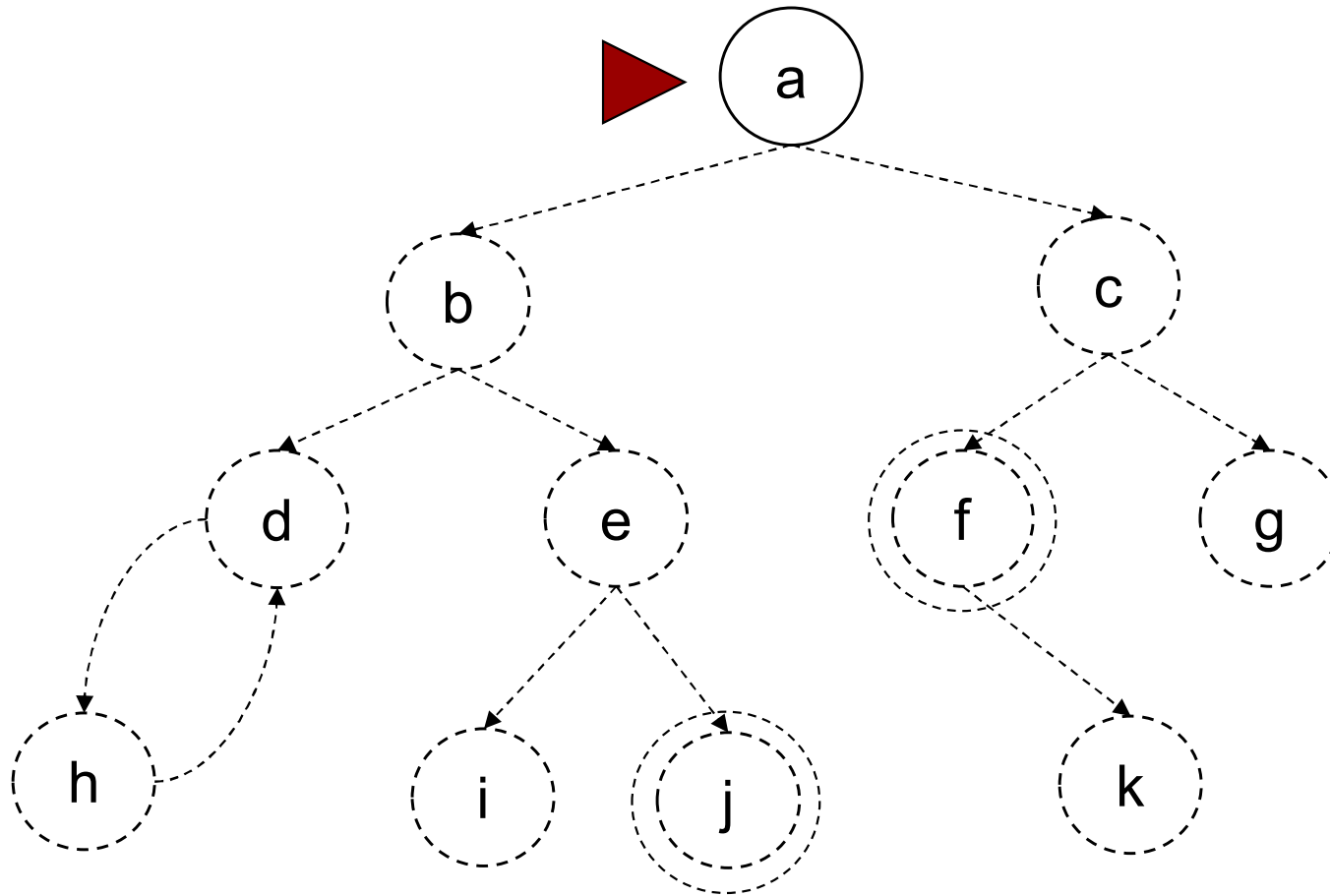
Busca em Profundidade Limitada ($L=2$)

Nenhuma solução adicional
para $L=2$



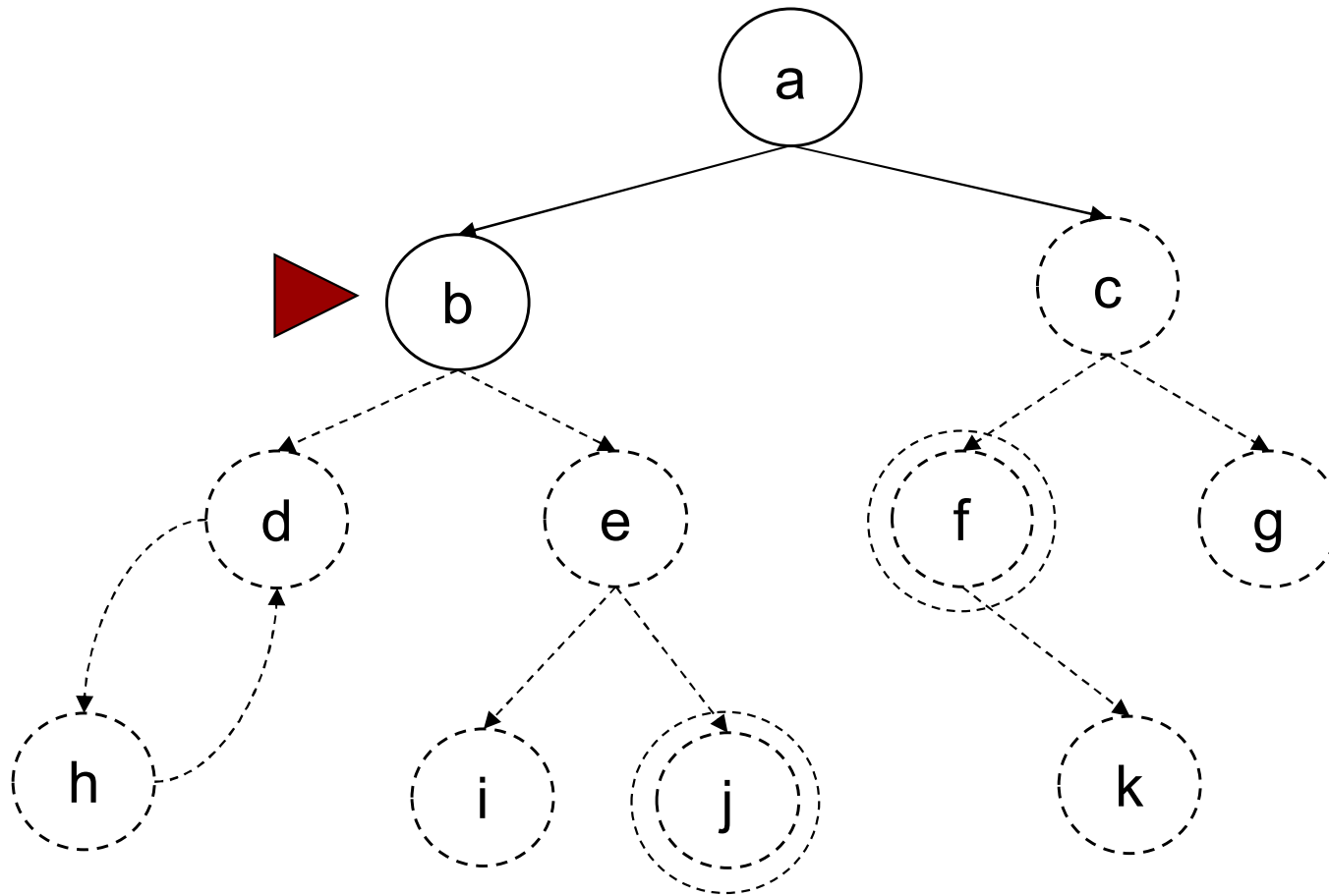
Inserir na frente, remover da frente: g

Busca em Profundidade Limitada ($L=3$)



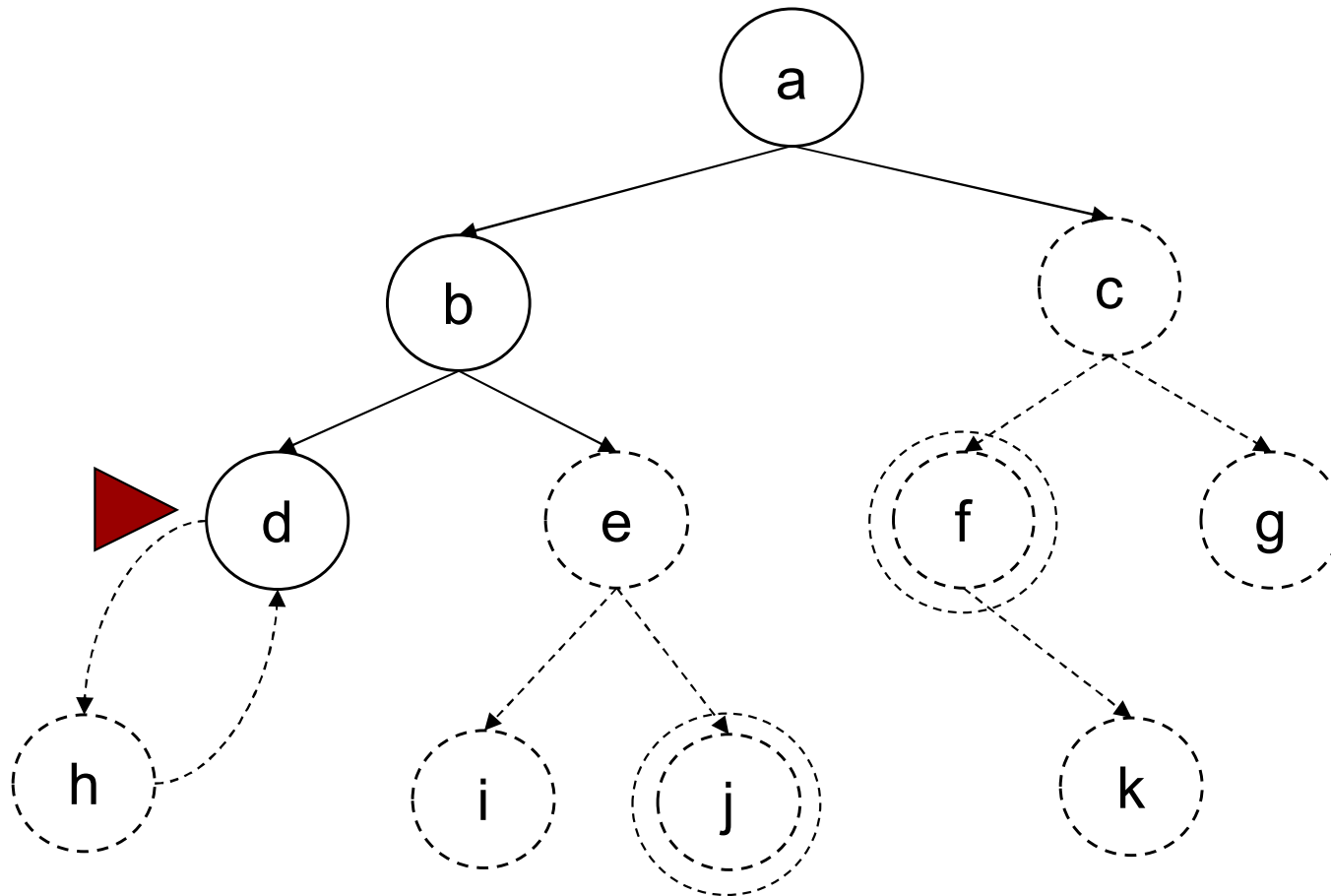
Inserir na frente, remover da frente: a

Busca em Profundidade Limitada ($L=3$)



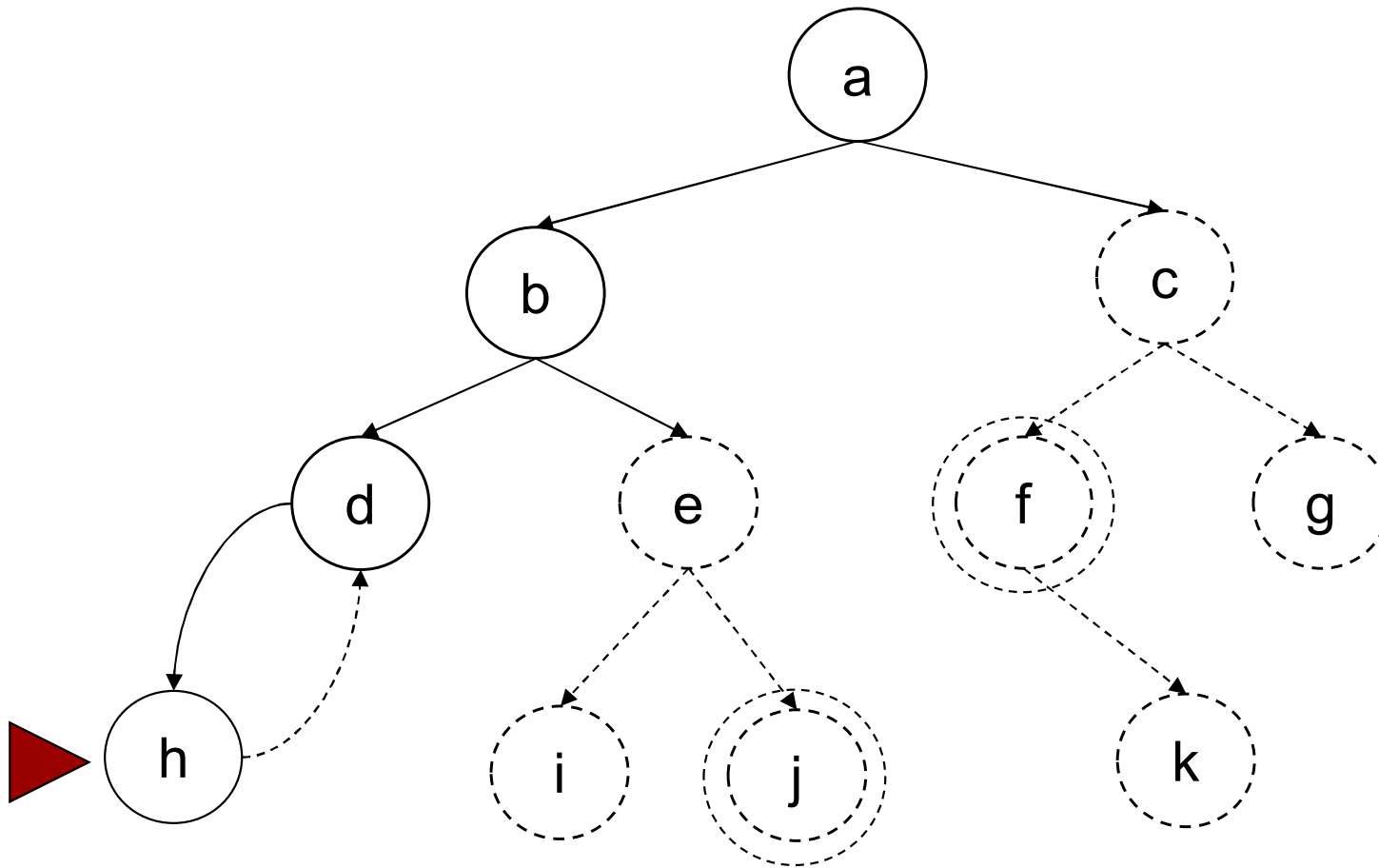
Inserir na frente, remover da frente: b, c

Busca em Profundidade Limitada ($L=3$)



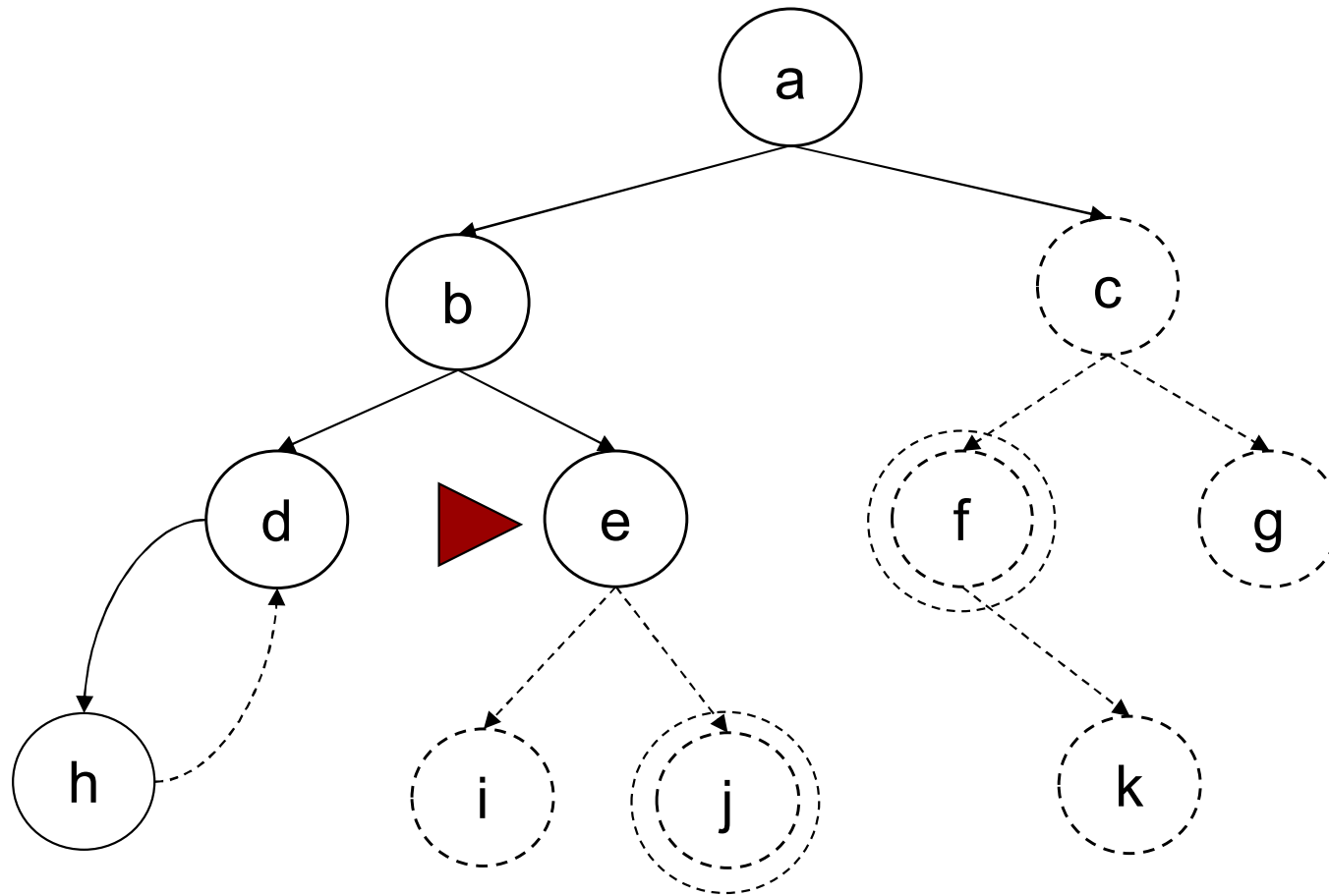
Inserir na frente, remover da frente: d, e, c

Busca em Profundidade Limitada ($L=3$)



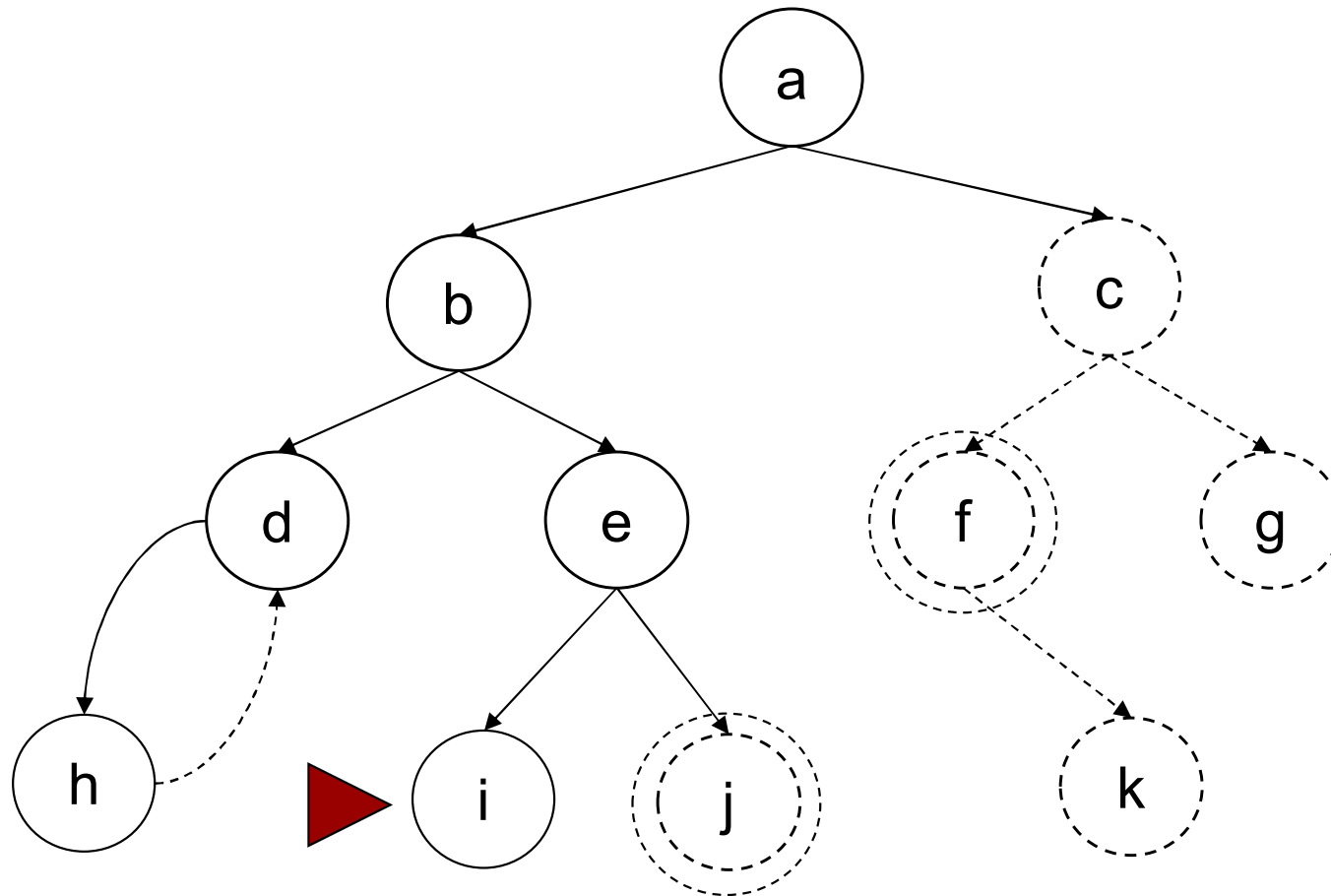
Inserir na frente, remover da frente: h, e, c

Busca em Profundidade Limitada ($L=3$)



Inserir na frente, remover da frente: e, c

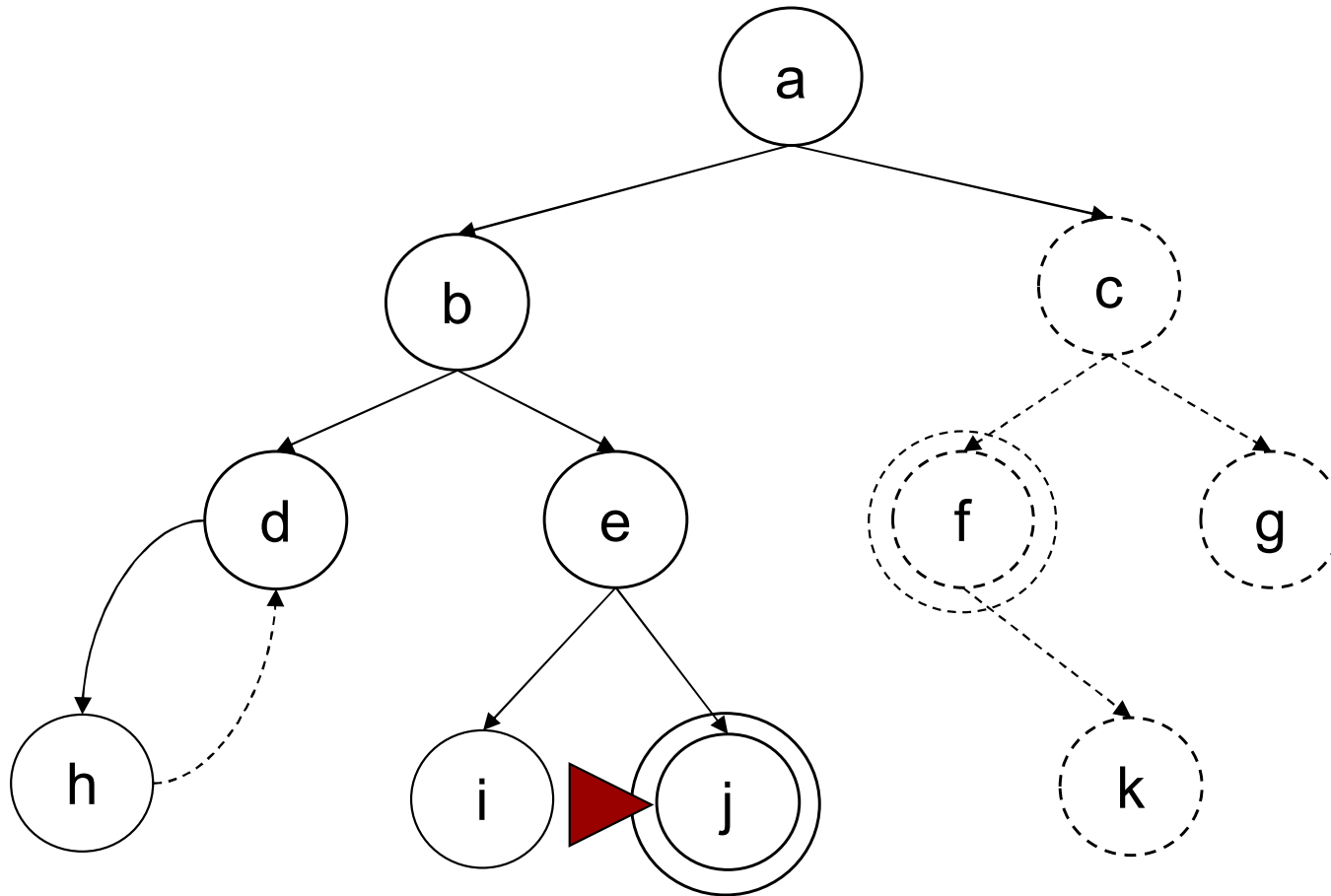
Busca em Profundidade Limitada ($L=3$)



Inserir na frente, remover da frente: i, j, c

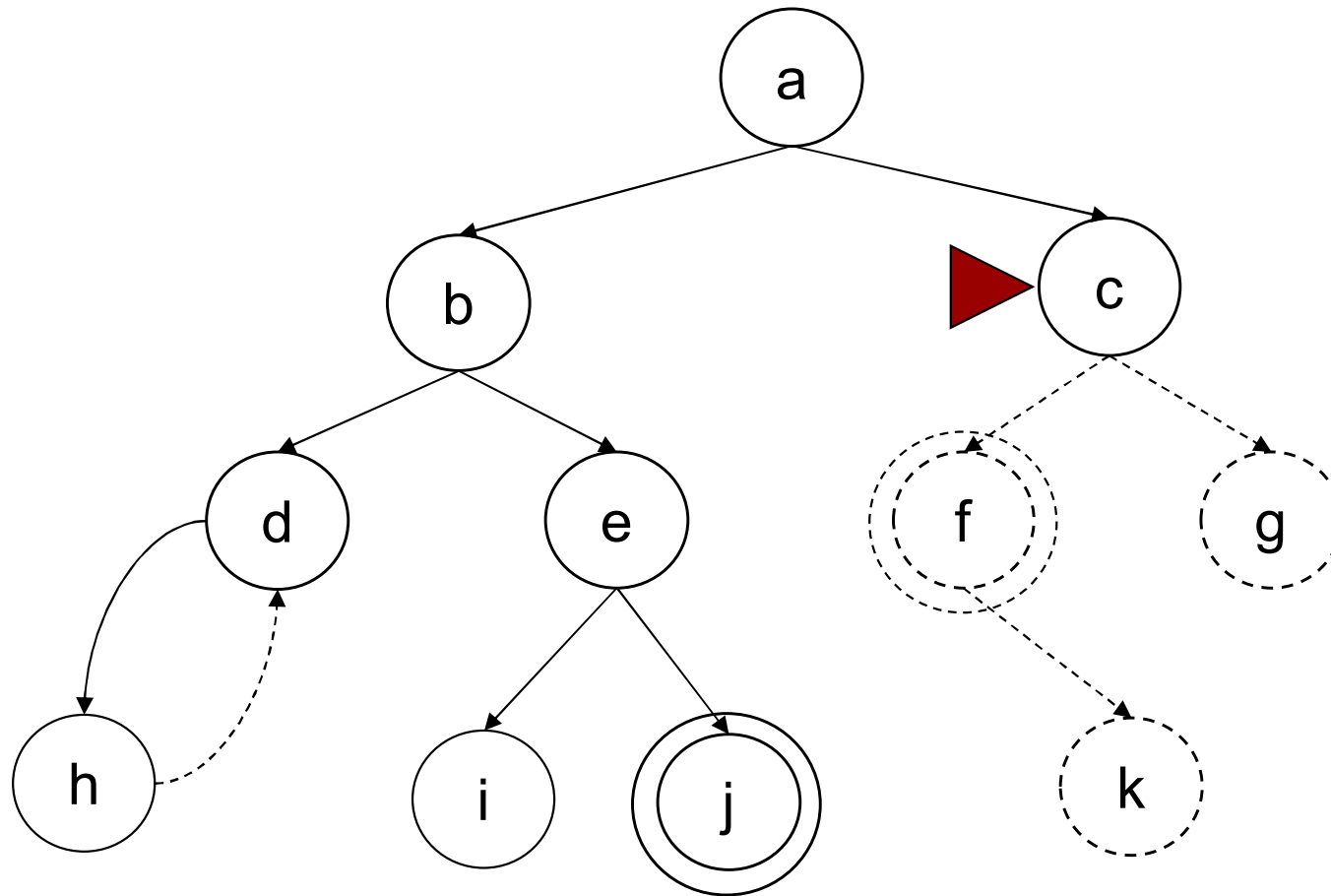
Busca em Profundidade Limitada (L=3)

1ª solução encontrada para
L=3: caminho [a,b,e,j]



Inserir na frente, remover da frente: j, c

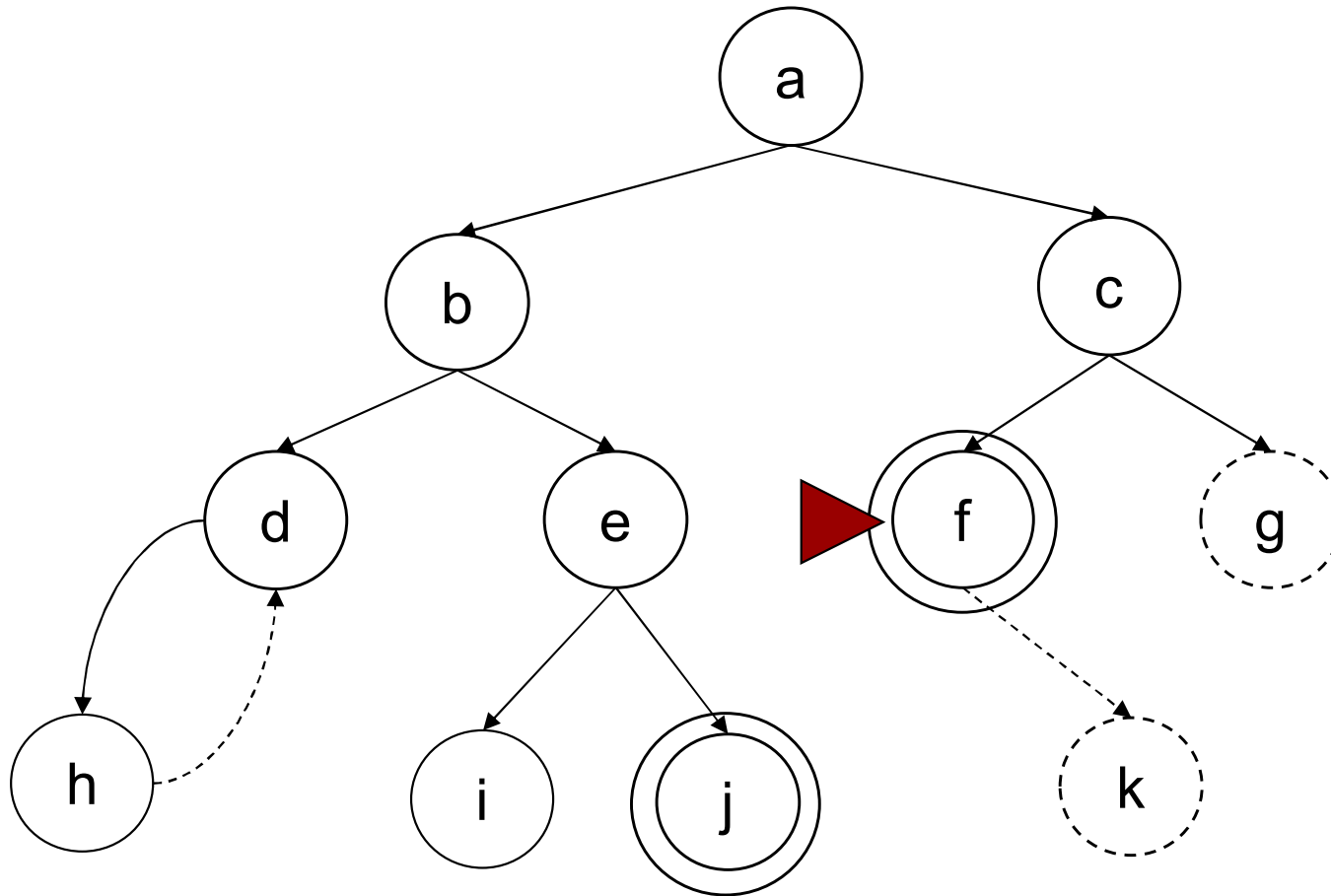
Busca em Profundidade Limitada ($L=3$)



Inserir na frente, remover da frente: c

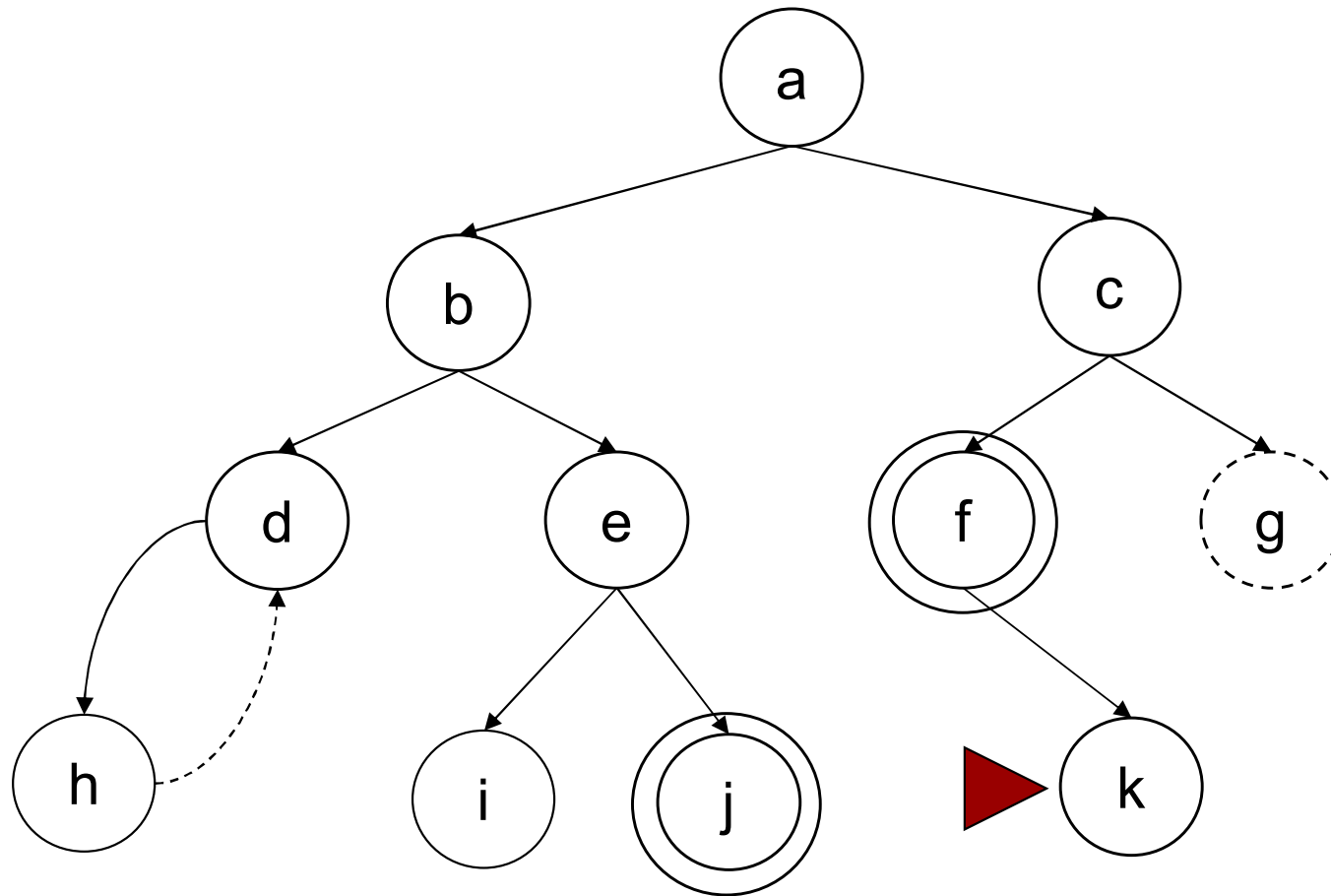
Busca em Profundidade Limitada (L=3)

2ª solução encontrada para
L=3: caminho [a,c,f]



Inserir na frente, remover da frente: f, g

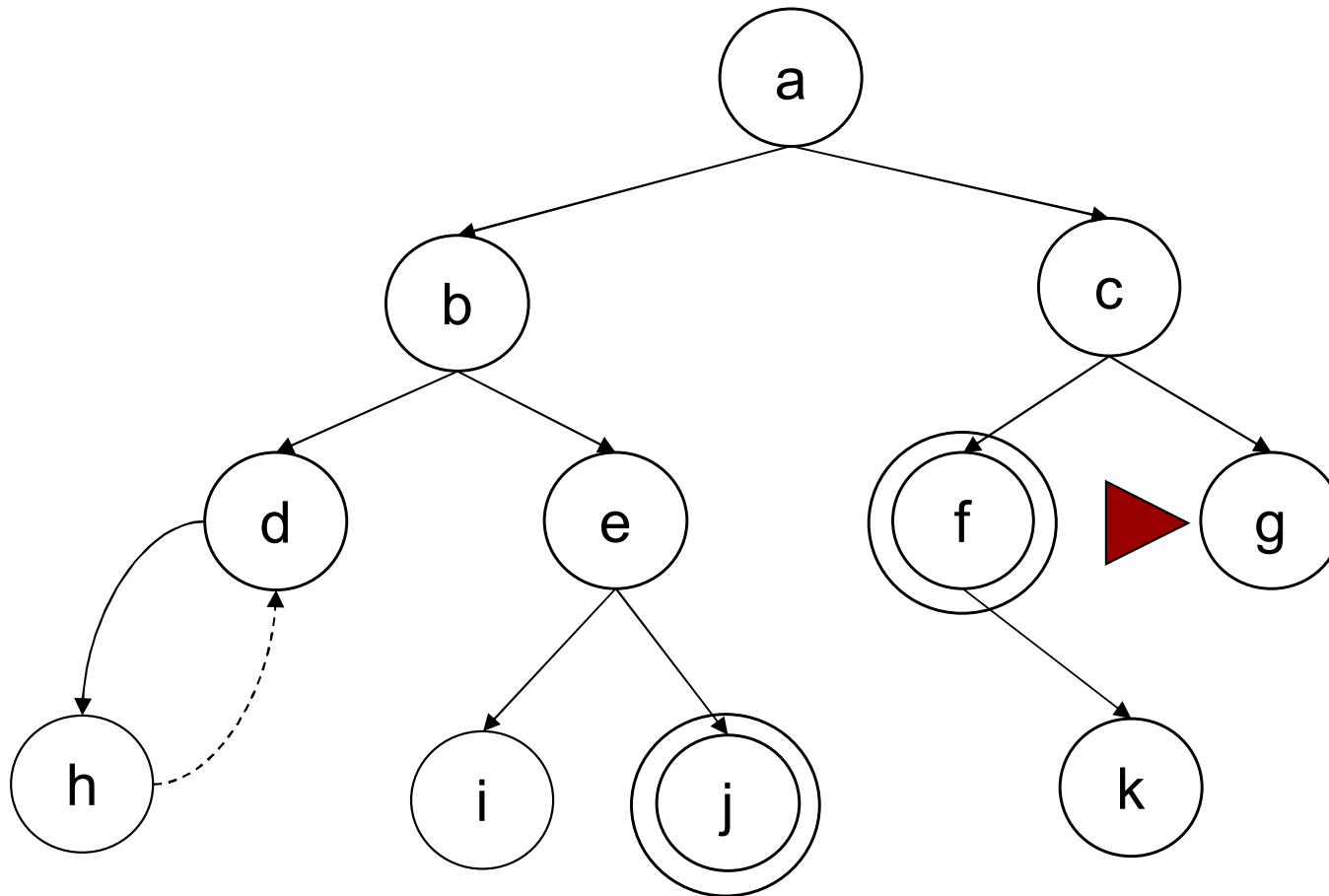
Busca em Profundidade Limitada ($L=3$)



Inserir na frente, remover da frente: k, g

Busca em Profundidade Limitada ($L=3$)

Nenhuma solução adicional
para $L=3$



Inserir na frente, remover da frente: g

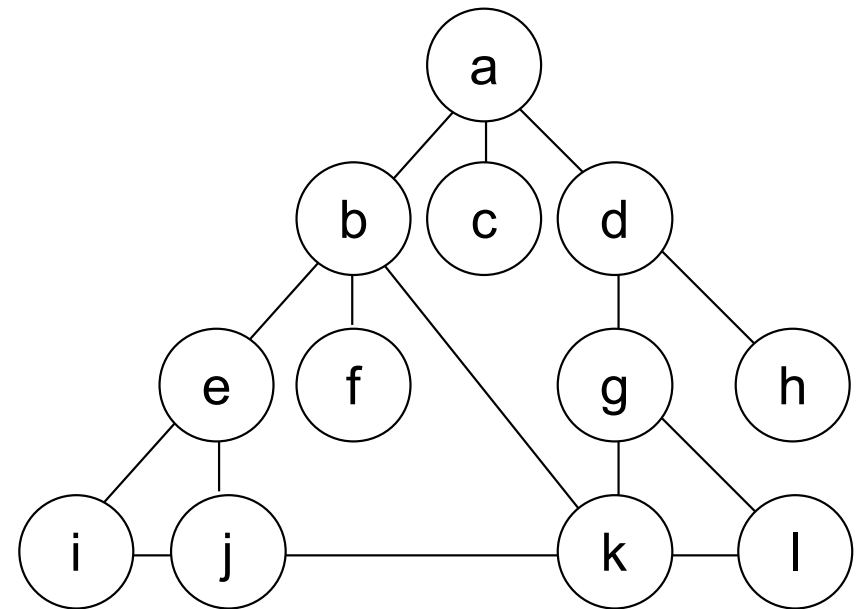
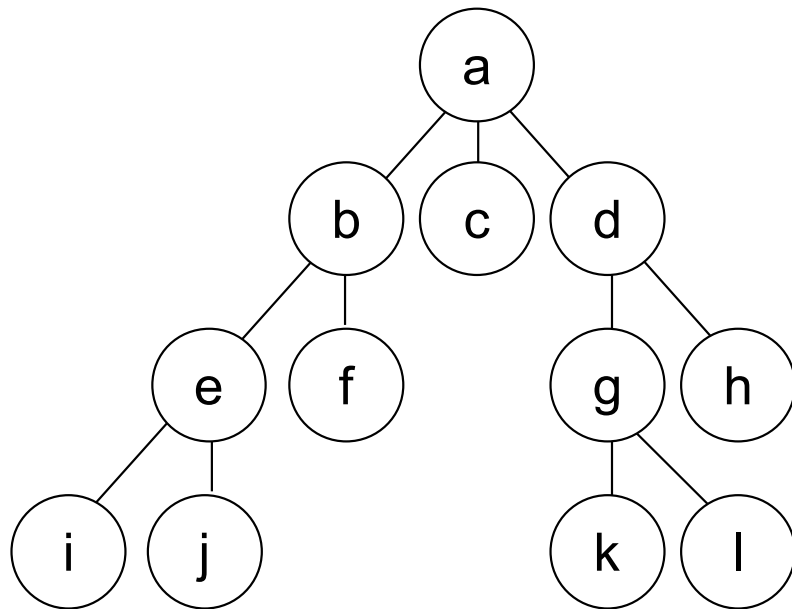
Busca em Profundidade Limitada

```
% resolva3(No,Solucao,L) Solucao é um caminho acíclico
% (na ordem reversa) entre nó inicial No uma solução
% L é o limite de profundidade da busca
resolva3(No,Solucao,L) :-
    depthFirstLimited([],No,Solucao,L).

% depthFirstLimited(Caminho,No,Solucao,L) estende o caminho
% [No|Caminho] até um nó final obtendo Solucao com
% profundidade não maior que L
depthFirstLimited(Caminho,No,[No|Caminho],_) :-
    final(No).
depthFirstLimited(Caminho,No,S,L) :-
    L > 0,
    s(No,No1),
    \+ pertence(No1,Caminho),           % evita um ciclo
    L1 is L - 1,
    depthFirstLimited([No|Caminho],No1,S,L1).
```

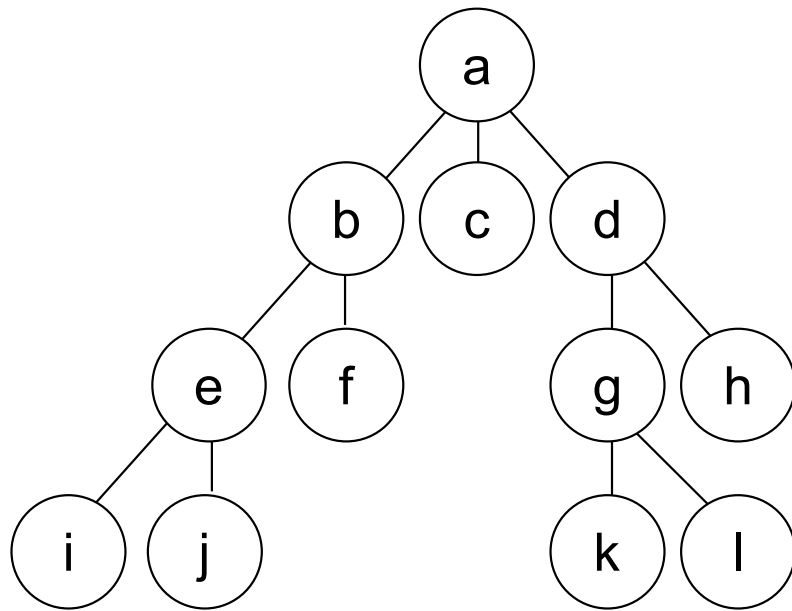
Exercício: Indique a ordem na qual os nós são visitados na busca em profundidade limitada ($L = 0, 1, 2$ e 3)

Assuma que os sucessores de um nó são definidos em ordem alfabética



Solução

Assuma que os sucessores de um nó são definidos em ordem alfabética

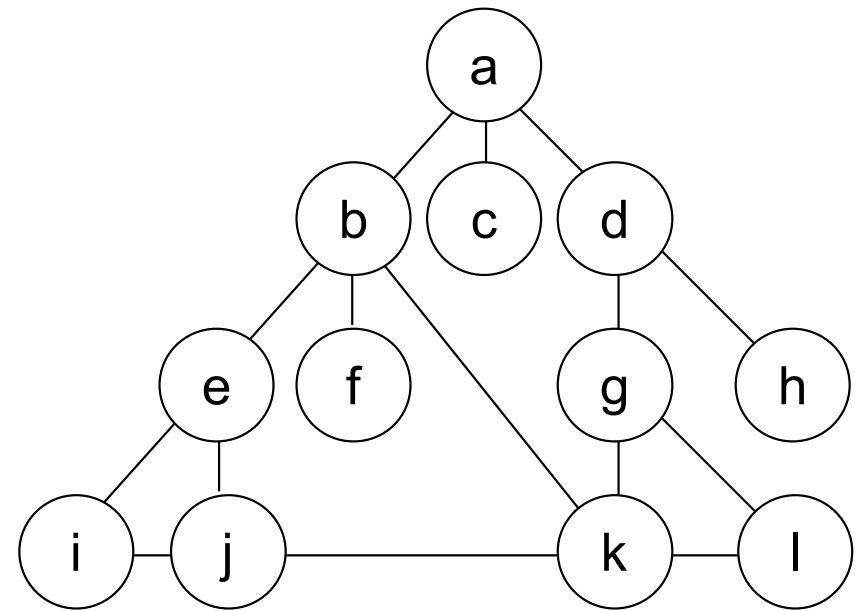


L=0: a

L=1: a,b,c,d

L=2: a,b,e,f,c,d,g,h

L=3: a,b,e,i,j,f,c,d,g,k,l,h



L=0: a

L=1: a,b,c,d

L=2: a,b,e,f,k,c,d,g,h

L=3: a,b,e,i,j,f,k,g,l,c,d,h

Busca em Profundidade Limitada

- ❑ Um problema com a busca em profundidade limitada é que não se tem previamente um limite razoável
 - Se o limite for muito pequeno (menor que qualquer caminho até uma solução) então a busca falha
 - Se o limite for muito grande, a busca se torna muito complexa
- ❑ Para resolver este problema a busca em profundidade limitada pode ser executada de forma iterativa, variando o limite: comece com um limite de profundidade pequeno e aumente gradualmente o limite até que uma solução seja encontrada
- ❑ Esta técnica é denominada busca em profundidade iterativa e pode ser implementada chamando o procedimento `depthFirstLimited/4` a partir de outro procedimento que, em cada chamada recursiva, incrementa o limite em uma unidade

Busca em Profundidade Iterativa

```
% resolve4(No,Solucao) Solucao é um caminho acíclico
% (na ordem reversa) entre nó inicial No uma solução
:- dynamic proximo_limite/1.

resolve4(No,Solucao) :-
    retract(proximo_limite(_)), % limpa proximo limite
    fail
;
assert(proximo_limite(0)), % inicializa proximo limite
depthFirstIterativeDeepening(No,Solucao).

depthFirstIterativeDeepening(No,Solucao) :-
    retract(proximo_limite(Limite)),
    NovoLimite is Limite + 1,
    assert(proximo_limite(NovoLimite)),
    depthFirstLimited([],No,Solucao,Limite).
depthFirstIterativeDeepening(No,Solucao) :-
    proximo_limite(Limite),
    Limite < 9999, % constante arbitrária
    depthFirstIterativeDeepening(No,Solucao).
```

Busca em Profundidade Iterativa (versão sem assert/retract)

```
% resolva41(No,Solucao) Solucao é um caminho acíclico
% (na ordem reversa) entre nó inicial No uma solução
resolva41(No,Solucao) :-
    depthFirstIterativeDeepening(No, Solucao, 0).

depthFirstIterativeDeepening(No,Solucao,Limite) :-
    depthFirstLimited([],No,Solucao,Limite).
depthFirstIterativeDeepening(No,Solucao,Limite) :-
    NovoLimite is Limite + 1,
    NovoLimite < 9999,           % constante arbitrária
    depthFirstIterativeDeepening(No,Solucao,NovoLimite).
```

Busca em Profundidade Iterativa

- Há uma implementação alternativa baseada no predicado **path(No1,No2,Caminho)** onde Caminho é um caminho acíclico (na ordem reversa) entre os nós No1 e No2 no espaço de estados

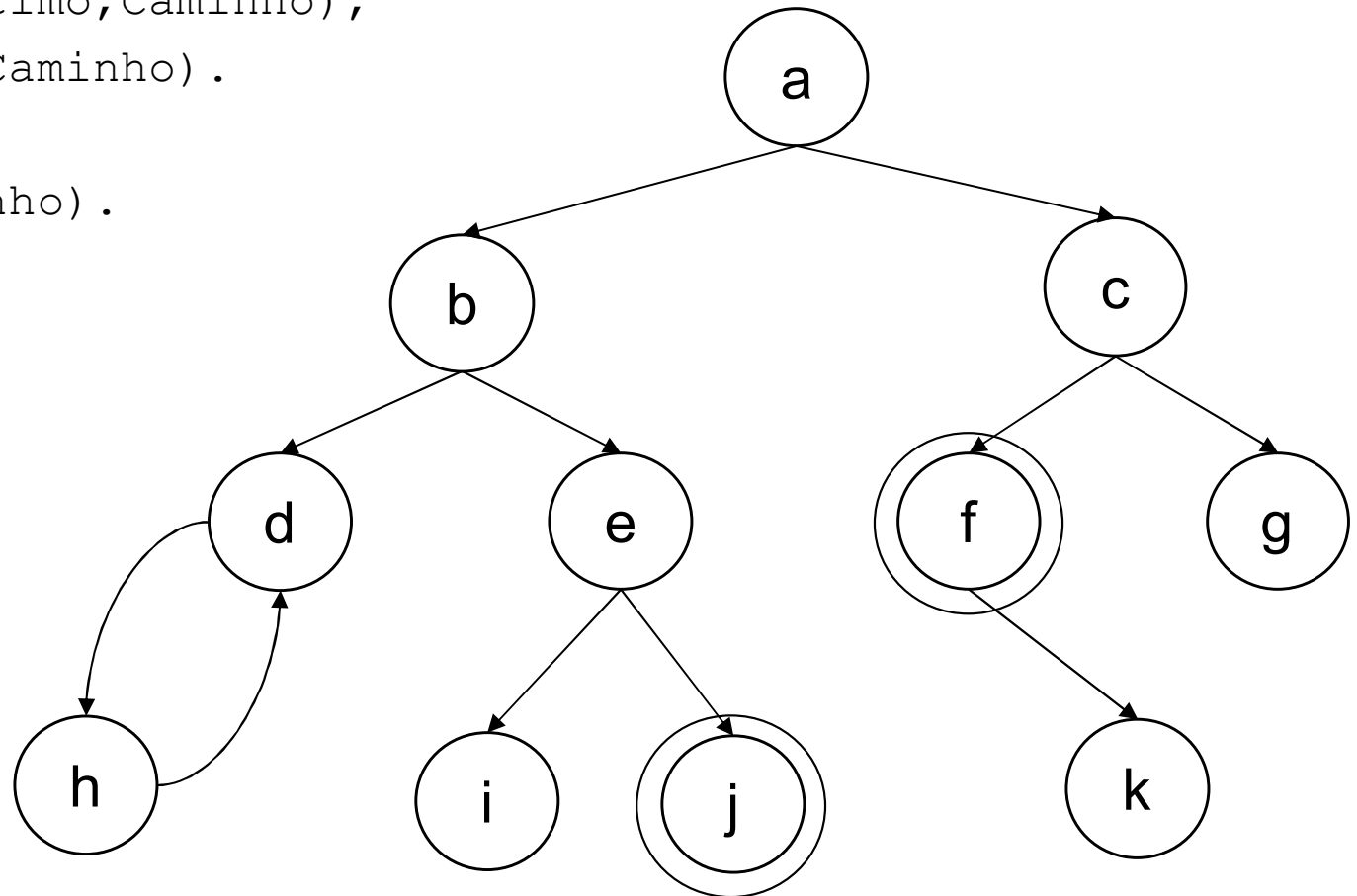
```
path(No,No,[No]).                % caminho com um único nó
path(Primeiro,Ultimo,[Ultimo|Caminho]) :-
    s(Penultimo,Ultimo),          % Há nó anterior ao último
    path(Primeiro,Penultimo,Caminho), % Há caminho até penúltimo
    \+ pertence(Ultimo,Caminho).    % evita um ciclo
```

Busca em Profundidade Iterativa

```
path(No,No,[No]).  
path(Primeiro,Ultimo,[Ultimo|Caminho]) :-  
    s(Penultimo,Ultimo),  
    path(Primeiro,Penultimo,Caminho),  
    \+ pertence(Ultimo,Caminho).
```

```
?- path(a,Ultimo,Caminho).
```

```
Ultimo = a  
Caminho = [a];  
Ultimo = b  
Caminho = [b,a];  
Ultimo = c  
Caminho = [c,a];  
Ultimo = d  
Caminho = [d,b,a];  
...
```



Busca em Profundidade Iterativa (outra versão)

```
% resolva42(No,Solucao) Solucao é um caminho acíclico
% (na ordem reversa) entre nó inicial No uma solução
resolva42(No,Solucao) :-
    depthFirstIterativeDeepening(No,Solucao).
```

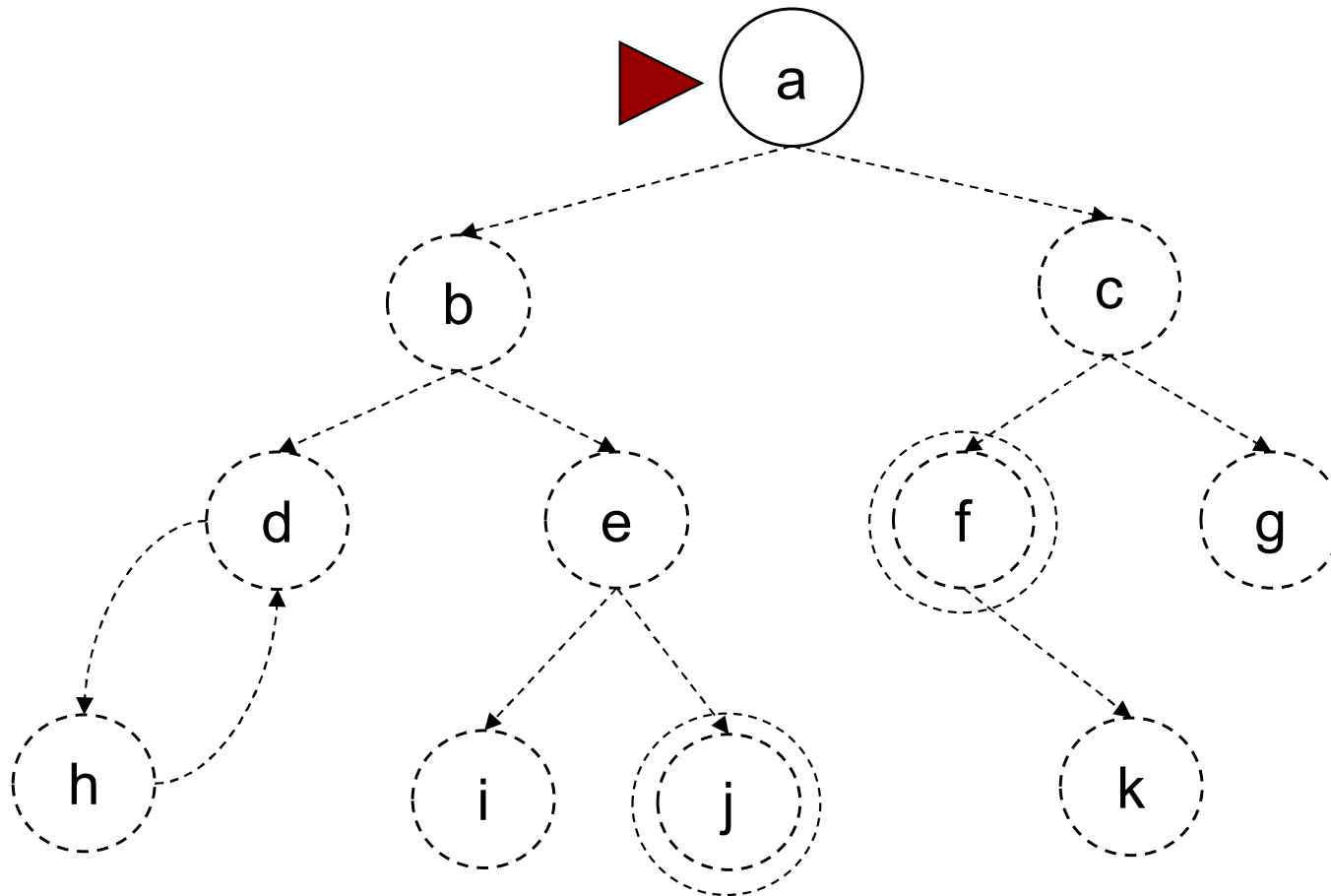
```
% path(No1,No2,Caminho) encontra Caminho acíclico entre No1 e No2
path(No,No,[No]). % caminho com um único nó
path(Primeiro,Ultimo,[Ultimo|Caminho]) :-
    s(Penultimo,Ultimo), % Há nó anterior ao último
    path(Primeiro,Penultimo,Caminho), % Há caminho até penúltimo
    \+ pertence(Ultimo,Caminho). % evita um ciclo
```

```
% depthFirstIterativeDeepening(No,Solução) iterativamente
% aumente a profundidade do caminho
depthFirstIterativeDeepening(No,Solucao) :-
    path(No,Final,Solucao),
    final(Final).
```

Busca em Largura

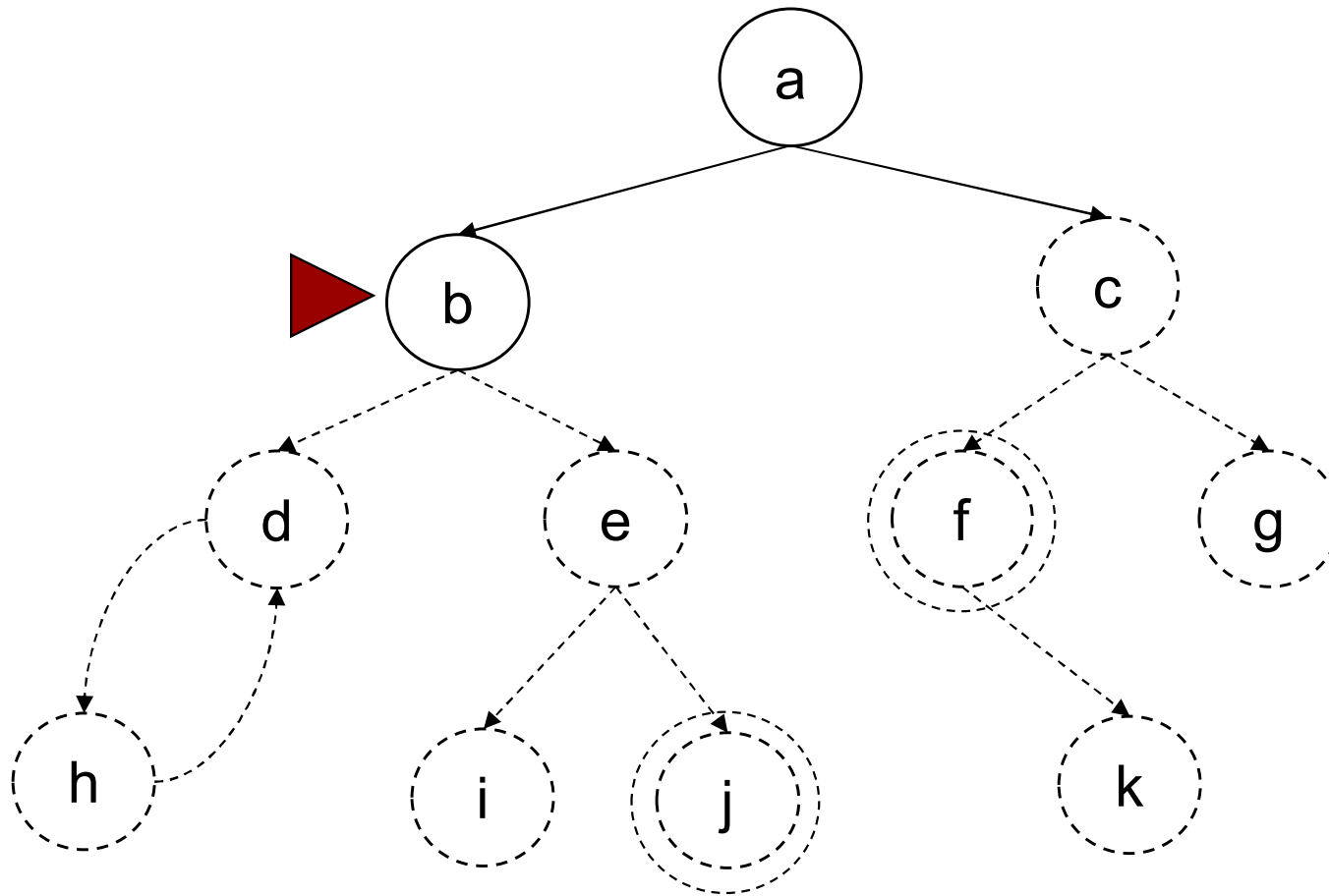
- ❑ Em contraste com a busca em profundidade, a busca em largura escolhe primeiro visitar aqueles nós mais próximos do nó inicial
- ❑ O algoritmo não é tão simples, pois é necessário manter um **conjunto** de nós candidatos alternativos e não apenas um único, como na busca em profundidade
- ❑ O conjunto é todo o nível inferior da árvore de busca
- ❑ Além disso, só o conjunto é insuficiente se o caminho da solução também for necessário
- ❑ Assim, ao invés de manter um conjunto de nós candidatos, é necessário manter um conjunto de caminhos candidatos

Busca em Largura



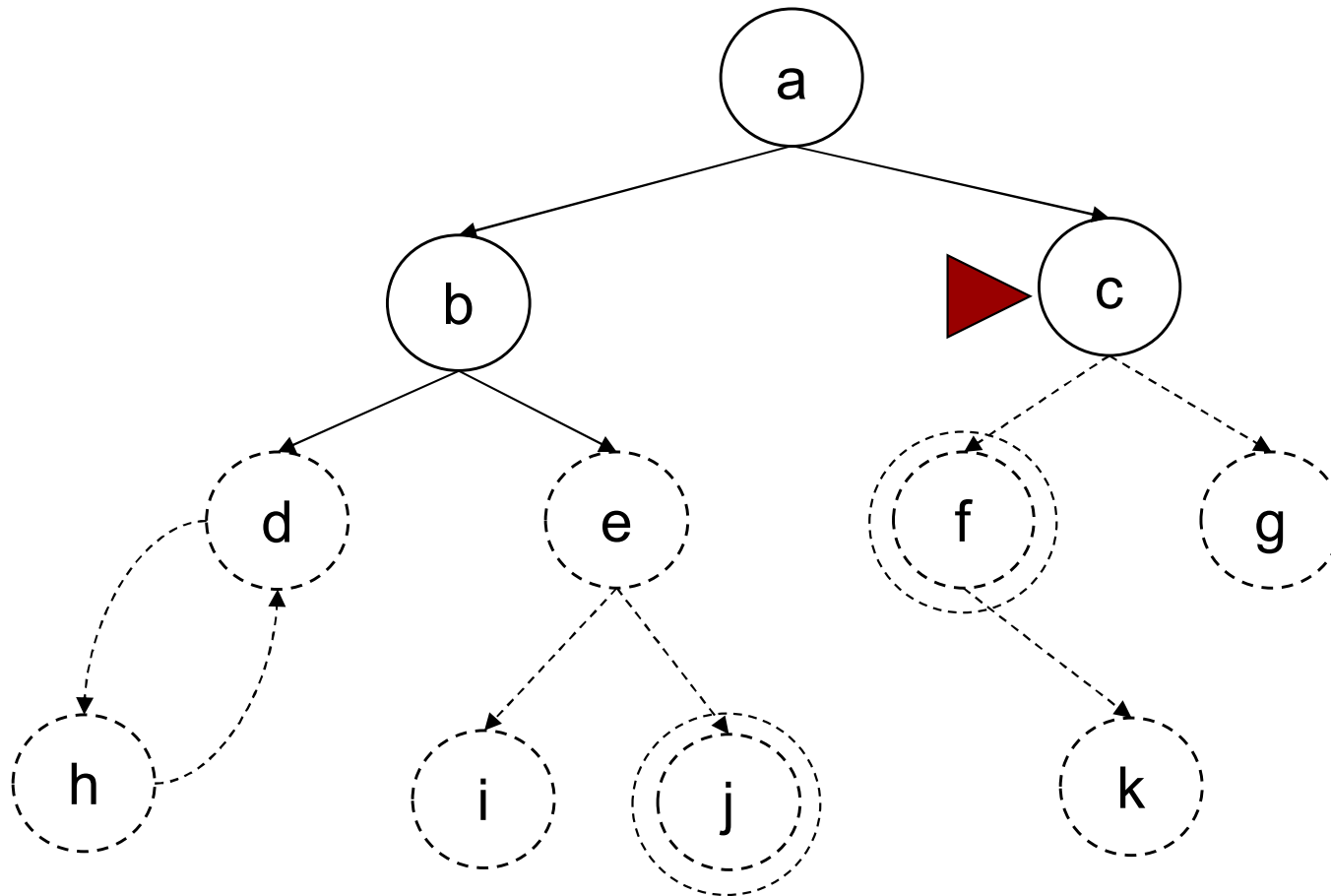
Inserir no final, remover da frente: a

Busca em Largura



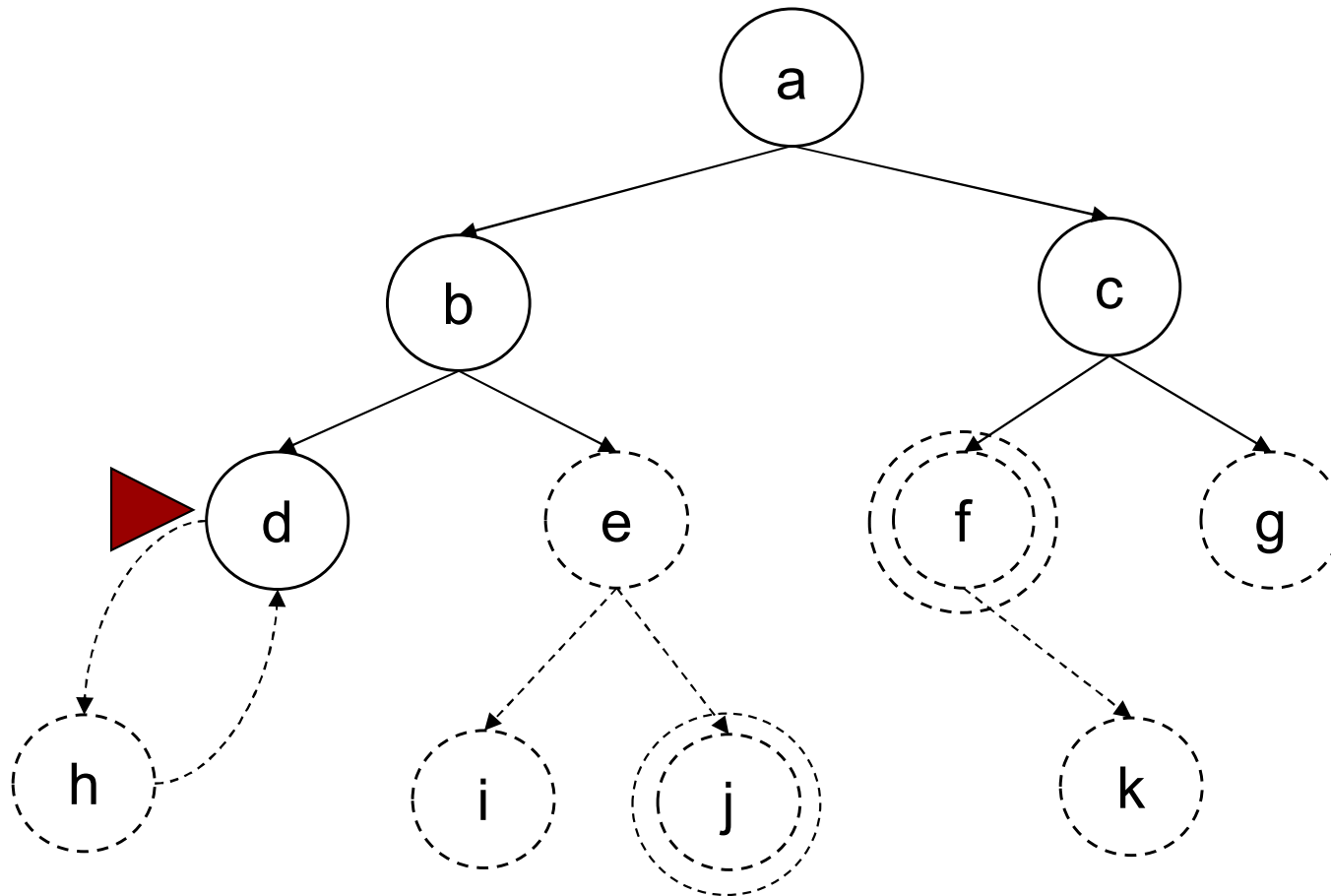
Inserir no final, remover da frente: b, c

Busca em Largura



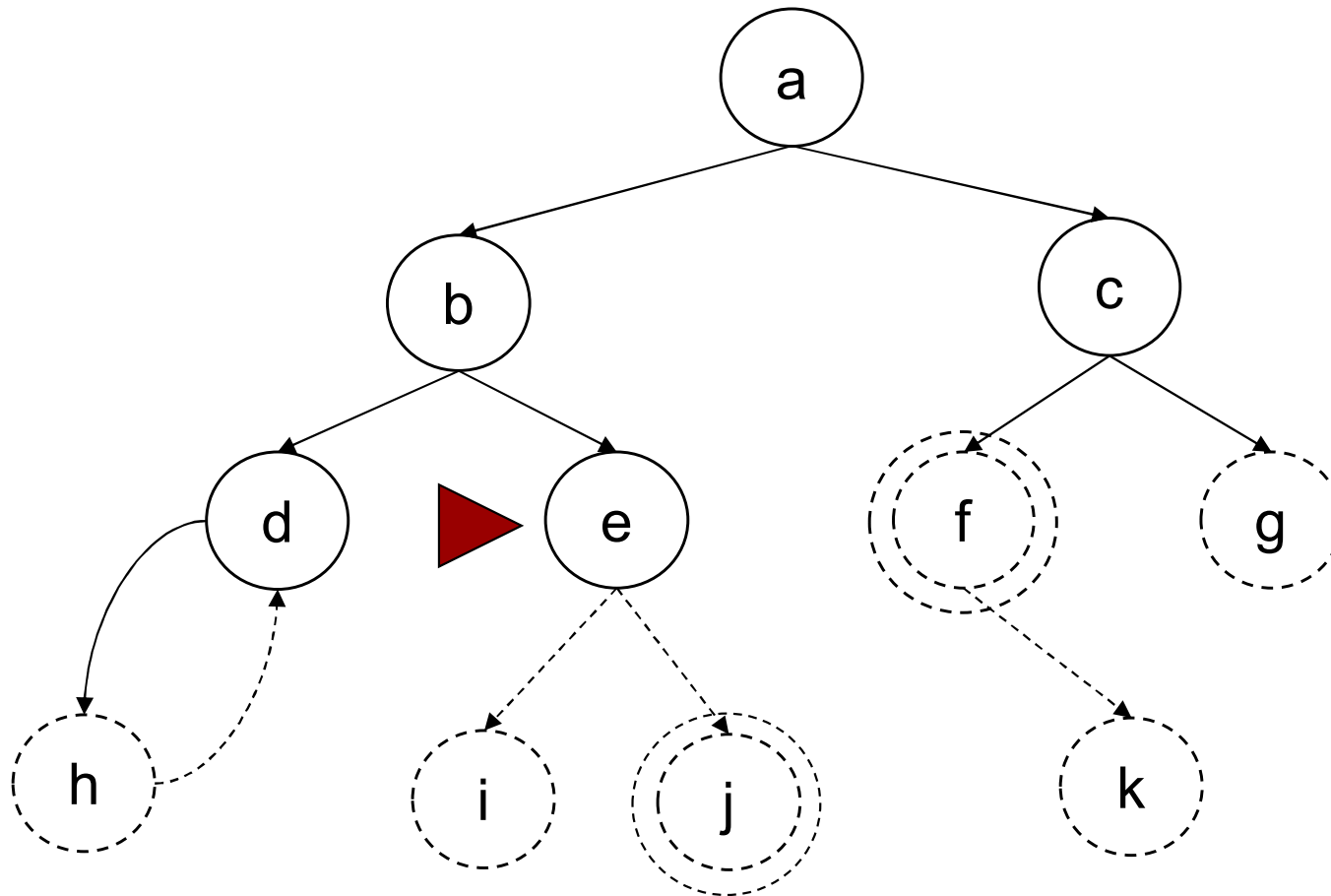
Inserir no final, remover da frente: c, d, e

Busca em Largura



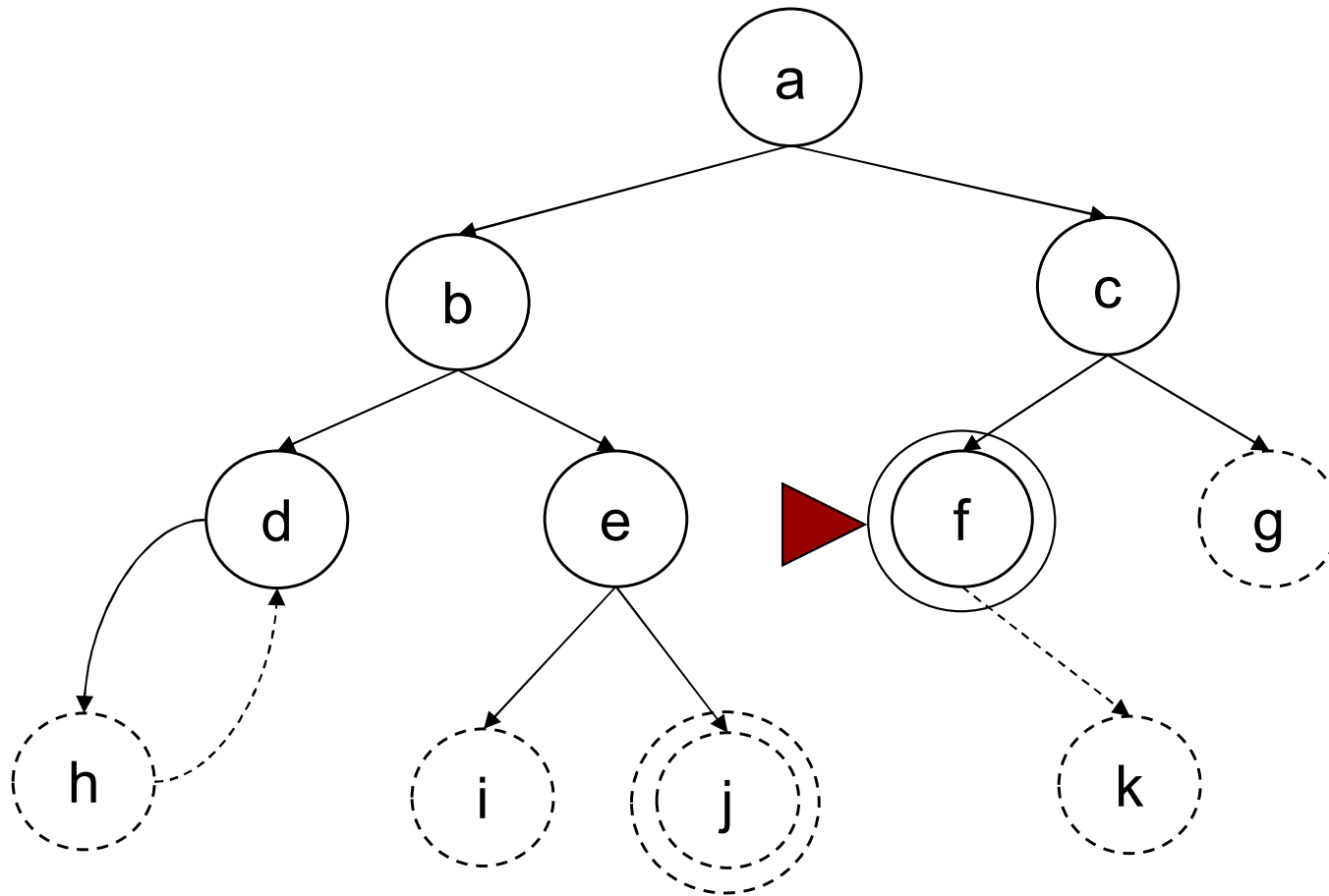
Inserir no final, remover da frente: d, e, f, g

Busca em Largura



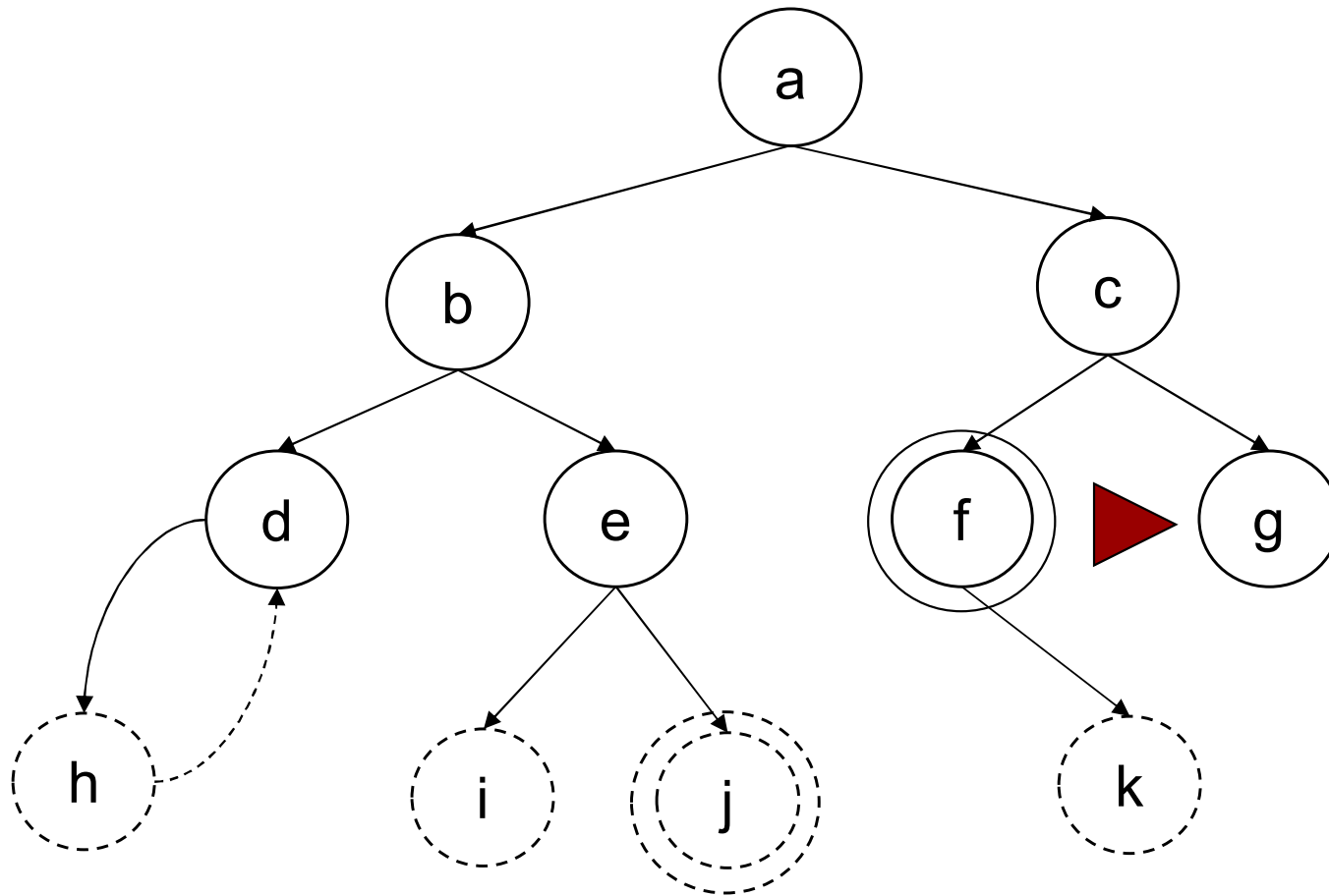
Inserir no final, remover da frente: e, f, g, h

Busca em Largura



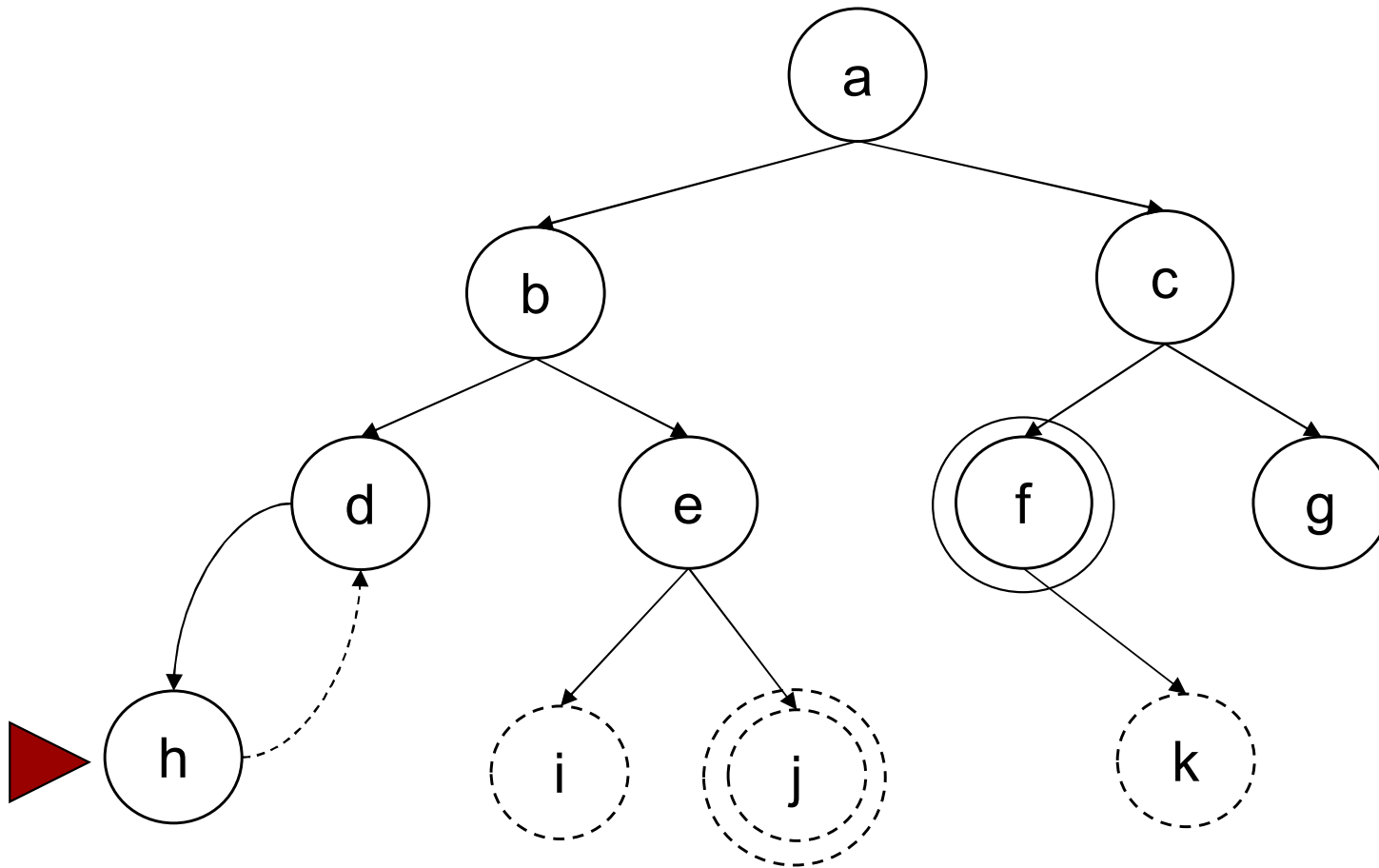
Inserir no final, remover da frente: f, g, h, i, j

Busca em Largura



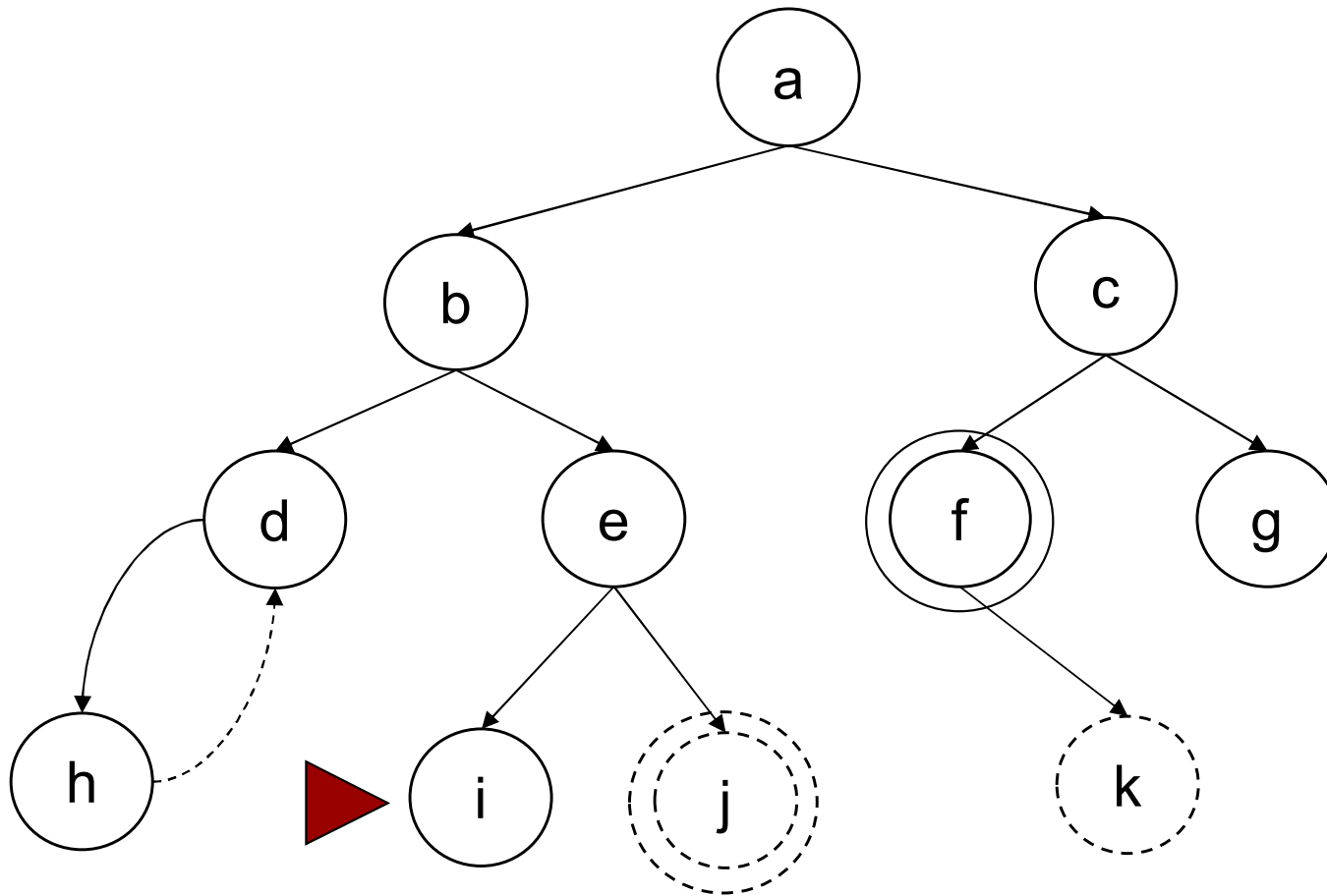
Inserir no final, remover da frente: g, h, i, j, k

Busca em Largura



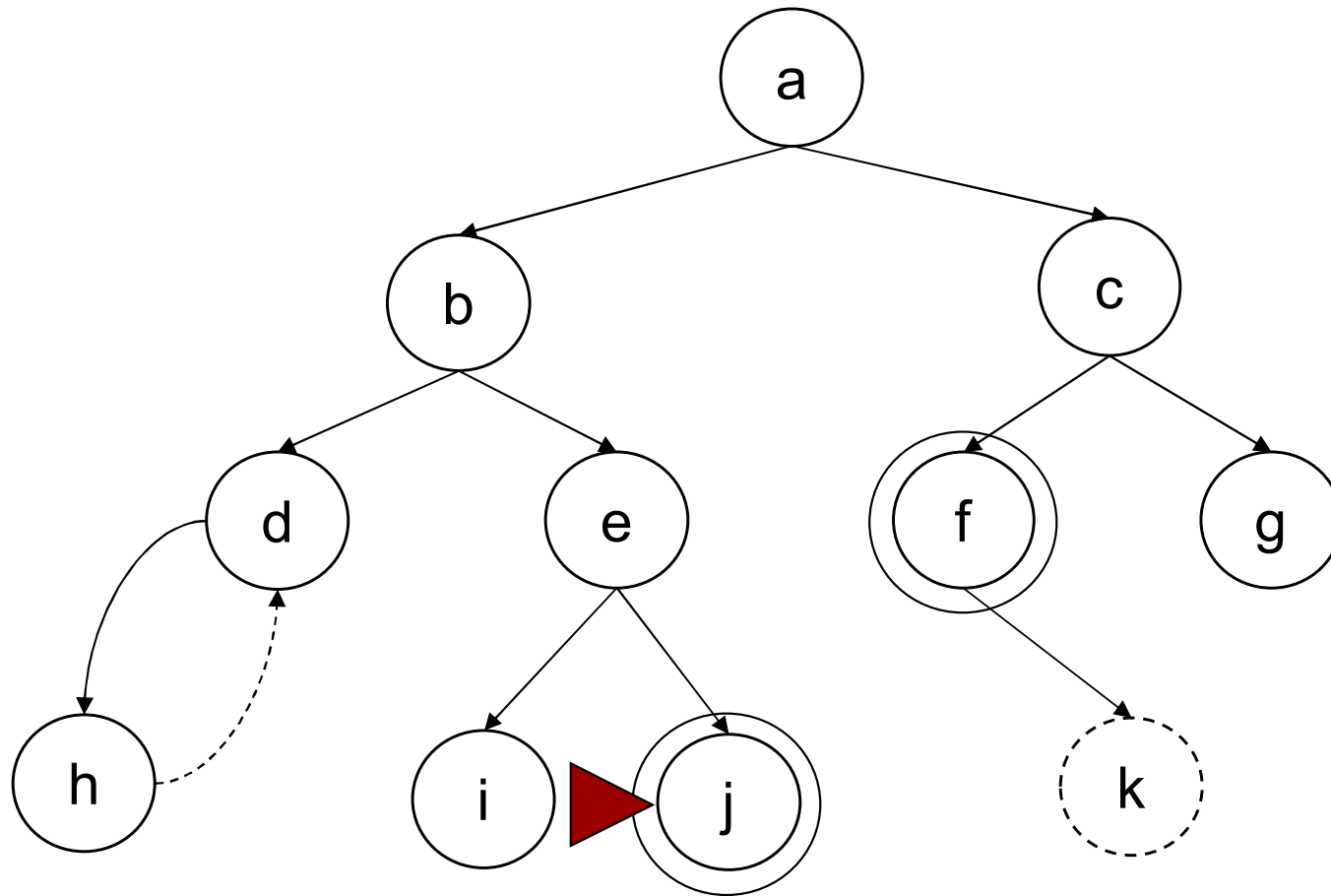
Inserir no final, remover da frente: h, i, j, k

Busca em Largura



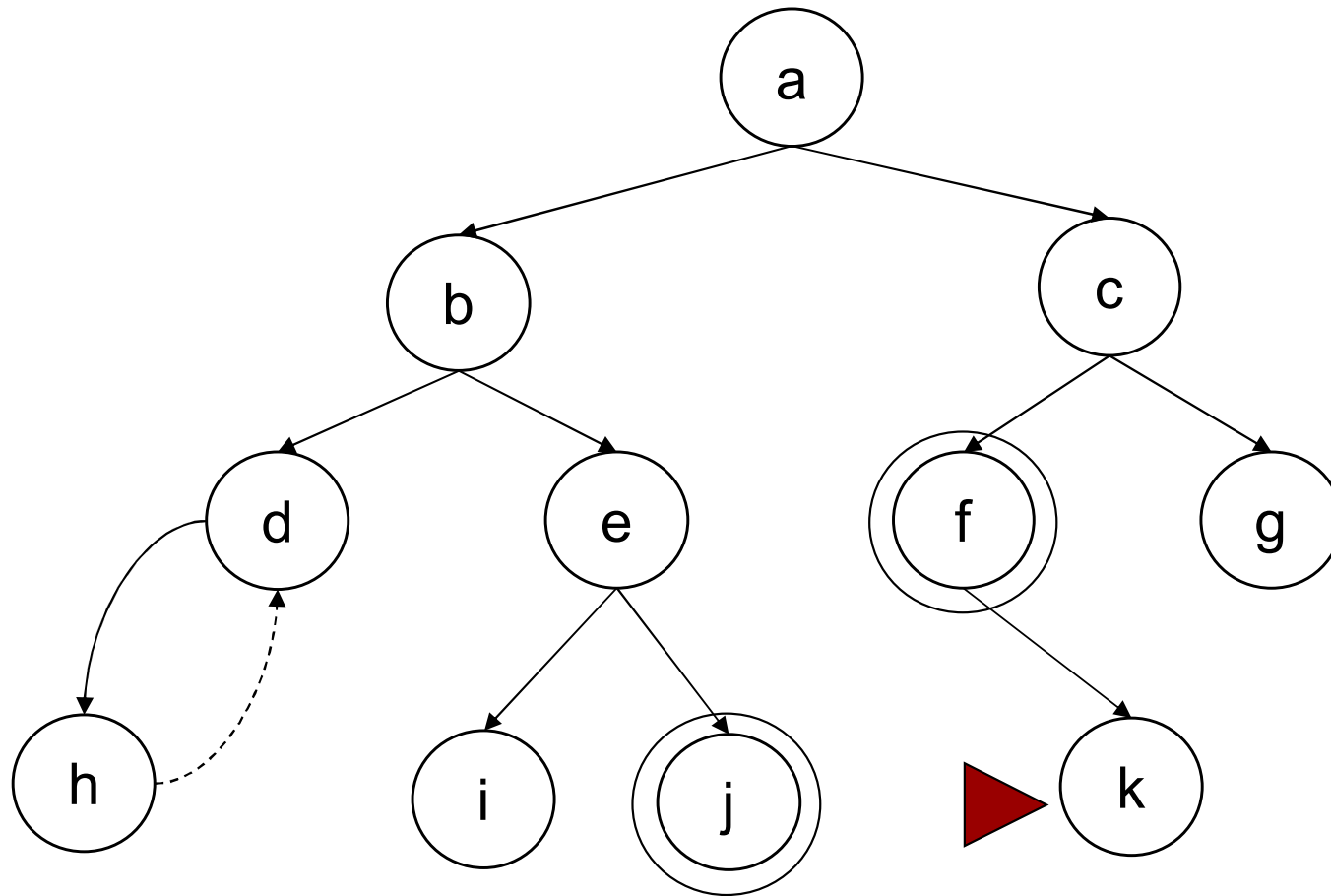
Inserir no final, remover da frente: i, j, k

Busca em Largura



Inserir no final, remover da frente: j, k

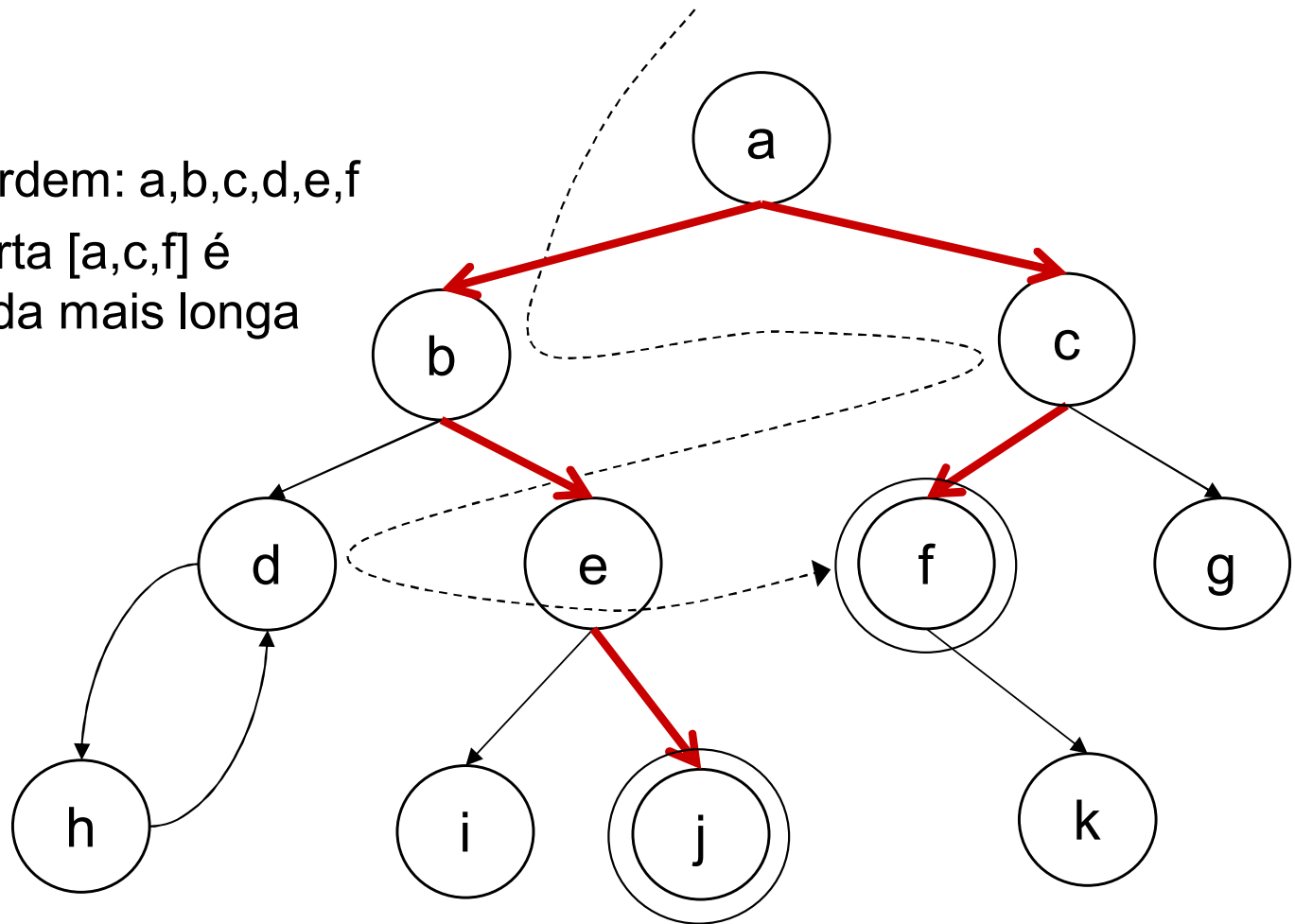
Busca em Largura



Inserir no final, remover da frente: k

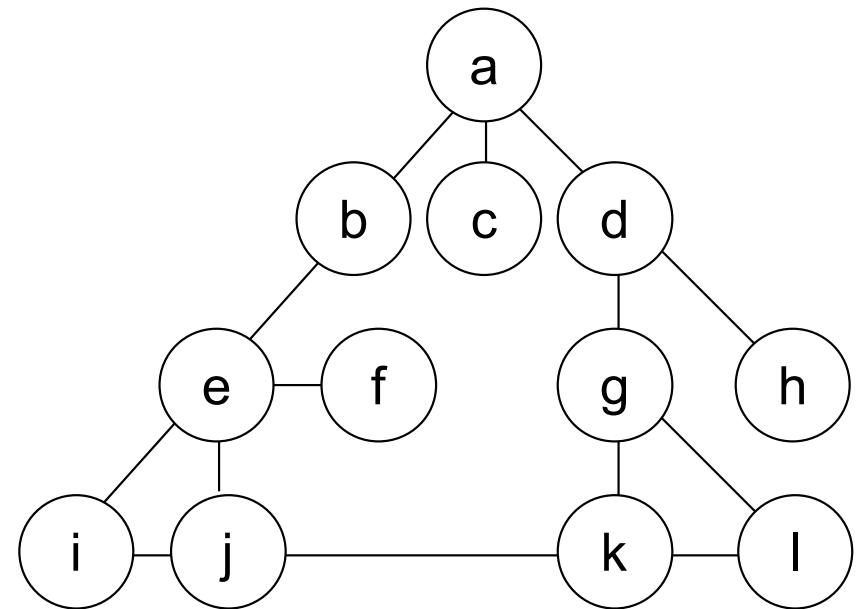
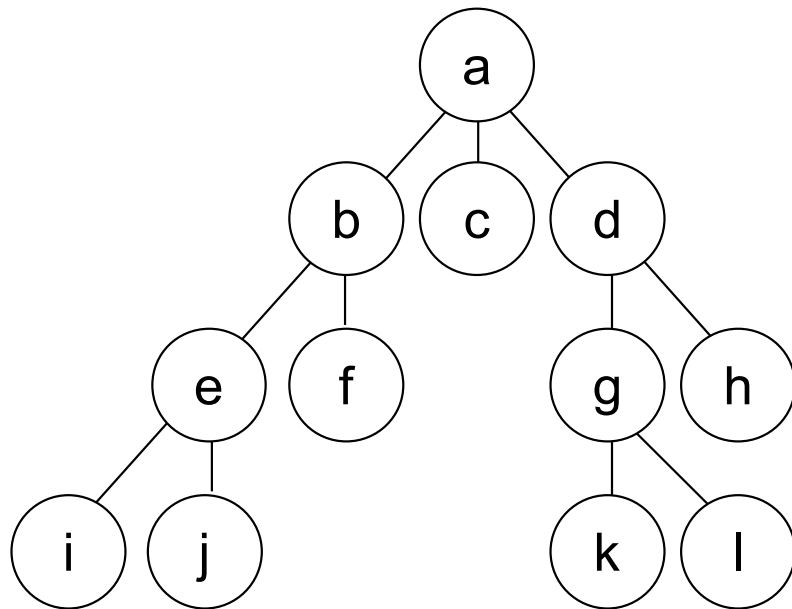
Busca em Largura

- ❑ Estado inicial: a
- ❑ Estados finais: j,f
- ❑ Nós visitados na ordem: a,b,c,d,e,f
- ❑ A solução mais curta [a,c,f] é encontrada antes da mais longa [a,b,e,j]



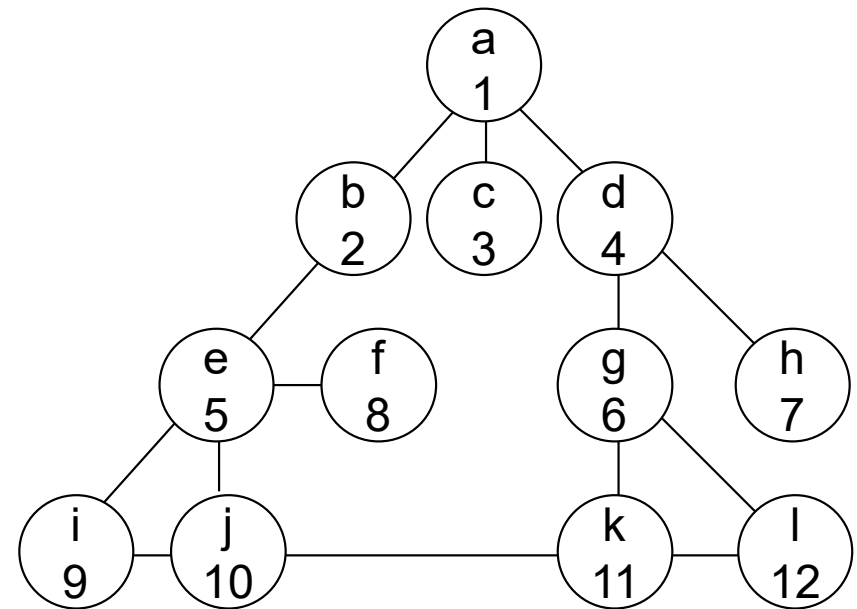
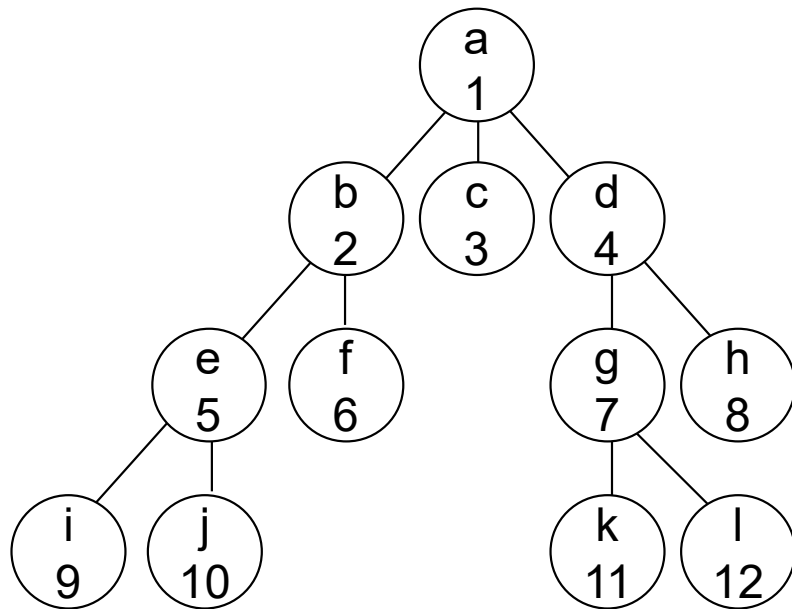
Exercício: Indique a ordem na qual os nós são visitados na busca em largura

Assuma que os sucessores de um nó são definidos em ordem alfabética

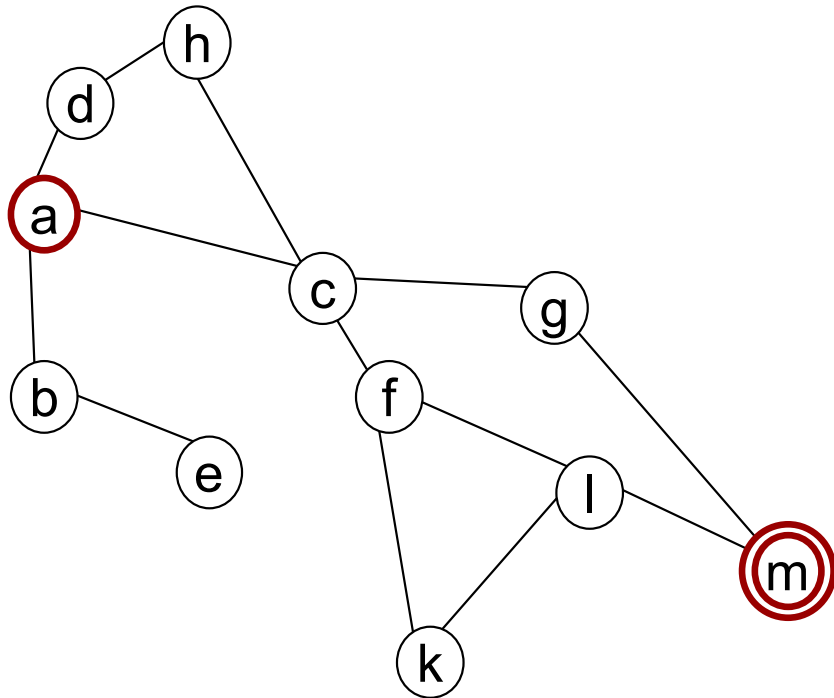


Solução

Assuma que os sucessores de um nó são definidos em ordem alfabética



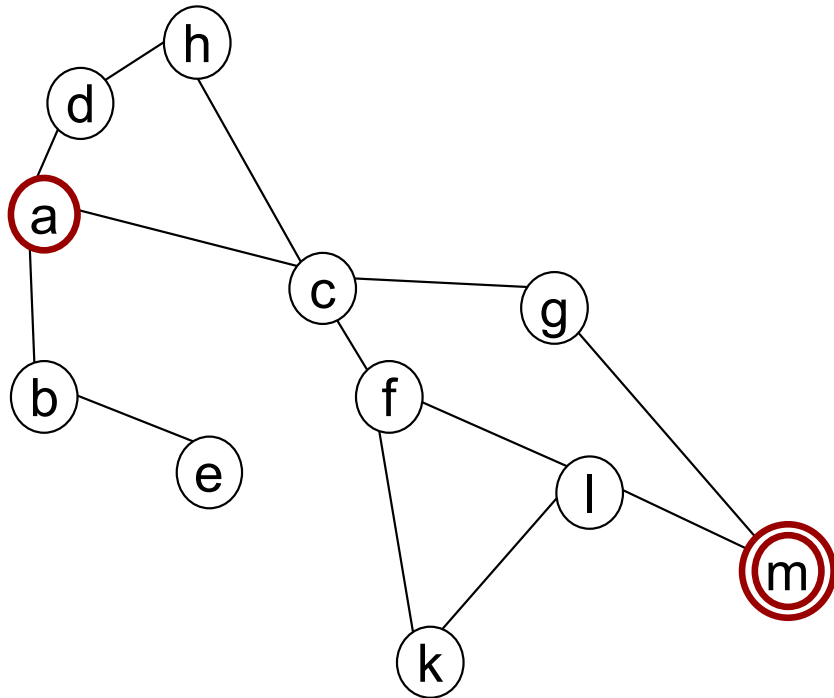
Exercício



Assuma que os sucessores de um nó são definidos em ordem alfabética

- ❑ Mostre a árvore de busca definida pelo algoritmo de busca em largura, partindo de **a** e chegando até **m**
- ❑ Mostre também os caminhos encontrados na ordem em que são encontrados pela busca em largura

Solução



Caminhos encontrados:

[a,c,g,m]

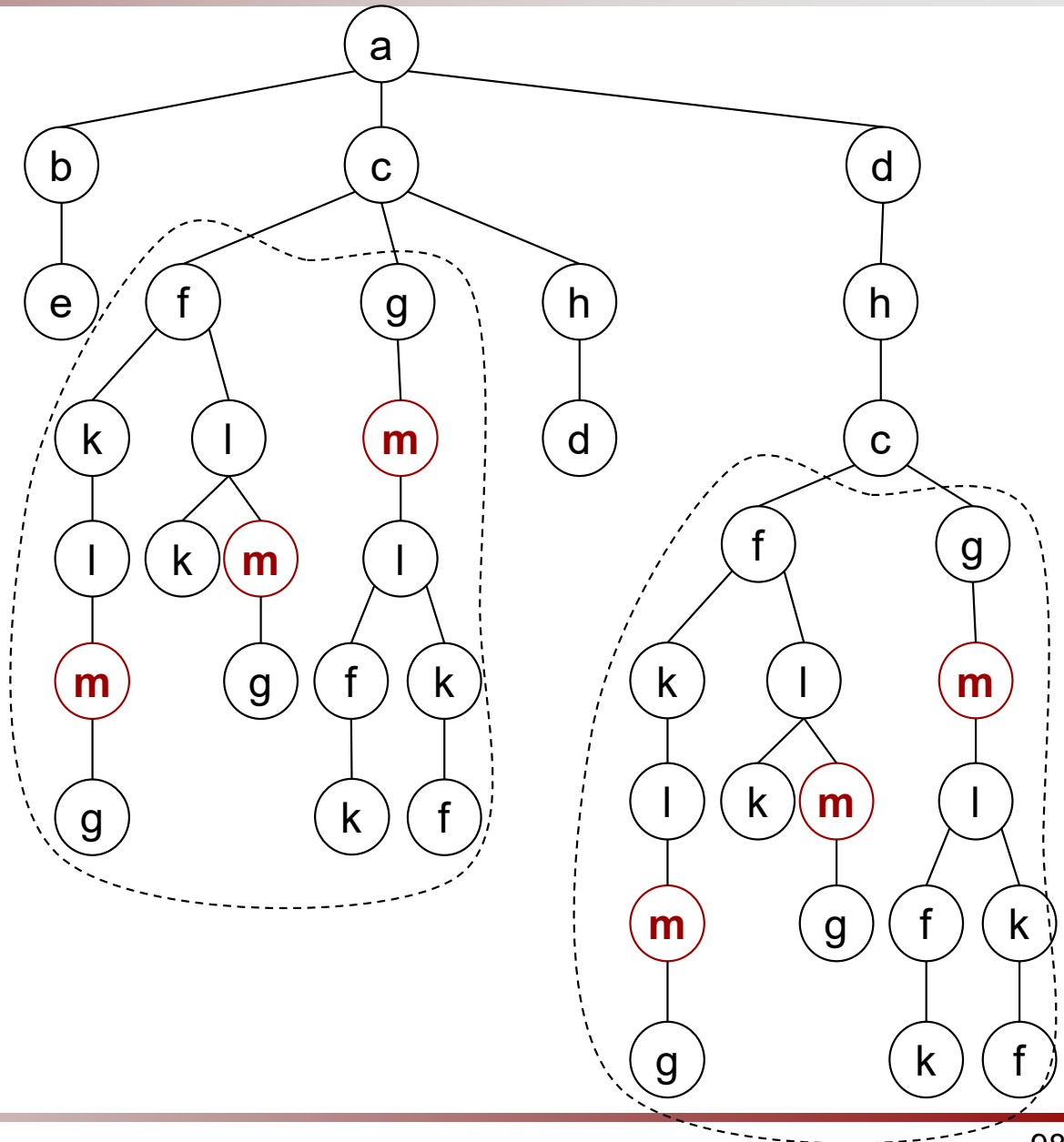
[a,c,f,l,m]

[a,c,f,k,l,m]

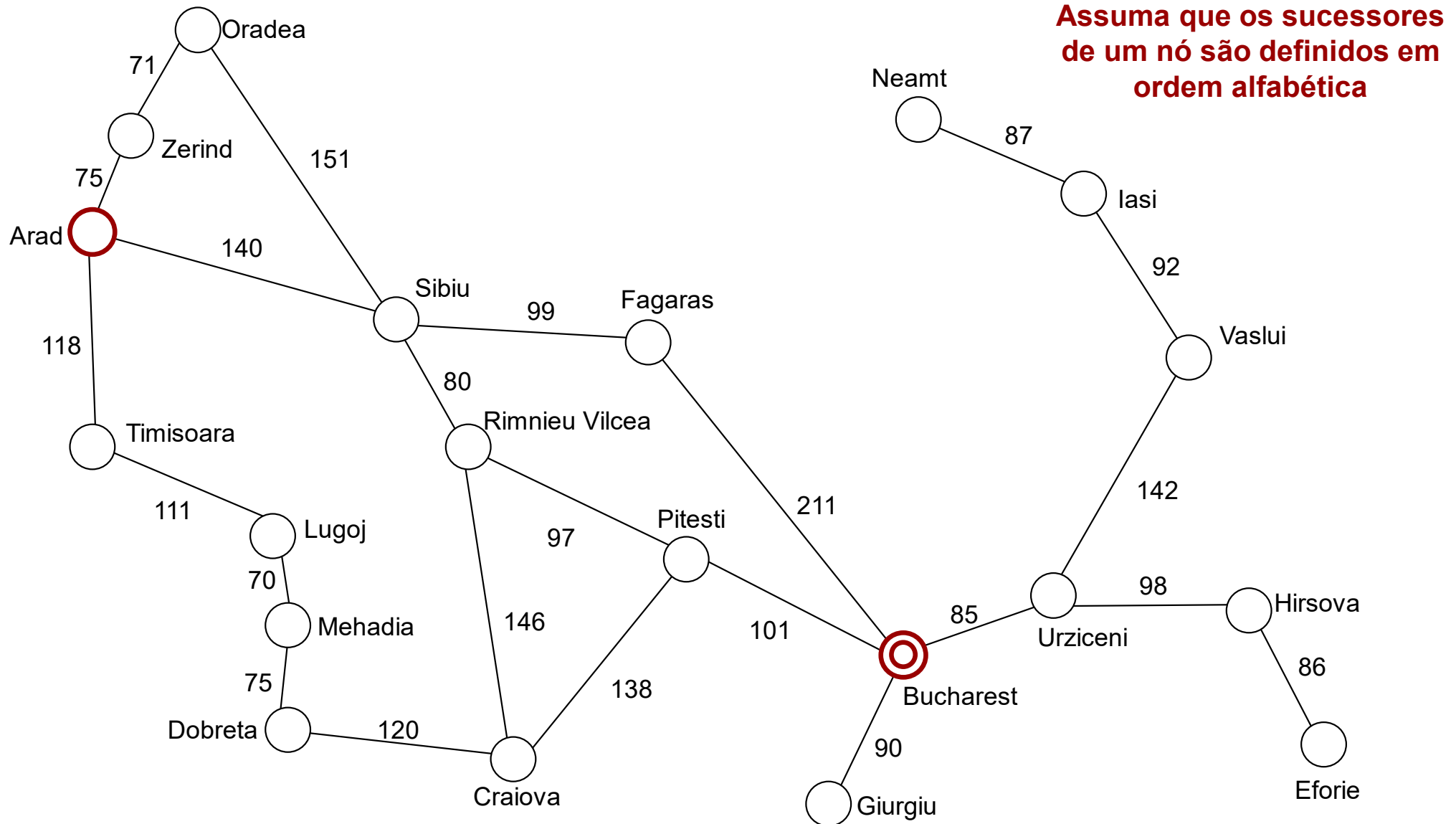
[a,d,h,c,g,m]

[a,d,h,c,f,l,m]

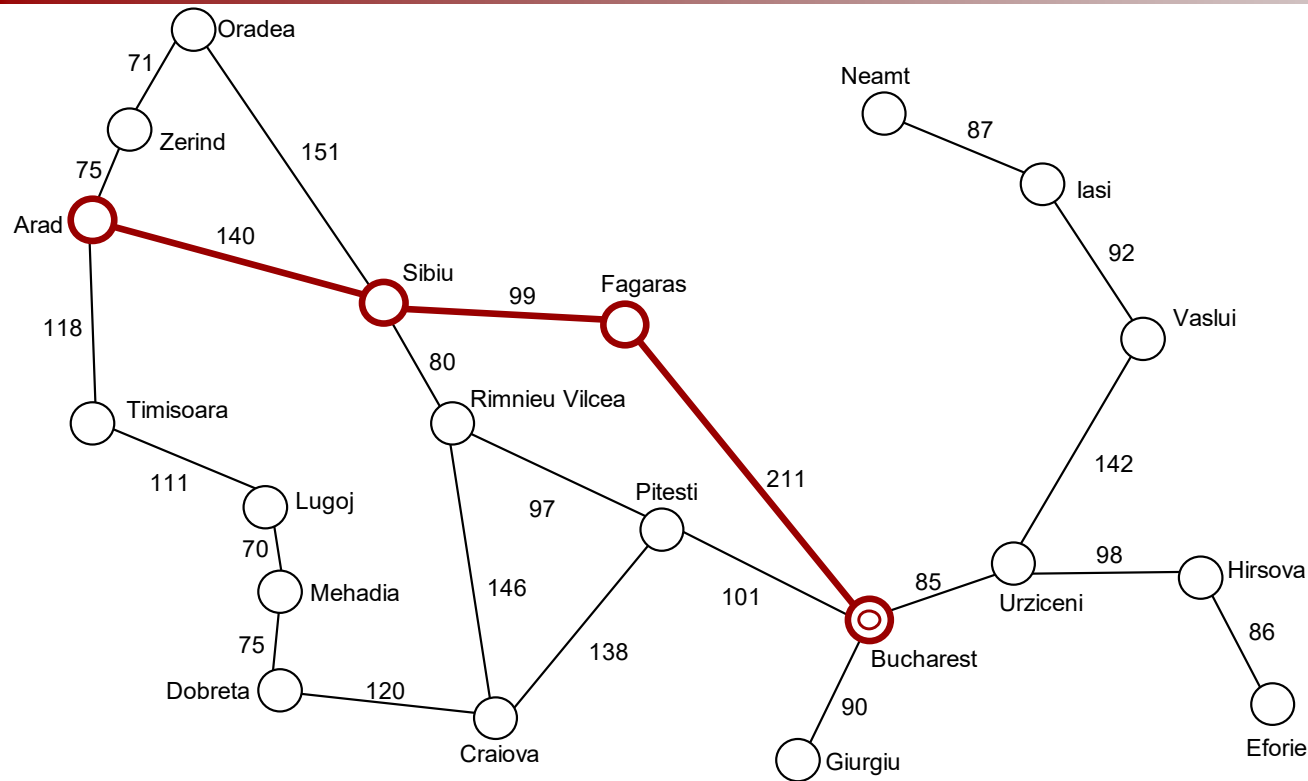
[a,d,h,c,f,k,l,m]



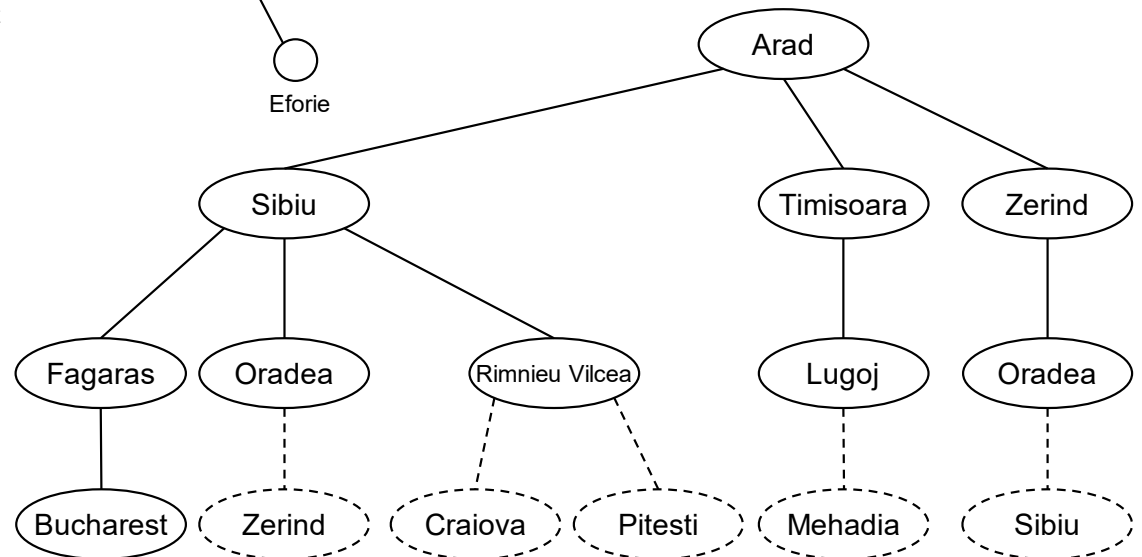
Encontre o Caminho e Árvore de Busca de Arad até Bucharest usando Busca em Largura



Solução



Árvore de Busca

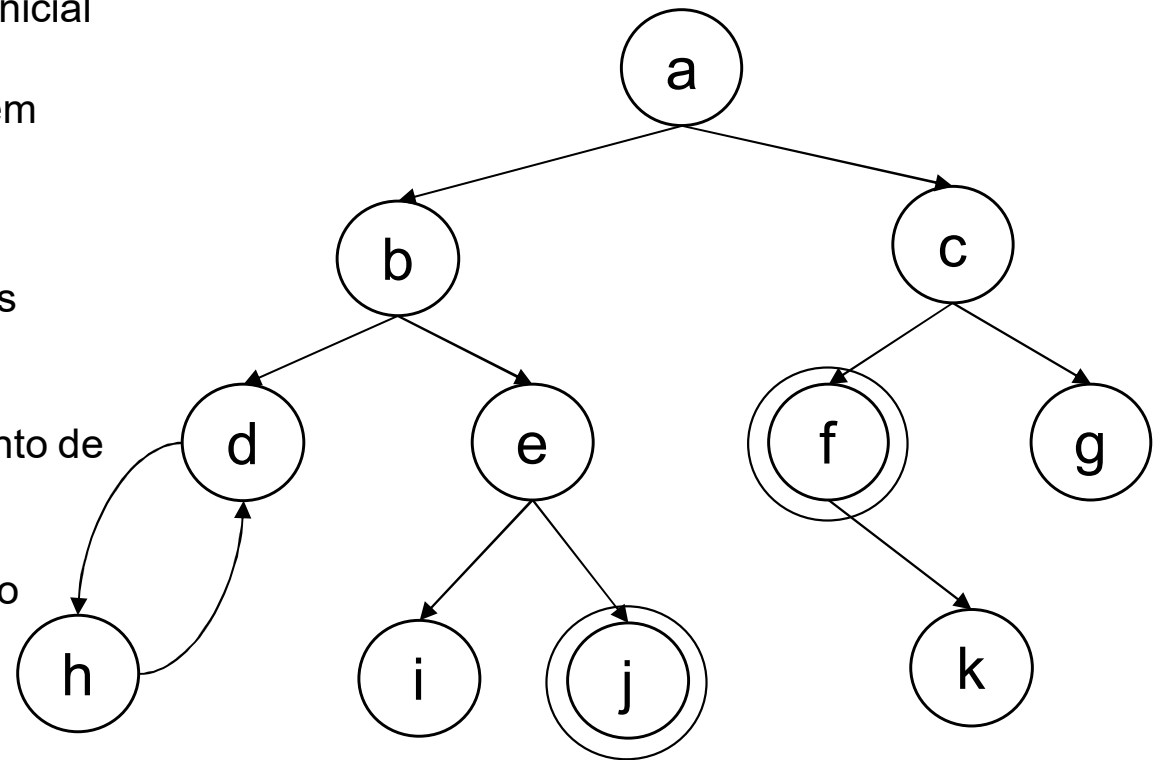


Busca em Largura

- ❑ **breadthFirst(Caminhos,Solução)** é verdadeiro se algum caminho a partir do conjunto de candidatos **Caminhos** pode ser estendido para um nó final; **Solução** é o caminho estendido
- ❑ O conjunto de caminhos candidatos será representado como uma lista de caminhos e cada caminho será uma lista de nós na ordem reversa
- ❑ A busca inicia com um conjunto de um único candidato:
 - [[Início]]
- ❑ O algoritmo é o seguinte:
 - Se a cabeça do primeiro caminho é um nó final então este caminho é uma solução; caso contrário
 - Remova o primeiro caminho do conjunto de candidatos e gere o conjunto de todas as extensões em um passo a partir deste caminho; adicione este conjunto de extensões ao final do conjunto de candidatos e execute busca em largura para atualizar este conjunto

Busca em Largura

- ❑ Comece com o conjunto de candidatos inicial
 - [[a]]
- ❑ Gere extensões de [a] (note que estão em ordem reversa):
 - [[b,a], [c,a]]
- ❑ Remova o primeiro candidato [b,a] do conjunto de candidatos e gere extensões deste caminho
 - [[d,b,a], [e,b,a]]
- ❑ Adicione as extensões ao final do conjunto de candidatos
 - [[c,a], [d,b,a], [e,b,a]]
- ❑ Remova [c,a] e adicione as extensões ao final
 - [[d,b,a], [e,b,a], [f,c,a], [g,c,a]]
- ❑ Estendendo [d,b,a]
 - [[e,b,a], [f,c,a], [g,c,a], [h,d,b,a]]
- ❑ Estendendo [e,b,a]
 - [[f,c,a], [g,c,a], [h,d,b,a], [i,e,b,a], [j,e,b,a]]
- ❑ A busca encontra [f,c,a] que contém um nó final, portanto o caminho é retornado como uma solução



Busca em Largura

```
% resolva5(No,Solucao) Solucao é um caminho acíclico
% (na ordem reversa) entre nó inicial No uma solução
resolva5(No,Solucao) :-
    breadthFirst([[No]],Solucao).

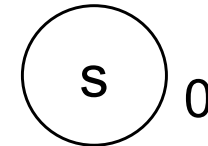
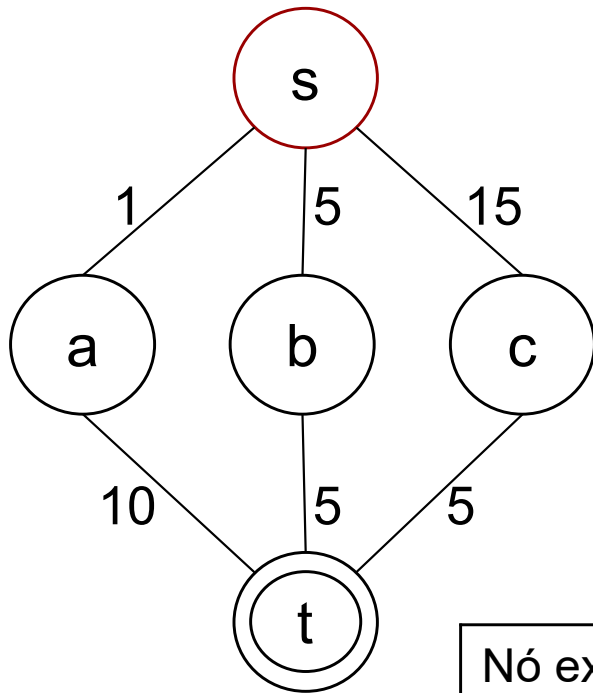
% breadthFirst([Caminho1,Caminhos2,...],Solucao) Solucao é uma
% extensão para um nó final de um dos caminhos
breadthFirst([[No|Caminho]|_],[No|Caminho]) :-
    final(No).
breadthFirst([Caminho|Caminhos],Solucao) :-
    estender(Caminho,NovosCaminhos),
    concatenar(Caminhos,NovosCaminhos,Caminhos1),
    breadthFirst(Caminhos1,Solucao).

estender([No|Caminho],NovosCaminhos) :-
    findall([NovoNo,No|Caminho],
        (s(No,NovoNo), \+ pertence(NovoNo,[No|Caminho])) ,
        NovosCaminhos).
```

Busca de Custo Uniforme

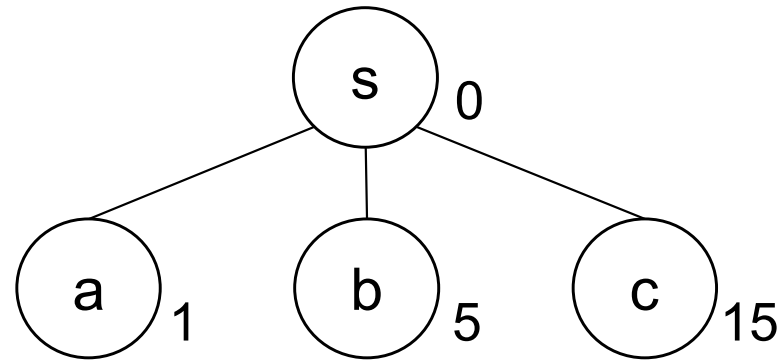
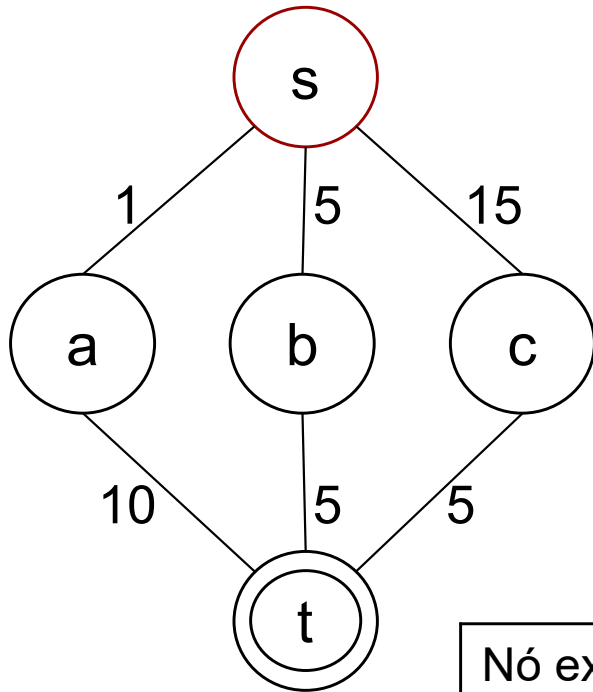
- ❑ A busca de custo uniforme (*uniform-cost search*) é similar à busca em largura, exceto pelo fato que os caminhos são colocados em uma **fila de prioridades**
 - A fila de prioridade se comporta como uma fila, exceto pelo fato que os elementos que a compõem são sempre ordenados em relação a algum valor, normalmente o custo associado
- ❑ O custo da raiz da busca até o nó atual ou o número de nós percorridos são exemplos de valores que são tipicamente colocados na fila de prioridades da busca de custo uniforme
 - **$g(n)$** = custo da raiz da busca até o nó **n**
- ❑ Assim, ao invés de expandir o nó mais próximo à raiz da busca, a busca de custo uniforme expande o nó **n** com o caminho de custo mais baixo **$g(n)$** , ou seja, partir do nó inicial, a busca continua para o próximo nó de menor custo desde a raiz
- ❑ Uma forma de busca do primeiro melhor (*best-first search*)
 - Utilizando $f(n) = g(n)$
- ❑ Se os custos forem iguais, a busca de custo uniforme torna-se idêntica à busca em largura
- ❑ O algoritmo é ótimo e completo se todos os custos forem maiores ou iguais a alguma constante positiva pequena ϵ

Busca de Custo Uniforme



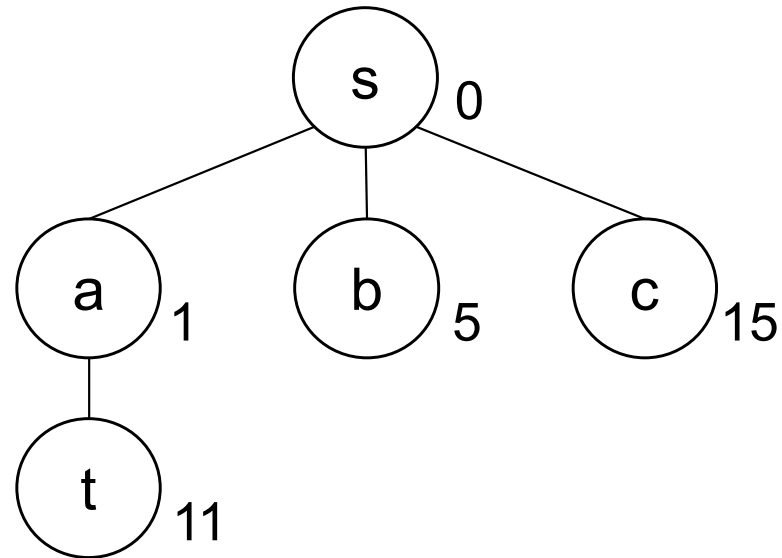
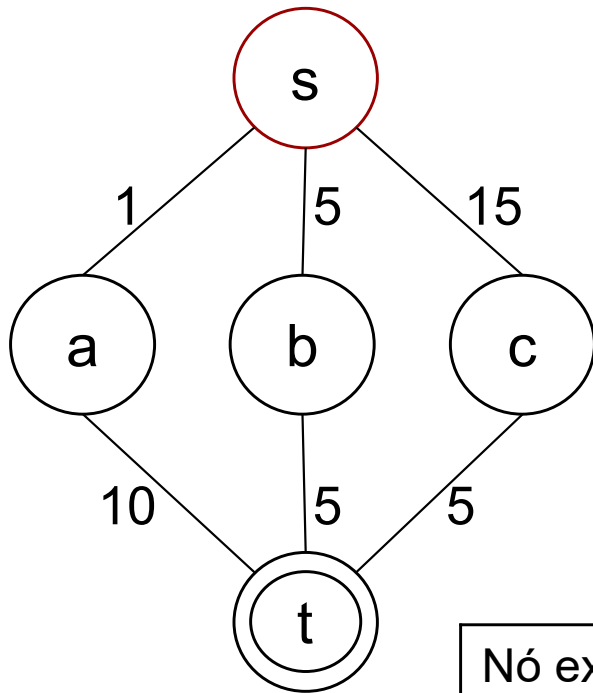
Nó expandido	Fila	Fila Ordenada pelo valor g
	[s]:0	[s]:0

Busca de Custo Uniforme



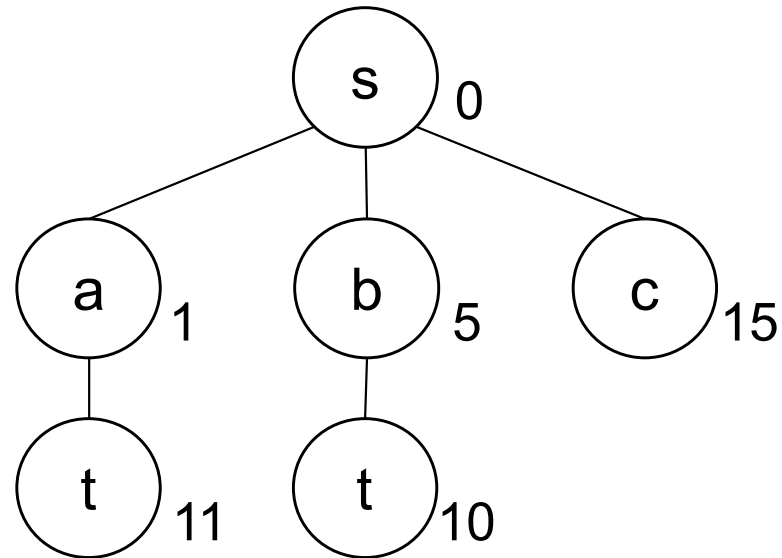
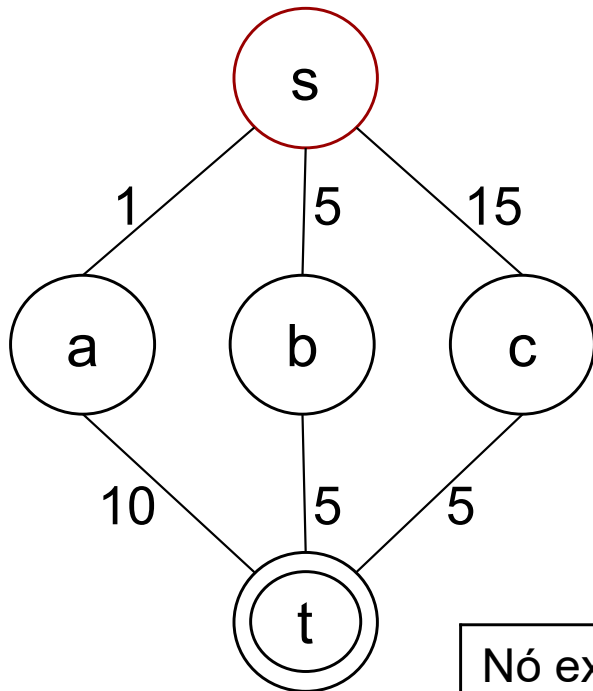
Nó expandido	Fila	Fila Ordenada pelo valor g
	[s]:0	[s]:0
s	[a,s]:1, [b,s]:5, [c,s]:15	[a,s]:1, [b,s]:5, [c,s]:15

Busca de Custo Uniforme



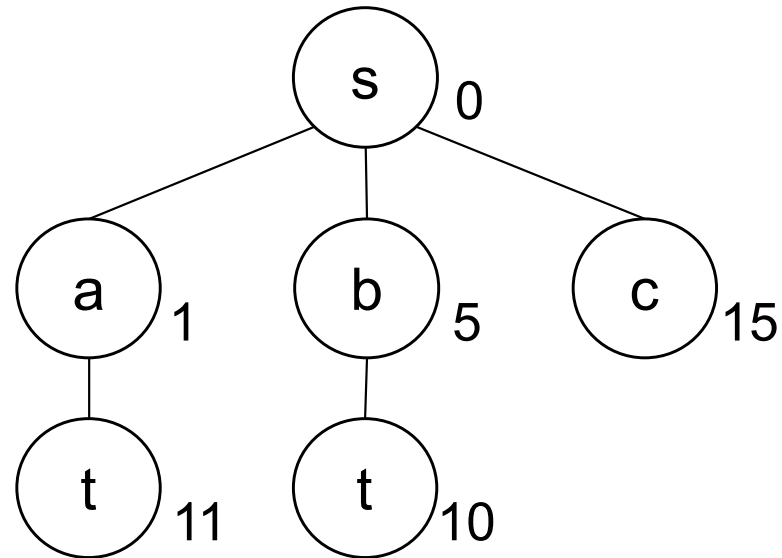
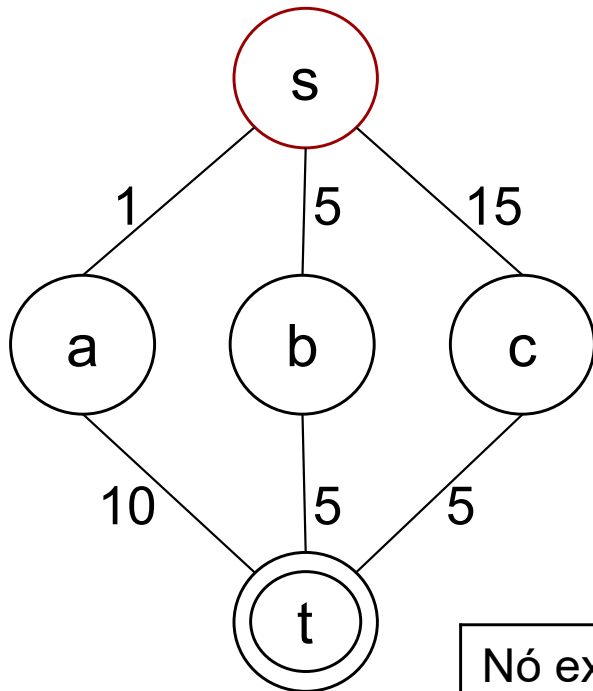
Nó expandido	Fila	Fila Ordenada pelo valor g
	[s]:0	[s]:0
s	[a,s]:1, [b,s]:5, [c,s]:15	[a,s]:1, [b,s]:5, [c,s]:15
a	[b,s]:5, [c,s]:15, [t,a,s]:11	[b,s]:5, [t,a,s]:11, [c,s]:15

Busca de Custo Uniforme



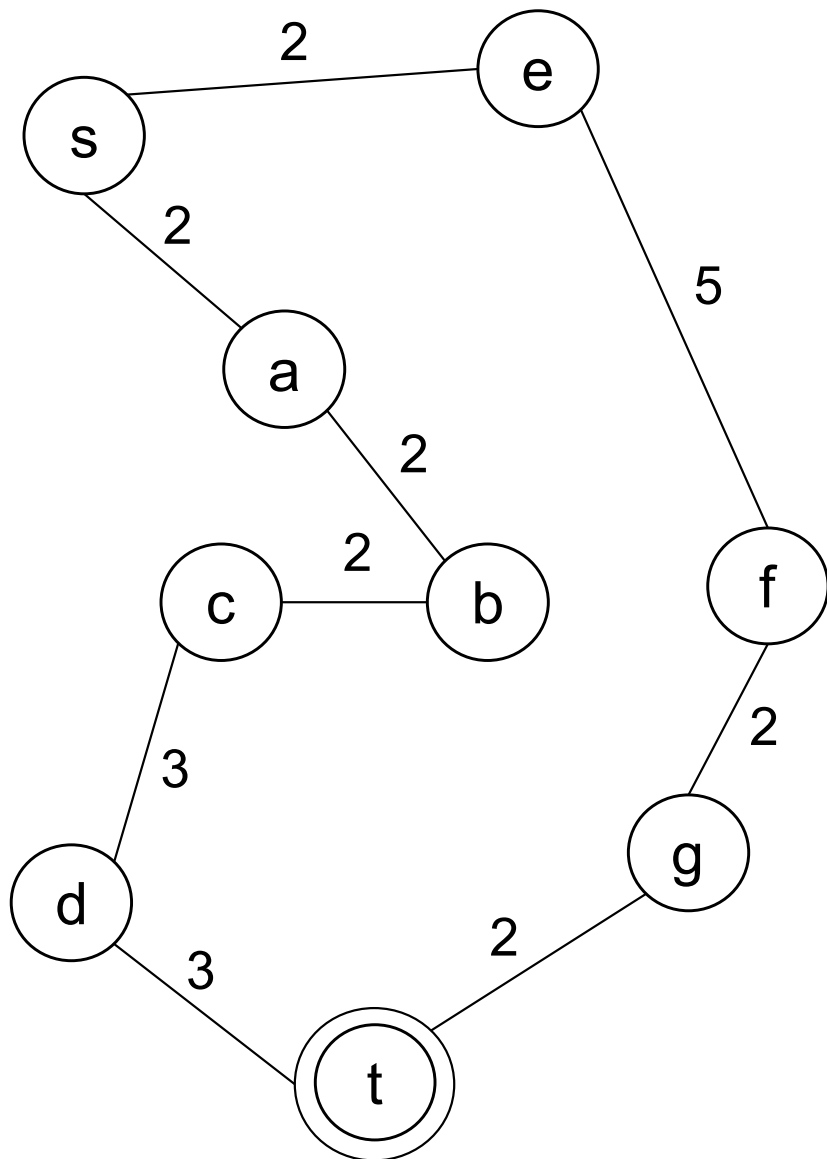
Nó expandido	Fila	Fila Ordenada pelo valor g
	[s]:0	[s]:0
s	[a,s]:1, [b,s]:5, [c,s]:15	[a,s]:1, [b,s]:5, [c,s]:15
a	[b,s]:5, [c,s]:15, [t,a,s]:11	[b,s]:5, [t,a,s]:11, [c,s]:15
b	[t,a,s]:11, [c,s]:15, [t,b,s]:10	[t,b,s]:10, [t,a,s]:11, [c,s]:15

Busca de Custo Uniforme



Nó expandido	Fila	Fila Ordenada pelo valor g
	[s]:0	[s]:0
s	[a,s]:1, [b,s]:5, [c,s]:15	[a,s]:1, [b,s]:5, [c,s]:15
a	[b,s]:5, [c,s]:15, [t,a,s]:11	[b,s]:5, [t,a,s]:11, [c,s]:15
b	[t,a,s]:11, [c,s]:15, [t,b,s]:10	[t,b,s]:10, [t,a,s]:11, [c,s]:15
t	Solução(início da fila): [t,b,s]	

Busca de Custo Uniforme: Exercício



Nó	Fila	Fila Ordenada pelo valor g
	[s]:0	[s]:0
s	[a,s]:2, [e,s]:2	[a,s]:2, [e,s]:2
a	[e,s]:2, [b,a,s]:4	[e,s]:2, [b,a,s]:4
e	[b,a,s]:4, [f,e,s]:7	[b,a,s]:4, [f,e,s]:7
b	[f,e,s]:7, [c,b,a,s]:6	[c,b,a,s]:6, [f,e,s]:7
c	[f,e,s]:7, [d,c,b,a,s]:9	[f,e,s]:7, [d,c,b,a,s]:9
f	[d,c,b,a,s]:9, [g,f,e,s]:9	[d,c,b,a,s]:9, [g,f,e,s]:9
d	[g,f,e,s]:9, [t,d,c,b,a,s]:12	[g,f,e,s]:9, [t,d,c,b,a,s]:12
g	[t,d,c,b,a,s]:12, [t,g,f,e,s]:11	[t,g,f,e,s]:11, [t,d,c,b,a,s]:12
t	Solução: [t,g,f,e,s]	

Busca de Custo Uniforme

```
% resolve6(No,Solucao) Solucao é um caminho acíclico (na ordem reversa) entre nó inicial No até uma solução
resolve6(No,Solucao) :-
    uniformCost([[No]:0],Solucao).

uniformCost([[No|Caminho]:Custo|_],[No|Caminho]:Custo) :-
    final(No).
uniformCost([Caminho:Custo|Caminhos],Solucao) :-
    estender2(Caminho:Custo,NovosCaminhos),
    fila_prioridade(NovosCaminhos,Caminhos,Caminhos1),
    uniformCost(Caminhos1,Solucao).

estender2([No|Caminho]:CustoAcumulado,NovosCaminhos) :-
    findall([NovoNo,No|Caminho]:NovoCusto,
            (s(No,NovoNo,Custo), \+ pertence(NovoNo,[No|Caminho]), NovoCusto is CustoAcumulado + Custo),
            NovosCaminhos).

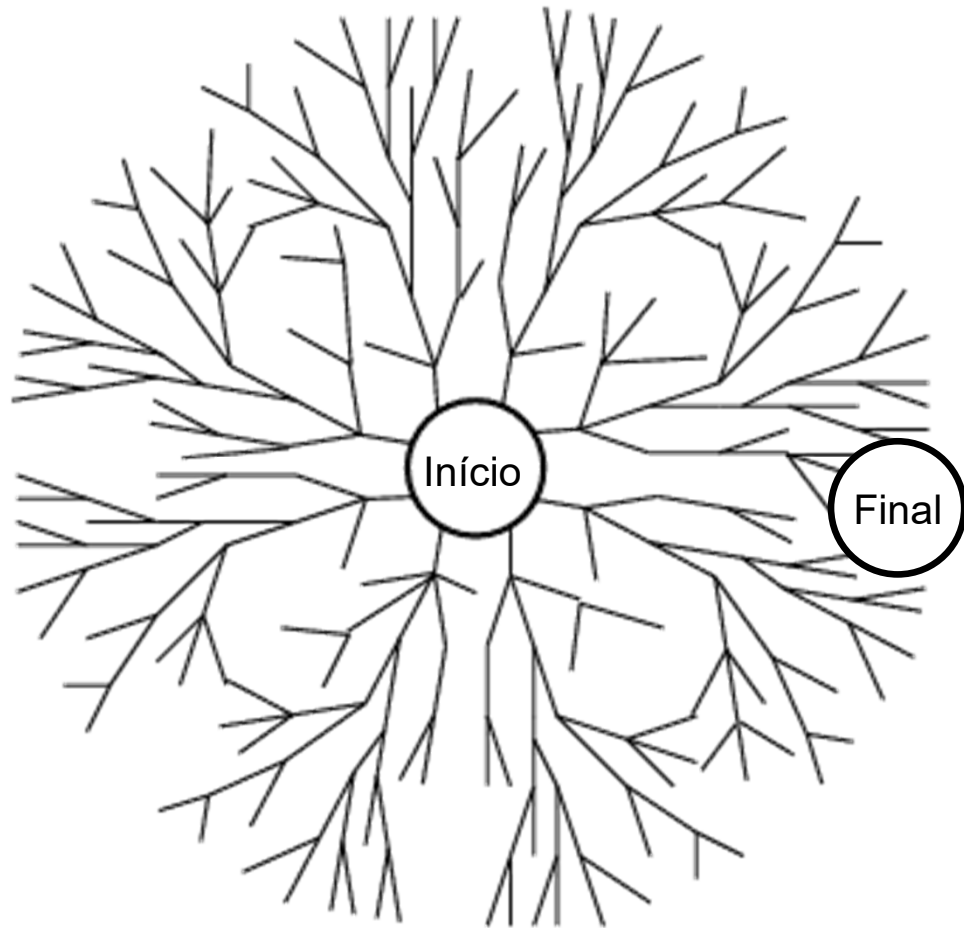
% fila_prioridade(NovosCaminhos,CaminhosExistentes,CaminhosOrdenados)
% concatena (com prioridade) os NovosCaminhos em CaminhosExistentes formando CaminhosOrdenados
% assumindo que CaminhosExistentes já está ordenado por prioridade (ou seja, por custo)
fila_prioridade([],L,L).
fila_prioridade([Caminho:Custo|Caminhos],Existente,Final) :-
    insert(Caminho:Custo,Existente,ExistenteAumentado),
    fila_prioridade(Caminhos,ExistenteAumentado,Final).

insert(Caminho:Custo,[Path:Cost|Paths],[Path:Cost|NewPaths]) :-
    Custo > Cost, !,
    insert(Caminho:Custo,Paths,NewPaths).
insert(C,Paths,[C|Paths]).
```

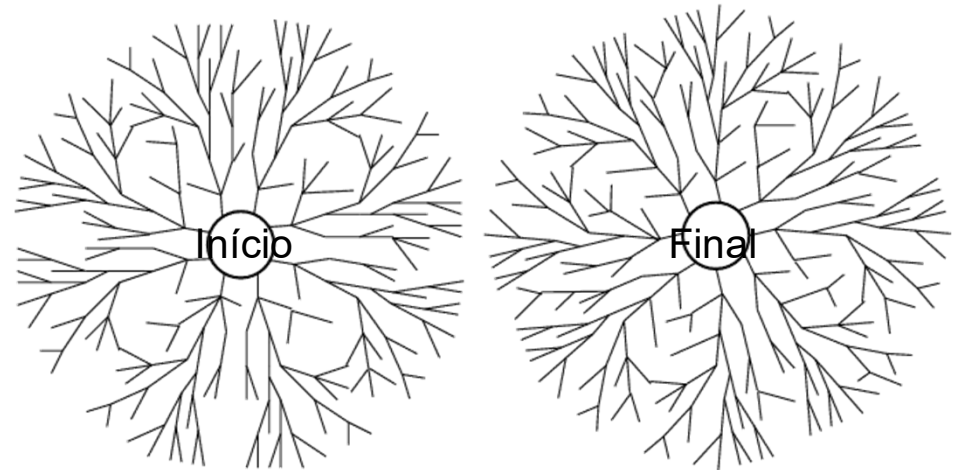
Busca Bidirecional

- ❑ Consiste em executar duas buscas simultâneas
 - Uma partindo do estado inicial em direção ao final
 - Outra partindo do estado final em direção ao inicial
- ❑ A motivação é que $O(b^{d/2}) + O(b^{d/2})$ é melhor do que $O(b^d)$, ou seja, a “área” de duas buscas é menor do que a “área” de uma única busca
 - d = profundidade da solução mais curta
 - b = fator de ramificação

Busca Bidirecional



Busca Unidirecional



Busca Bidirecional

Busca em Árvore, Todas as Soluções

```
Algoritmo: buscaEmArvore(Grafo, S) // S é nó inicial no Grafo
    Fronteira = { }                // Frontier ou Open List
    [G] Fechados = { }             // Explored Set ou Closed List
    Expansoes = 0                  // Node Expansions
    Fronteira.inserir( [S] )       // Caminho inicial contendo apenas S
    Solucao = { }
    [F] while Fronteira ≠ { } and Solucao = { } do
        [U | C] = Fronteira.remover() // U é o nó atual; C é a cauda (ancestrais)
        [G] if U ∉ Fechados then // nó não foi expandido anteriormente
            [G] Fechados = Fechados ∪ {U}
            if final(U) then // u é um nó final?
                Solucao = Solucao ∪ reverso( [U | C] )
            else
                Expansoes = Expansoes + 1
                for each V ∈ Grafo.sucessores(U) do
                    [G] if V ∉ [U | C] and V ∉ Fechados then Fronteira.inserir([V, U | C]) endif
                endfor
            endif
        [G] endif
    endwhile
    return (Solucao, Expansoes)
```

Busca em Árvore, Somente a Primeira Solução

```
Algoritmo: buscaEmArvore(Grafo, S) // S é nó inicial no Grafo
    Fronteira = { }                // Frontier ou Open List
    [G] Fechados = { }             // Explored Set ou Closed List
    Expansoes = 0                  // Node Expansions
    Fronteira.inserir( [S] )       // Caminho inicial contendo apenas S
    Solucao = { }
    [F] while Fronteira ≠ { } and Solucao = { } do
        [U | C] = Fronteira.remover() // U é o nó atual; C é a cauda (ancestrais)
        [G] if U ∉ Fechados then // nó não foi expandido anteriormente
            [G] Fechados = Fechados ∪ {U}
            if final(U) then // u é um nó final?
                Solucao = Solucao ∪ reverso( [U | C] )
            else
                Expansoes = Expansoes + 1
                for each V ∈ Grafo.sucessores(U) do
                    [G] if V ∉ [U | C] and V ∉ Fechados then Fronteira.inserir([V, U | C]) endif
                endfor
            endif
        [G] endif
    endwhile
    return (Solucao, Expansoes)
```

Complexidade dos Algoritmos de Busca

Algoritmo	Tempo	Espaço	Ótima? (solução mais curta garantida)	Completa? (encontra uma solução quando ela existe)
Profundidade	$O(b^m)$	$O(bm)$	Não	Sim, em espaços finitos
Profundidade limitada	$O(b^l)$	$O(bl)$	Não	Sim, se $l \geq d$
Profundidade iterativa	$O(b^d)$	$O(bd)$	Sim	Sim
Largura	$O(b^d)$	$O(b^d)$	Sim	Sim
Custo Uniforme	$O(b^{\lceil C^*/\epsilon \rceil})$	$O(b^{\lceil C^*/\epsilon \rceil})$	Sim	Sim, se $\epsilon > 0$
Bidirecional	$O(b^{d/2})$	$O(b^{d/2})$	Sim	Sim
Busca Gulosa (Greedy)	$O(b^m)$	$O(b^m)$	Não	Sim, em espaços finitos
A*	$O(b^{\lceil C^*/\epsilon \rceil})$	$O(b^{\lceil C^*/\epsilon \rceil})$	Sim (se heurística admissível)	Sim
IDA*	$O(b^{\lceil C^*/\epsilon \rceil})$	$O(bd)$	Sim (se heurística admissível)	Sim
RBFS	$O(b^{\lceil C^*/\epsilon \rceil})$	$O(bd)$	Sim (se heurística admissível)	Sim
Beam Search (com caminho)	$O(kbm)$	$O(km)$	Não	Não
Local Beam Search	$O(kbm)$	$O(k)$	Não	Não
Subida de Encosta (Hill-Climbing)	∞ (Pode ficar preso em platôs)	$O(1)$	Não	Não
Têmpera Simulada	Depende da taxa de resfriamento (pode tender a ∞)	$O(1)$	Não (mas chega muito perto se esfriar bem devagar)	Não

Legenda;

- b = número de caminhos alternativos/fator de bifurcação/ramificação (branching factor)
- d = profundidade da solução (profundidade do nó final mais raso)
- m = profundidade máxima da árvore de busca (pode ser infinita)
- l = limite de profundidade configurado manualmente pelo usuário
- k = tamanho do feixe
- C^* = custo total do caminho da solução ótima
- $\epsilon > 0$ é uma constante, assumindo que todos os custos no espaço de estados são $\geq \epsilon$, ou seja, representa o menor custo de transição (aresta) possível em todo o espaço de estados, por exemplo, se você está em um tabuleiro onde cada movimento tem custo 1, então $\epsilon = 1$

Entendendo a Complexidade

$$O(b^{\lceil C^*/\varepsilon \rceil})$$

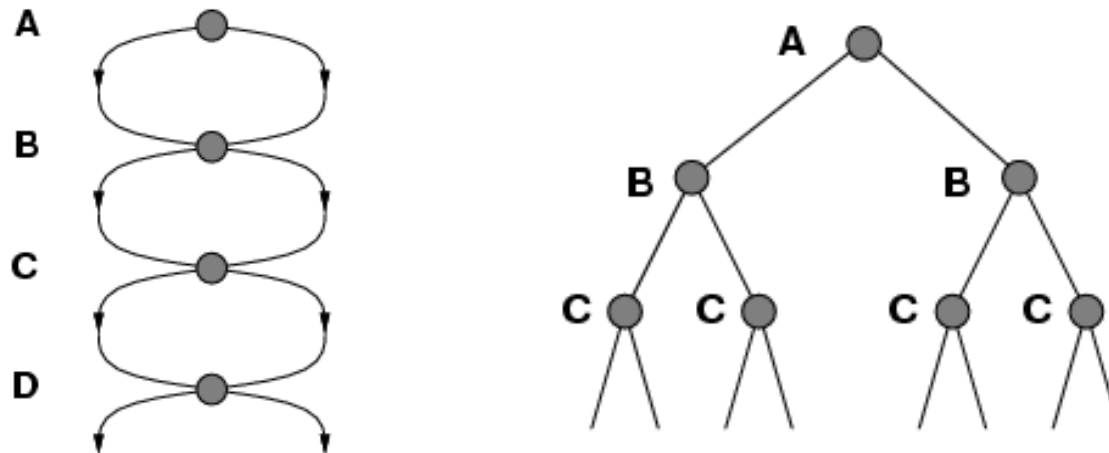
- ❑ O limite teórico $O(b^{\lceil C^*/\varepsilon \rceil})$ descreve o pior cenário absoluto do espaço de estados, onde
 - C^* : é o custo total da solução ótima encontrada
 - ε : é o menor custo possível no espaço de estados ($\varepsilon > 0$)
 - A divisão $\lceil C^*/\varepsilon \rceil$ calcula a *profundidade máxima teórica*, ou seja, no pior caso, quantos passos (transições) mínimos de tamanho ε o algoritmo seria forçado a dar para alcançar o custo C^*
- ❑ Exemplo de busca de um caminho de custo $C^* = 100$
 - Caso Péssimo ($\varepsilon = 1$): se todas as arestas custarem apenas 1, o algoritmo terá que dar 100 passos e a complexidade explode para a profundidade 100, gerando uma árvore enorme de $O(b^{100})$
 - Caso Médio ($\varepsilon = 50$): se existirem caminhos expressos onde cada passo custa 50, a solução será encontrada em apenas 2 passos e a árvore gerada cai drasticamente para $O(b^2)$

O Fator de Ramificação Efetivo (b^*)

- ❑ A fórmula $O(b^{\lceil C^*/\varepsilon \rceil})$ assume que estamos fazendo uma Busca Cega (ou seja, usando uma heurística inútil onde $h(n) = 0$)
- ❑ Na prática, o A^* usa uma heurística *inteligente* que guia a busca e poda os caminhos ruins
 - Essa *inteligência* reduz o fator de ramificação base (b) para o que chamamos de Fator de Ramificação Efetivo (b^*)
 - Com uma boa heurística, o algoritmo avalia pouquíssimos vizinhos por nível
 - Assim, a complexidade se transforma em $O((b^*)^d)$, onde o $b^* \approx 1$ e d é a profundidade real do caminho percorrido
 - Quando usamos o Fator de Ramificação Efetivo ($b^* \approx 1$) para medir o quão boa é a heurística, a literatura abandona o pior caso $\lceil C^*/\varepsilon \rceil$
 - ❖ Passamos a medir o desempenho baseados em quantos passos reais o algoritmo deu (d)
 - ❖ Como no mundo real os custos variam e costumam ser maiores que ε , na imensa maioria das vezes $d \ll \lceil C^*/\varepsilon \rceil$ (a profundidade real é muito menor que o teto teórico)
 - ❖ Assim, a avaliação de desempenho sai do pior caso $O(b^{\lceil C^*/\varepsilon \rceil})$ para o caso médio $O((b^*)^d)$

O Problema de Caminhos Redundantes

- ❑ Em diversos problemas de busca, é comum que o espaço de estados possua múltiplos caminhos diferentes que convergem para o mesmo estado
- ❑ Quando isso ocorre, algoritmos baseados puramente em árvores correm o risco de se tornarem infinitos; no entanto, ao detectarmos e podarmos estados repetidos, garantimos que a árvore de busca se torne finita
- ❑ Mesmo em espaços finitos, evitar a reexpansão de estados repetidos transforma um tempo de execução exponencial em polinomial ou linear
- ❑ No exemplo abaixo, o espaço contém apenas $m+1$ estados, onde m é a profundidade máxima; como a árvore de busca contém cada caminho possível no espaço, ela possui 2^m ramos



Evitando Estados/Caminhos Repetidos

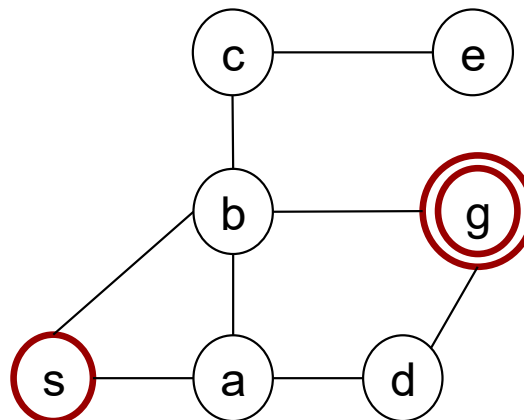
- Para evitar o desperdício computacional, a literatura clássica propõe três abordagens com diferentes custos de memória:
 1. Evitar Retorno Imediato (Poda de Pai): A função sucessor simplesmente não gera um nó que seja idêntico ao estado de onde o agente acabou de vir
 - Trata-se de um filtro estritamente local. É uma otimização de custo mínimo, com complexidade de tempo e espaço $O(1)$
 - É eficiente para reduzir o fator de ramificação efetivo em problemas com ações reversíveis (como movimentação em malhas)
 - Contudo, não garante a terminação do algoritmo, pois o sistema permanece suscetível a laços infinitos caso o grafo possua ciclos de comprimento igual ou superior a três nós ($A \rightarrow B \rightarrow C \rightarrow A$)
 2. Evitar Ciclos no Caminho (Busca em Árvore Pura): Nenhum estado é gerado se ele já existir como ancestral no caminho atual em exploração
 - Exige uma varredura ao longo da estrutura que armazena o caminho atual; a complexidade espacial é limitada a $O(d)$, onde d é a profundidade do nó atual, o que a torna viável para algoritmos de restrição de memória (como DLS e IDA*)
 - Previne a formação de ciclos, garantindo que o algoritmo não entre em *loops* infinitos
 - No entanto, é incapaz de detectar caminhos redundantes convergentes; ou seja, se o algoritmo alcançar o mesmo estado a partir de duas ramificações distintas, a subárvore a partir desse estado será reprocessada
 3. Evitar Caminhos Redundantes (Busca em Grafo): Nenhum estado é expandido se ele já foi visitado por qualquer outro caminho em toda a história da busca, o que exige manter um registro global em memória (Conjunto de Explorados / Lista Fechada)
 - Requer estruturas de busca otimizadas, como Tabelas Hash ou Árvores de Busca Balanceadas, para garantir acesso em tempo médio $O(1)$ ou $O(\log N)$, N = número de estados
 - Elimina por completo a reexpansão de estados redundantes, reduzindo drasticamente o tempo de execução (muitas vezes de exponencial para polinomial em problemas topológicos)
 - A grande desvantagem é a severa penalidade em complexidade de espaço, que passa a ser $O(b^d)$, exigindo o armazenamento em memória de todo o histórico de expansão

Busca em Grafo (com Lista Fechada)

```
Algoritmo: buscaEmGrafo(Grafo, S) // S é nó inicial no Grafo
    Fronteira = { }                // Frontier ou Open List
    [G] Fechados = { }             // Explored Set ou Closed List
    Expansoes = 0                  // Node Expansions
    Fronteira.inserir( [S] )       // Caminho inicial contendo apenas S
    Solucao = { }
    [F] while Fronteira ≠ { } and Solucao = { } do
        [U | C] = Fronteira.remover() // U é o nó atual; C é a cauda (ancestrais)
        [G] if U ∉ Fechados then // nó não foi expandido anteriormente
            [G] Fechados = Fechados ∪ {U}
            if final(U) then // u é um nó final?
                Solucao = Solucao ∪ reverso( [U | C] )
            else
                Expansoes = Expansoes + 1
                for each V ∈ Grafo.sucessores(U) do
                    [G] if V ∉ [U | C] and V ∉ Fechados then Fronteira.inserir([V, U | C]) endif
                endfor
            endif
        [G] endif
    endwhile
    return (Solucao, Expansoes)
```

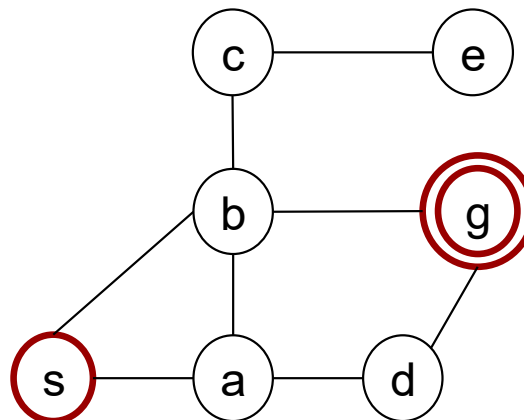
Exemplo DFS

Passo	Nó Extraído [U C]	DFS Árvore (Fronteira)	DFS Grafo (Fronteira)	DFS Grafo (Fechados)
0	(Início)	[[s]]	[[s]]	{ } (vazio)
1	[s]	[[a,s], [b,s]]	[[a,s], [b,s]]	{s}
2	[a,s]	[[b,a,s], [d,a,s], [b,s]]	[[b,a,s], [d,a,s], [b,s]]	{s, a}
3	[b,a,s]	[[c,b,a,s], [g,b,a,s], [d,a,s], [b,s]]	[[c,b,a,s], [g,b,a,s], [d,a,s], [b,s]]	{s, a, b}
4	[c,b,a,s]	[[e,c,b,a,s], [g,b,a,s], [d,a,s], [b,s]]	[[e,c,b,a,s], [g,b,a,s], [d,a,s], [b,s]]	{s, a, b, c}
5	[e,c,b,a,s]	[[g,b,a,s], [d,a,s], [b,s]]	[[g,b,a,s], [d,a,s], [b,s]]	{s, a, b, c, e}
6	[g,b,a,s]	Objetivo: [g,b,a,s]	Objetivo: [g,b,a,s]	{s, a, b, c, e, g}



Exemplo BFS

Passo	Nó Extraído [U C]	BFS Árvore (Fronteira)	BFS Grafo (Fronteira)	BFS Grafo (Fechados)
0	(Início)	[[s]]	[[s]]	{ }
1	[s]	[[a,s], [b,s]]	[[a,s], [b,s]]	{s}
2	[a,s]	[[b,s], [b,a,s], [d,a,s]]	[[b,s], [b,a,s], [d,a,s]]	{s, a}
3	[b,s]	[[b,a,s], [d,a,s], [a,b,s], [c,b,s], [g,b,s]]	[[b,a,s], [d,a,s], [c,b,s], [g,b,s]] (nó a já estava fechado, [a,b,s] foi ignorado)	{s, a, b}
4	[b,a,s]	[[d,a,s], [a,b,s], [c,b,s], [g,b,s], [c,b,a,s], [g,b,a,s]]	[[d,a,s], [c,b,s], [g,b,s]] (nó b já estava fechado, foi ignorado)	{s, a, b}
5	[d,a,s]	[[a,b,s], [c,b,s], [g,b,s], [c,b,a,s], [g,b,a,s], [g,d,a,s]]	[[c,b,s], [g,b,s], [g,d,a,s]]	{s, a, b, d}
6	Árv: [a,b,s] Grafo: [c,b,s]	[[c,b,s], [g,b,s], [c,b,a,s], [g,b,a,s], [g,d,a,s], [d,a,b,s]]	[[g,b,s], [g,d,a,s], [e,c,b,s]]	{s, a, b, d, c}
7	Árv: [c,b,s] Grafo: [g,b,s]	[[g,b,s], [c,b,a,s], [g,b,a,s], [g,d,a,s], [d,a,b,s], [e,c,b,s]]	Objetivo: [g,b,s]	{s, a, b, d, c, g}
8	Árv: [g,b,s] Grafo: -	Objetivo: [g,b,s]	-	-



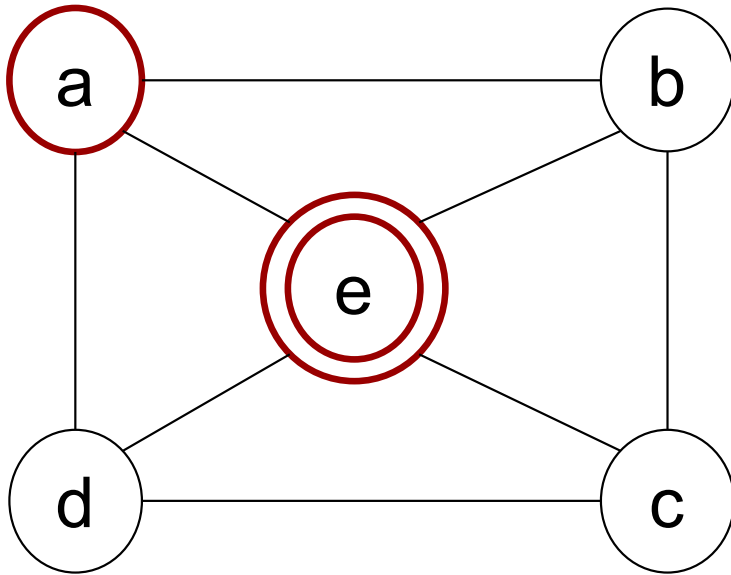
Algoritmos e Estruturas da Fronteira e Fechados

Algoritmo	Fronteira	Fechados
Profundidade (DFS)	Pilha (LIFO)	Conjunto simples
Profundidade Limitada (DLS)	Pilha Limitada	Dicionário (Nó, Profundidade do Nó). Reabre (analisa novamente) o nó se achar um caminho mais curto.
Profundidade Iterativa (IDS)	Pilha Limitada	Dicionário (Nó, Profundidade do Nó). Reseta-se a lista a cada nova iteração (novo limite).
Largura (BFS)	Fila (FIFO)	Conjunto simples
Custo Uniforme (UCS)	Fila de Prioridade ($g(n)$)	Conjunto simples
Bidirecional (BDS)	2 Filas (Geralmente FIFO)	2 Conjuntos Simples (Hash Sets). Usados não só para podar, mas para detectar a interseção das duas buscas.
A* (Heurística Consistente/Monotônica)	Fila de Prioridade ($f(n)$)	Conjunto simples
A* (Heurística Admissível, mas Inconsistente)	Fila de Prioridade ($f(n)$)	Dicionário (n , Custo $g(n)$). Reabre o nó se achar um custo menor.
IDA*	Pilha Limitada (pelo valor $f(n)$)	Geralmente não usa Lista Fechada. Usa Busca em Árvore pura para não estourar a RAM. Se usar, deve ser Dicionário (n , $f(n)$).
RBFS	Pilha de Recursão	Geralmente não usa. Segue a mesma lógica do IDA*, mantendo os limites na própria recursão em vez de uma lista fechada global.
Greedy Best-First	Fila de Prioridade ($h(n)$)	Conjunto simples
Beam Search (Busca em Feixe)	Fila Truncada (mantém apenas os k melhores)	Geralmente não usa. Para economizar RAM, costuma apenas avançar de nível. Se usar para evitar que os feixes explorem a mesma coisa, é um Conjunto Simples.
Hill-Climbing (Subida de Encosta)	Não possui fronteira real. Mantém apenas 1 nó (o atual) na fronteira.	Não usa. Ele só olha para frente. Se você acoplar uma Lista Fechada para evitar voltar a nós recentes, o algoritmo muda de nome e vira Tabu Search.
Têmpera Simulada	Não (Idem ao HC Puro)	Não usa (Pode visitar o mesmo nó)

O Impacto da Lista de Fechados (Closed List) na Complexidade

Algoritmo	Impacto no Tempo	Impacto no Espaço
Profundidade (DFS)	+ MELHORA: Cai de $O(b^m)$ para $O(V)$. Evita caminhos redundantes	⚠ PIORA: Passa de Linear $O(bm)$ para Exponencial $O(b^m)$ ou $O(V)$. Perde sua vantagem teórica
Prof. Limitada (DLS)	+ MELHORA: Evita caminhos redundantes dentro do limite de profundidade l	⚠ PIORA: Passa de Linear $O(bl)$ para Exponencial $O(b^l)$
Prof. Iterativa (IDS)	+ MELHORA: Evita caminhos repetidos durante cada iteração.	⚠ PIORA: Passa de Linear $O(bd)$ para Exponencial $O(b^d)$
Largura (BFS)	+ MELHORA: Reduz o tempo prático cortando caminhos redundantes	= NÃO MUDA: Já era exponencial por causa da Fronteira. Adicionar Fechados mantém a mesma complexidade
Custo Uniforme (UCS)	+ MELHORA: Evita a re-expansão inútil de estados que já foram alcançados por um custo menor	= NÃO MUDA: Continua Exponencial
Bidirecional (BDS)	+ MELHORA: Impede que as duas frentes de busca andem em círculos antes de se encontrarem	= NÃO MUDA: Continua Exponencial
A* (Consistente)	+ MELHORA: Corta redundâncias e nunca precisa reabrir um nó	= NÃO MUDA: Continua Exponencial
A* (Inconsistente)	⚠ PIORA: O tempo pode degenerar para $O(2^{ V })$ no pior caso, pois a inconsistência obriga a reabrir nós Fechados várias vezes	= NÃO MUDA: Continua Exponencial
IDA*	+ MELHORA: Evita caminhos repetidos, mas o custo em RAM inviabiliza o uso prático	⚠ PIORA: Passa de Linear para Exponencial
RBFS	+ MELHORA: Mesma situação do IDA*. Evita recálculos, mas mata a economia de RAM	⚠ PIORA: Passa de Linear para Exponencial
Greedy Best-First	+ MELHORA: Sem fechados, o tempo pode ser ∞ (fica preso em loops). Com fechados, limita-se a $O(V)$	= NÃO MUDA: Continua Exponencial
Beam Search	+ MELHORA: Evita que os k feixes fiquem andando em círculos nos mesmos platôs	⚠ PIORA: Deixa de ser estritamente limitado $O(k)$ e passa a crescer linearmente para manter o histórico global
Hill-Climbing	+ MELHORA: Impede que o algoritmo trave oscilando eternamente entre dois nós iguais	⚠ PIORA: O espaço deixa de ser $O(1)$ e passa a ser $O(m)$ ou maior. (Vira uma Busca Tabu)

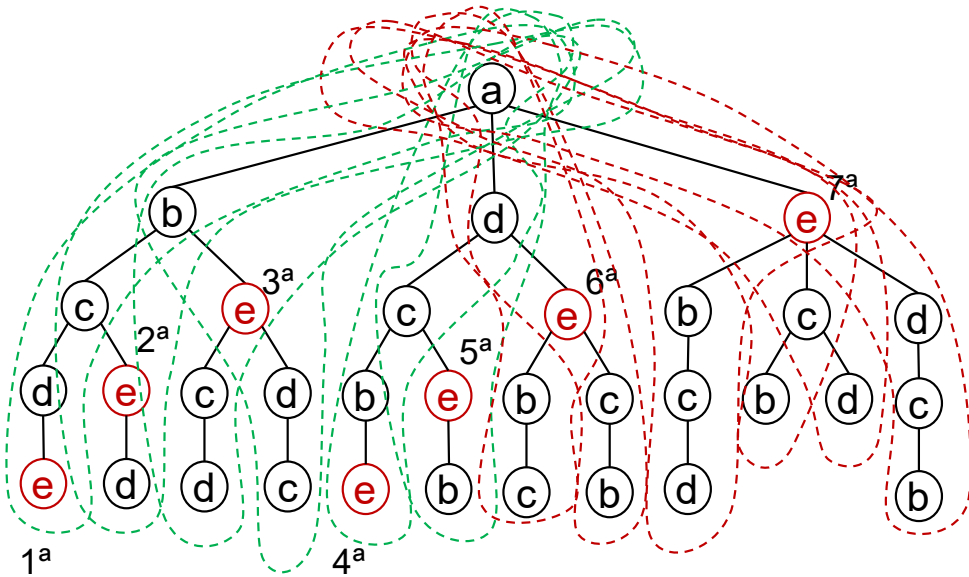
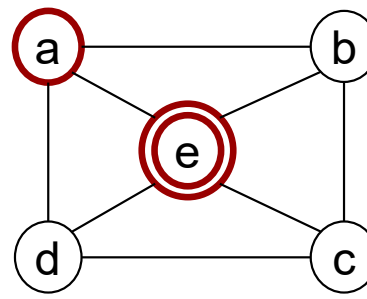
Exercício



Assuma que os sucessores de um nó são definidos em ordem alfabética

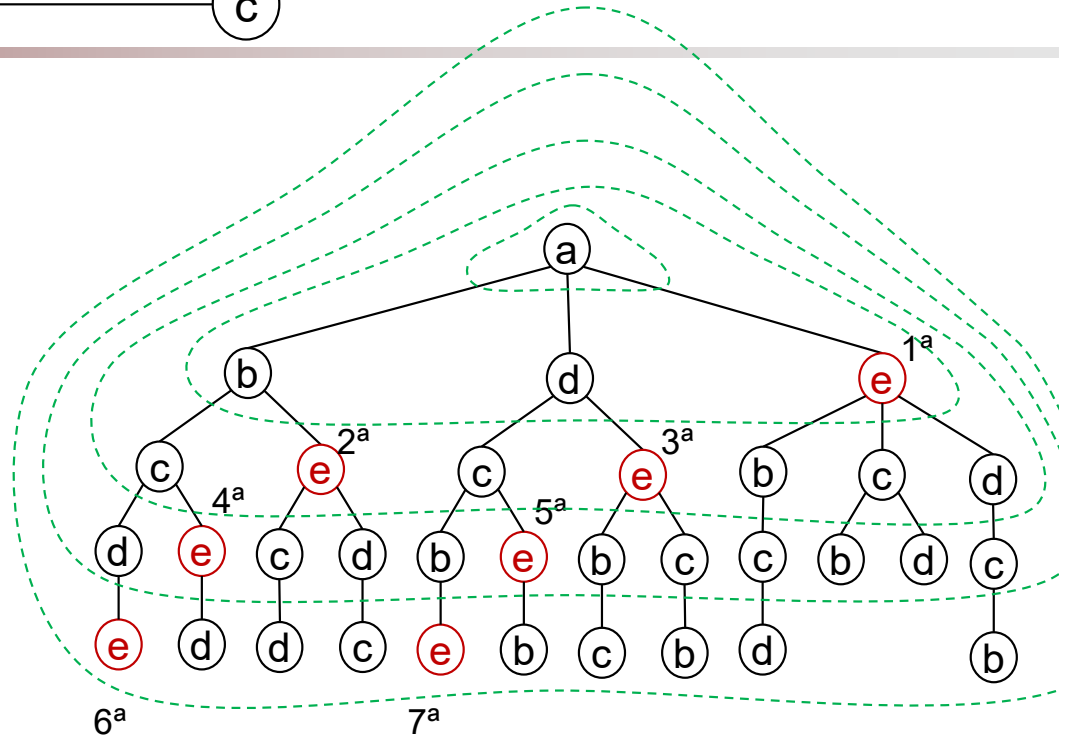
- ❑ Mostre a árvore de busca definida pelo algoritmo de busca em profundidade, partindo de **a** e chegando até **e**
- ❑ Mostre também os caminhos encontrados na ordem em que são encontrados pela
 - busca em profundidade
 - busca em largura

Solução



Busca em Profundidade (caminhos):

[e, d, c, b, a] ;
 [e, c, b, a] ;
 [e, b, a] ;
 [e, b, c, d, a] ;
 [e, c, d, a] ;
 [e, d, a] ;
 [e, a] ;



Busca em Largura (caminhos):

[e, a] ;
 [e, b, a] ;
 [e, d, a] ;
 [e, c, b, a] ;
 [e, c, d, a] ;
 [e, d, c, b, a] ;
 [e, b, c, d, a] ;

Resumo

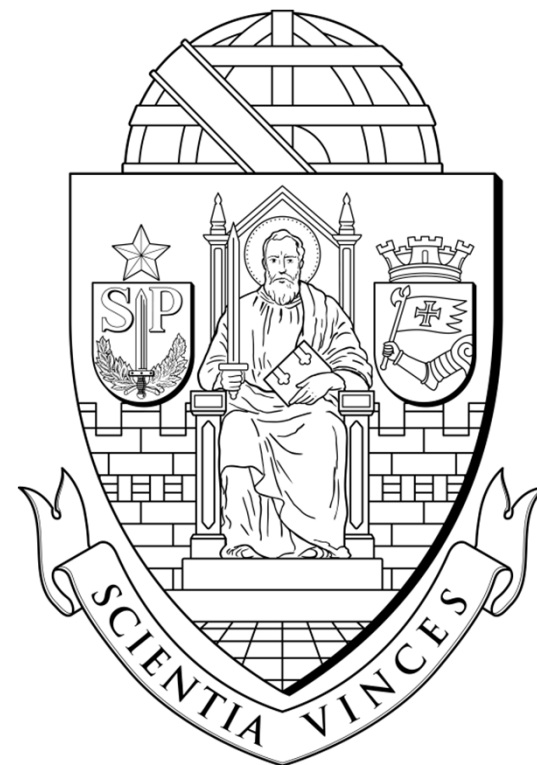
- ❑ Na estratégias de Busca Não Informada (Cega), exploramos o espaço de estados usando apenas a topologia do problema
- ❑ A escolha ideal depende de um equilíbrio entre Tempo, Espaço e a Estrutura da Fronteira
- ❑ A Gestão da Fronteira dita o comportamento
 - Pilha (LIFO - Memória Linear): DFS, DLS e IDS priorizam o mergulho profundo. Economizam muita memória $O(bm)$, mas exigem limites estritos para não caírem em loops infinitos
 - Fila e Prioridade (Memória Exponencial): BFS (FIFO), UCS (Custo) e BDS (Bidirecional) varrem o espaço em anéis (contornos). Garantem a solução ótima, mas o custo é a explosão exponencial da memória na Fronteira $O(b^d)$
- ❑ O Paradoxo da Lista de Fechados
 - Memorizar o passado acelera o Tempo (evitando redundâncias em BFS, UCS e BDS).
 - Porém, destrói o Espaço ao aplicar Fechados na família DFS/IDS anula sua única vantagem teórica, transformando sua memória linear em exponencial
- ❑ A questão que surge é como resolver o problema da explosão de memória sem perder a otimização
 - Na próxima apresentação, introduziremos as heurísticas: Algoritmos de Busca Informada que usam "dicas" (informações) do domínio para focar a exploração

*Copyright© Apresentação 2026 por
José Augusto Baranauskas
Universidade de São Paulo*

*Professores são convidados a utilizarem esta
apresentação da maneira que lhes for conveniente para
uso não comercial, desde que esta nota de copyright
permaneça intacta.*

Slides baseados nos livros:

- ❑ *Bratko, I.; Prolog Programming for Artificial Intelligence, 3rd Edition, Pearson Education, 2001.*
- ❑ *Clocksin, W.F.; Mellish, C.S.; Programming in Prolog, 5th Edition, Springer-Verlag, 2003.*



USP